F017 – Programming - Group Assignment

Group < 24 >

Name	ID	Contact Number
Tan Xuan Liang	20803374	+601162958925
Yap Kai Shawn	20804754	+60108330908
Ng Gene Lee	20802715	+601110208218
Jerone Teo Jay Roy	20804711	+60124519899
Quek Jia Wen	20811843	+60127202696

Table of Contents

Analysis – Requirement Analysis 3
- Problem Statement 3
- Target Users 3
- System Objectives 3
- Expected Outputs 3
- Assumptions 3
- Input-Process-Output Chart 4
- Algorithms 14

Design – System Design 33
- Flowchart 33

Coding – System Development 63
- Code 63
- Limitations 83

Testing – System Testing and Evaluation 84
- Test Data 84

Artificial Intelligence (AI) Usage Declaration 105

Analysis – Requirement Analysis

Problem Statement

Design an event management system that allows people to add, update, delete, manage attendees, and print the events.

System should check for clashing events at the same venue and time when adding new events.

System should provide appropriate error messages and validation checks throughout the program.

Target Users

A system designed for students, lecturers, staff to book event slots for activities and/or events.

System Objectives

- Store event data
- Add event
- Update event information
- Delete event
- Add, update and delete attendees
- Print events with their information
- Check for clashes in events' date, time and venue
- Check for clashes in attendees' date and time

Expected Outputs

- Show information of events
- Show attendees of each event
- Sorted list of events without clashing bookings

Assumptions

- There is no initial data
- Event date can be any time from the year 0 to year 9999
- Only allow same day bookings
- No maximum or minimum time limit for each event
- No location format, is case sensitive
- Events can have no attendees

Input-Process-Output Charts

- Main Code ---- 5
- Add Events ---- 5
- Update Events ---- 6
- Insert Events ---- 9
- Delete Events ---- 10
- Output Events ---- 10
- Attendee Actions ---- 11
- Others ---- 13
    - Check Clashes ---- 13

Input-Process-Output Chart (Main Code)

Input	Process	Output
READFILE “Events_Information.txt”	Add events Update events Delete events Manage attendees Display events Check for clashing events at the same venue and time Validate inputs	WRITEFILE “Events_Information.txt” Error messages

Input-Process-Output Chart (Add Events)

FUNCTION AddEvent()

Input	Process	Output
Name Date Time Venue Description	Validate input Check for any clashes in terms of date, venue and time	Information is stored

Input-Process-Output Chart (Update Events)

FUNCTION UpdateEvent()

Input	Process	Output
User's choice (Y/N).	Determine whether to proceed on updating an event	Continue the update module or exit
Event list.	Check whether any events are available	Print event list or stating that there are no events.
Enter event number to update or B for back.	Check whether user input is B. Check whether user input is numeric.	If user input is B, exit. If user input is nonnumeric, print error message. Otherwise, print selected events for updating.
Enter options to update from (1-7).	Identify the event attribute to be updated.	Call the corresponding update functions.
New information to update (name, date, venue, time and description).	Validate the updated information and check for clashes where applicable. Update the event details if there are no clashes.	Restore the original date, time, and venue if there are clashes with other events. If updating event details are successful, print "update successful."
User continuation choice (Y/N).	Determine whether the user wants to continue updating the same event, another event or exit.	Continue updating the same event, return to the Event name list or terminate the update process.

FUNCTION UpdateEventTime()

Input	Process	Output
Selected Event	Store the original start time and end time	Original time saved and call InsertEventTime()
New event time	Check whether the user input is valid and call CheckClashes() to check whether there are any clashes with other events.	Restore original time if there are clashes. Retry updating if user input is invalid. Time updated successfully and return true
User's response on whether to re-enter the time (Y/N).	Determine whether the user wants to re-enter the time.	Retry updating or return False.

FUNCTION UpdateEventDate()

Input	Process	Output
Selected Event	Store the original month, year and day	Original date saved and call InsertEventDate()
New event date	Check whether the user input is valid and call CheckClashes() to check whether there are any clashes with other events.	Restore original date if there are clashes. Retry updating if user input is invalid. Date updated successfully and return true
User's response on whether to re-enter the date (Y/N).	Determine whether the user wants to re-enter the time.	Retry updating or return False.

FUNCTION UpdateEventVenue()

Input	Process	Output
Selected Event	Store the original venue.	Original venue saved and call InsertEventVenue()
New event venue	Call CheckClashes() to check whether there are any clashes with other events.	Restore original venue if there are clashes. Venue updated successfully and return true
User's response on whether to re-enter the venue (Y/N).	Determine whether the user wants to re-enter the time.	Retry updating or return False.

FUNCTION AskContinueUpdating()

Input	Process	Output
User's choice on continue updating the same event (Y/N).	Determine whether the user want to continue updating the same event and a valid response	Return "same". Otherwise, print error message.
User's choice on updating another event.	Determine whether the user want to update another event and a valid response.	Return "another" or "quit". Otherwise print error meesage.

Input-Process-Output Chart (Insert Events)

FUNCTION InsertEventName()

Input	Process	Output
The name of the event	Validate input	Store event name

FUNCTION InsertEventDate()

Input	Process	Output
The date of the event	Validate input	Store event date

FUNCTION InsertEventTime()

Input	Process	Output
The time of the event	Validate input	Store event time

FUNCTION InsertEventVenue()

Input	Process	Output
The venue of the event	Validate input	Store event venue

FUNCTION InsertEventDescription()

Input	Process	Output
The description of the event	Validate input	Store event description

FUNCTION InsertEventInOrder()

Input	Process	Output
Index of event to be inserted	Move information of Index to make the list in order in terms of date and time	Ordered list by date and time

FUNCTION InsertAndRemove()

Input	Process	Output
InsertIndex RemoveIndex	Move information from RemoveIndex to InsertIndex Place information to temporary variables Remove information at RemoveIndex Insert temporary variables into InsertIndex	Information is moved

Input-Process-Output Chart (Delete Events)

FUNCTION DeleteEvent()

Input	Process	Output
None	Get index of event to be deleted Remove information of event	Lists without user deleted event

FUNCTION RemoveEvent()

Input	Process	Output
Index of event to be removed	Remove all information of event indexed	Lists without removed event

Input-Process-Output Chart (Output Events)

FUNCTION Output()

Input	Process	Output
None	Format information	OUTPUT all events' information

Input-Process-Output Chart (Attendee Actions)

FUNCTION ChooseEvent()

Input	Process	Output
NumberEvent, EventName	Use Output() to display events. Loop to get input Validate Digit	Integer Index

FUNCTION ChooseAttendee()

Input	Process	Output
AttendeeID	Use ShowAttendee() to display attendees Validate digit	IDChoice

FUNCTION CheckClashAttendee()

Input	Process	Output
ID, EventIndex, EventID, EventDateDay/Month/Year, EventStartTime, EventEndTime	Loop Events Check if there are clashing dates	True or False Clash and Duplicate Message

FUNCTION RegisterAttendee()

Input	Process	Output
Name, Details, ID	Determine new and existing user IF New: Validate name/ details IF Existing: Validate ID, reject or register	Updated Confirmation or Error Messages

FUNCTION UpdateAttendee()

Input	Process	Output
IDChoice, AttendeeID, AttendeeName, AttendeeDetails	Check attendees exist. Get attendee ID. Loop until valid option (1=name, 2=details) Get new value Update corresponding list	Updated AttendeeName or AttendeeDetails Confirmation or Error Messages

FUNCTION DeleteAttendee()

Input	Process	Output
IDChoice, AttendeeID, AttendeeName, AttendeeDetails, NumberAttendee	Check attendees exist. Remove ID from every event Delete attendee record OR List attendee's events, Get event index, Remove ID from the event	Updated EventID, NumberAttendee OR AttendeeID/ AttendeeName/ AttendeeDetails Confirmation or Error Messages

FUNCTION ShowAttendee()

Input	Process	Output
AttendeeID, AttendeeName, AttendeeDetails, EventID, EventName	Check attendees exist. For each attendee, scan all events Build a list of event names they're registered in	Printed table of ID, Name, Details, and Registered Events

FUNCTION AttendeeAction()

Input	Process	Output
EventName User Choice (1-5)	Check events exist. Loop: Display menu, Get choice, Dispatch to: RegisterAttendee/ UpdateAttendee/ DeleteAttendee/ ShowAttendee or exit on "5"	No return value Error Messages if any

Input-Process-Output Chart (Check Clashes)

FUNCTION CheckClashes()

Input	Process	Output
Index to be checked	Check for any clashes in terms of date, venue and time	Return TRUE if there are clashes

Algorithms

- Main Code ---------- 15
- Add Events ---------- 16
- Update Events ---------- 17
- Insert Events ---------- 21
- Delete Events ---------- 25
- Output Events ---------- 26
- Attendee Actions ---------- 27
- Others ---------- 32
    - Check Clashes ---------- 32

Algorithms (Main Code)

Step 1: OPENFILE “Events_Information.txt” FOR READ
Step 2: READFILE “Events_Information.txt”
Step 3: PRINT Menu
Step 4: INPUT “Action”
    Step 5a: IF (Add Event)
        Step 5b: THEN CALL FUNCTION AddEvent()
        Step 5c: GOTO Step 3
    Step 5d: ELSE IF (Update Event)
        Step 5e: THEN CALL FUNCTION UpdateEvent()
        Step 5f: GOTO Step 3
    Step 5g: ELSE IF (Delete Event)
        Step 5h: THEN CALL FUNCTION DeleteEvent()
        Step 5i: GOTO Step 3
    Step 5j: ELSE IF Action == “4”
        Step 5k: THEN CALL FUNCTION AttendeeAction()
        Step 5l: GOTO Step 3
    Step 5m: ELSE IF Action == “5”
        Step 5n: THEN CALL FUNCTION Output()
        Step 5o: GOTO Step 3
    Step 5p: ELSE IF Action == “6”
        Step 5q THEN GOTO Step 12
    Step 5r: ELSE
        Step 5s: PRINT Error Message
        Step 5t: ENDIF
Step 6: OPENFILE “Events_Information.txt” FOR WRITE
Step 7: WRITEFILE “Events_Information.txt”
Step 8: END

Algorithms (Add Events)

Algorithm for AddEvent():

Step 1: Start FUNCTION AddEvent()
Step 2: INPUT "Are you sure you want to add a new event?"
    Step 3a: IF (answer is yes)
        Step 3a.1: THEN CALL FUNCTION InsertEventName()
        Step 3a.2: CALL FUNCTION InsertEventDate()
        Step 3a.3: CALL FUNCTION InsertEventTime()
        Step 3a.4: CALL FUNCTION InsertEventVenue()
        Step 3a.5: CALL FUNCTION InsertEventDescription()
            Step 3a.6a: IF (number of events is not 1)
                Step 3a.6b.1: THEN CALL FUNCTION CheckClashes()
                    Step 3a.6b.2a: IF (there are clashes)
                        Step 3a.6b.2b: THEN CALL FUNCTION RemoveEvent()
                    Step 3a.6b.3a: ELSE
                        Step 3a.6b.3b: PRINT "Event added successfully."
                        Step 3a.6b.3c: CALL FUNCTION InsertEventInOrder()
                        Step 3a.6b.3d: ENDIF
            Step 3a.7a: ELSE
                Step 3a.7b: PRINT "Event added successfully."
                Step 3a.7c: ENDIF
    Step 3b.1: ELSE IF (answer is no)
        Step 3b.2: THEN PRINT "Event addition cancelled"
        Step 3b.3: RETURN
    Step 3c.1: ELSE
        Step 3c.2: PRINT "Invalid input"
        Step 3c.3: ENDIF
Step 4: END FUNCTION AddEvent()

Algorithms (Update Events)

Algorithm for UpdateEvent()

Step 1: Start FUNCTION UpdateEvent()
Step 2: Ask the user whether they want to update an event (Y/N).
Step 3a: IF the user enters N
Step 3b: THEN end the algorithm.
Step 4a: IF the user enters an invalid input
Step 4b: THEN PRINT an error message and repeat step 1.
Step 5a: IF the user enters Y
Step 5b: THEN count the total number of events.
Step 6a: IF the number of events is 0
Step 6b: THEN PRINT a message telling the user that there are no events available.
Step 6c: End the algorithm.
Step 7: Repeat until the user decided to stop updating the events
Step 7a: CALL FUNCTION Output().
Step 7b: Ask the user to select an event to update or press B to go back.
Step 7c: IF the user enters B
Step 7c.1: THEN end the algorithm.
Step 7d: IF the user input is not a numeric
Step 7d.1: THEN print an error message and repeat Step 7b.
Step 7e: IF the user enters a numeric number, is the selected number within range
Step 7e.1: IF the user's selected event number is outside of the range
Step 7.e.1.a: THEN print an error message and repeat Step 7b.
Step 7f: IF the user's selected event number is within the range
Step 7f.1: THEN print the menu:
1. Update Event Name
2. Update Event Date
3. Update Event Venue
4. Update Event Time
5. Update Event Description
6. Update Everything
7. Back
Step 7g: Ask the user to select an option (1-7).
Step 7h: IF the user's option is not numeric
Step 7h.1: THEN print an error message and repeat Step 7.g.
Step 7i: Perform the selected operations:
Step 7i.1: IF the user's option is 1
Step 7i.1.a: THEN CALL FUNCTION InsertEventName()
Step 7i.1.b: PRINT "Name updated."
Step 7i.2: IF the user's option is 2
Step 7i.2.a: THEN CALL FUNCTION UpdateEventDate()
Step 7i.2.b: IF the function returns true
Step 7i.2.b.1: THEN PRINT "Date updated."
Step 7i.2.c: IF the function returns false
Step 7i.2.c.1: THEN PRINT "Date not updated."
Step 7i.3: IF the user's option is 3

Step 7i.3.a: THEN CALL FUNCTION UpdateEventVenue()
Step 7i.3.b: IF the function returns true
Step 7i.2.3.1: THEN PRINT “Venue updated.”
Step 7i.3.c: IF the function returns false
Step 7i.2.3.1: THEN PRINT “Venue not updated.”
Step 7i.4: IF the user’s option is 4
Step 7i.4.a: THEN CALL FUNCTION UpdateEventTime()
Step 7i.4.b: IF the function returns true
Step 7i.2.b.1: THEN PRINT “Time updated.”
Step 7i.4.c: IF the function returns false
Step 7i.2.c.1: THEN PRINT “Time not updated.”
Step 7i.5: IF the user’s option is 5
Step 7i.5.a: THEN call InsertEventDescription()
Step 7i.5.b: PRINT “Description Updated”
Step 7i.6: IF the user’s option is 6
Step 7i.6.a: THEN CALL FUNCTION InsertEventName()
Step 7i.6.b: CALL FUNCTION UpdateEventDate()
Step 7i.6.c: CALL FUNCTION UpdateEventVenue()
Step 7i.6.d: CALL FUNCTION UpdateEventTime()
Step 7i.6.e: CALL FUNCTION InsertEventDescription()
Step 7i.6.f: IF the function for date, venue and time are true
Step 7i.6.f.1: THEN PRINT “Event updated.”
Step 7i.6.f.2: ELSE:
Step 7i.6.f.2.a: PRINT “Event updated with some issues.”
Step 7i.7: IF the user’s option is 7
Step 7i.7.a: Return to Step 7.2.
Step 7i.8: ELSE
Step 7i.8.a: PRINT error message and return to Step 7.f.
Step 7j: CALL FUNCTION AskContinueUpdating()
Step 7k.1: IF the function returns “same”
Step 7k.2: THEN return to Step 7g.
Step 7l.1: IF the function returns “another
Step 7l.2: THEN return to Step 7a.
Step 7m.1: IF the function returns “quit”
Step 7m.2: THEN end the algorithm:
Step 8: END FUNCTION UpdateEvent()

Algorithm for UpdateEventDate(choice)

Step 1: Start FUNCTION UpdateEventDate(choice)
Step 2: Store the chosen day, month and year.
Step 3: Repeat until the update is successful or the user chooses not to re-enter the date.
Step 3a: CALL FUNCTION InsertEventDate(choice) to enter a new date.
Step 3b.1: IF the new date clashes with other events date
Step 3b.2: THEN PRINT “There is clashing with other events.”
Step 3b.3: Restore the original day, month and year.
Step 3b.4: Repeat the following until a valid response is entered:

Step 3b.4.a: Ask the user whether they want to re-enter the date (Y/N).
Step 3b.4.b: IF the user enters Y
Step 3b.4.b.1: THEN return to Step 3a.
Step 3b.4.c: IF the user enters N
Step 3.b.3.c.1: THEN return False.
Step 3b.4.d: ELSE
Step 3b.4.d.1: PRINT an invalid message
Step 3c.1: If the new date does not clash with other events date
Step 3c.2: THEN return True.
Step 4: END FUNCTION UpdateEventDate(choice)

Algorithm for UpdateEventTime(choice)
Step 1: Start FUNCTION UpdateEventTime(choice)
Step 2: Store the chosen start time and end time.
Step 3: Repeat until the update is successful, or the user chooses not to re-enter the time:
Step 3a: CALL FUNCTION InsertEventTime(choice) to enter a new time.
Step 3b.1: IF the new time clashes with other events time
Step b.2: THEN PRINT "There is clashing with other events."
Step 3b.3: Restore the original start time and end time.
Step 3b.4: Repeat the following until a valid response is entered:
Step 3b.4.a: Ask the user whether they want to re-enter the time (Y/N).
Step b.4.b: IF the user enters Y
Step 3.b.4.b.1: THEN return to Step 3a.
Step 3.b.4.c: IF the user enters N
Step 3.b.4.c.1: THEN return False.
Step 3b.4.d: ELSE
Step 3b.4.d.1: PRINT an invalid message
Step 3c.1: If the new time does not clash with other events time
Step 3c.2: THEN return True.
Step 4: END FUNCTION UpdateEventTime(choice)

Algorithm for UpdateEventVenue(choice)
Step 1: Start FUNCTION UpdateEventVenue(choice)
Step 2: Store the chosen venue.
Step 3: Repeat until the update is successful, or the user chooses not to re-enter the venue:
Step 3a: CALL FUNCTION InsertEventVenue(choice) to enter a new venue.
Step 3b.1: IF the new venue clashes with other events venue
Step 3b.2: THEN PRINT "There is clashing with other events."
Step 3b.3: Restore the original venue.
Step 3b.4: Repeat the following until a valid response is entered:
Step 3b.4.a: Ask the user whether they want to re-enter the venue (Y/N).
Step 3b.4.b: IF the user enters Y
Step 3b.4.b.1: THEN return to Step 3a.
Step 3b.4.c: IF the user enters N
Step 3b.4.c.1: THEN return False.

Step 3b.3.d: ELSE

Step 3b.3.d.1: PRINT an invalid message

Step 3c: If the new venue does not clash with other events venue,

Step 3c.1: return True.

Step 4: END FUNCTION UpdateEventVenue(choice)

Algorithm for AskContinueUpdating()

Step 1: Start FUNCTION AskContinueUpdating()

Step 2: Repeat until a valid response is entered:

Step 2a: Ask the user whether they want to update the same event (Y/N).

Step 2b.1: If the user enters Y

Step 2b.: THEN return “same”.

Step 2c.1: If the user enters N

Step 2c.2: THEN repeat until a valid response is entered:

Step 2c.2.a: Ask the user whether they want to update another event (Y/N).

Step 2c.2.b: If user enters Y

Step 2c. 2.b.1: THEN return “another”.

Step 2c.2.c: If user enters N

Step 2c.2.c,1: THEN return “quit”.

Step 2c.2.d: ELSE

Step 2c.2.d.1: PRINT error message

Step 2c.2.d.2: GOTO Step 2c.1.a.

Step 2d.1: ELSE

Step 2d.2: PRINT error message and return to Step 2a.

Step 3: END FUNCTION AskContinueUpdating()

Algorithms (Insert Events)

Algorithm for InsertEventName(Index):

Step 1: Start FUNCTION InsertEventName(Index)
Step 2: TempEventName = INPUT “Enter event name”
    Step 3a.1: IF (name is not empty)
        Step 3a.2a: THEN IF (name is not repeating)
            Step 3a.2b: THEN EventName[Index] = TempEventName
        Step 3a.3a: ELSE (ask user if they want to use the same name)
            Step 3a.3b.1: IF (answer is yes)
                Step 3a.3b.2: THEN EventName[Index] = TempEventName
            Step 3a.3c.1: ELSE IF (answer is no)
                Step 3a.3c.2: PRINT “Please reenter the event name.”
                Step 3a.3c.3: GOTO Step 2
            Step 3a.3d.1: ELSE
                Step 3a.3d.2: PRINT “Invalid input.”
                Step 3a.3d.3: ENDIF
    Step 3b.1: ELSE
        Step 3b.2: PRINT “Event name cannot be empty”
        Step 3b.3: GOTO Step 2
        Step 3b.4: ENDIF
Step 4: END FUNCTION InsertEventName(Index)

Algorithm for InsertEventDate(Index):

Step 1: Start FUNCTION InsertEventDate(Index)
Step 2: TempEventDateYear = INPUT “Enter event year”
    Step 3a.1: IF (year is not empty)
        Step 3a.2a: IF (year is within 0 to 10000)
            Step 3a.2b.1: THEN IF (event year fulfilled leap year conditions)
                Step 3a.2b.2: LeapYear = TRUE
            Step 3a.2c.1: ELSE
                Step 3a.2c.2: LeapYear = FALSE
                Step 3a.2c.3: ENDIF
        Step 3a.3a: ELSE
            Step3a.3b: PRINT “Event year is invalid”
            Step 3a.3c: GOTO Step 2
            Step 3a.3d: ENDIF
    Step 3b.1: ELSE
        Step 3b.2: PRINT “Event year cannot be empty”
            Step 3b.3: GOTO Step 2
            Step 3b.4: ENDIF
Step 4: TempEventDateMonth = INPUT “Enter event month”
    Step 5a.1: IF (month is not empty)
        Step 5a.2a: IF (month entered is valid)
            Step 5a.2b: THEN (check if month have 31 days)
            Step 5a.2c.1: IF (month have 31 days)

Step 5a.2c.2: LongMonth = True
Step 5a.2d.1: ELSE
Step 5a.2d.2: LongMonth = False
Step 5a.2d.3: ENDIF
Step 5a.3a: ELSE
Step 5a.3b: PRINT “Event month is invalid”
Step 5a.3c: GOTO Step 4
Step 5a.3d: ENDIF
Step 5b.1: ELSE
Step 5b.2: PRINT “Event month cannot be empty”
Step 5b.3: GOTO Step 4
Step 5b.4: ENDIF
Step 6: TempEventDateDay = INPUT “Enter event day”
Step 7a.1: IF (day is not empty)
Step 7a.2a: IF (day entered is valid based on month and year)
Step 7a.2b: THEN ValidDate = TRUE
Step 7a.3a: ELSE
Step 7a.3b: PRINT “Event date does not exist”
Step 7a.3c: GOTO Step 6
Step 7a.3d: ENDIF
Step 7b.1: ELSE
Step 7b.2: PRINT “Event day cannot be empty”
Step 7b.3: GOTO Step 6
Step 7b.4: ENDIF
Step 8: PRINT (the event date)
Step 9: INPUT (Is the date correct?)
Step 10a.1: IF (answer is yes)
Step 10a.2: THEN EventDateYear[Index] = TempEventDateYear
Step 10a.3: EventDateMonth[Index] = TempEventDateMonth
Step 10a.4: EventDateDay[Index] = TempEventDateDay
Step 10b.1: ELSE IF (answer is no)
Step 10b.2: THEN PRINT “Please reenter the date”
Step 10b.3: GOTO Step 2
Step 10c.1: ELSE
Step 10c.2: PRINT “Invalid input.”
Step 10c.3: GOTO Step 9
Step 10c.4: ENDIF
Step 11: END FUNCTION InsertEventDate(Index)

Algorithm for InsertEventTime(Index):

Step 1: Start FUNCTION InsertEventTime(Index)
Step 2: TempEventStartTime = INPUT “Enter event start time”
Step 3a.1: IF (start time is not empty)
Step 3a.2a: THEN IF (start time is valid)
Step 3a.2b: THEN ValidStartTime = TRUE
Step 3a.3a: ELSE

Step 3a.3b: PRINT “Event start time is invalid”
Step 3a.3c: GOTO Step 2
Step 3a.3d: ENDIF
Step 3b.1: ELSE
Step 3b.2: PRINT “Event start time cannot be empty”
Step 3b.3: GOTO Step 2
Step 3b.4: ENDIF
Step 4: TempEventEndTime = INPUT “Enter event end time”
Step 5a.1: IF (end time is not empty)
Step 5a.2a: THEN IF (end time is valid)
Step 5a.2b.1: THEN IF (end time after start time)
Step 5a.b.2: THEN ValidEndTime = TRUE
Step 5a.2c.1: ELSE
Step 5a.2c.2: PRINT “Event end time cannot be earlier than event start time”
Step 5a.2c.3: GOTO Step 2
Step 5a.2c.4: ENDIF
Step 5a.3a: ELSE
Step 5a.3b: PRINT “Event end time is invalid”
Step 5a.3c: GOTO Step 4
Step 5a.3d: ENDIF
Step 5b.1: ELSE
Step 5b.2: PRINT “Event end time cannot be empty”
Step 5b.3: GOTO Step 4
Step 5b.4: ENDIF
Step 6: PRINT (the event time)
Step 7: INPUT “Is the time correct?”
Step 8a.1: IF (answer is yes)
Step 8a.2: THEN EventStartTime[Index] = TempEventStartTime
Step 8a.3: EventEndTime[Index] = TempEventEndTime
Step 8b.1: ELSE IF (answer is no)
Step 8b.2: THEN PRINT “Please reenter the time”
Step 8b.3: GOTO Step 2
Step 8c.1: ELSE
Step 8c.2: PRINT “Invalid input.”
Step 8c.3: GOTO Step 7
Step 8c.4: ENDIF
Step 9: END FUNCTION InsertEventTime(Index)

Algorithm for InsertEventVenue(Index):

Step 1: Start FUNCTION InsertEventVenue(Index)
Step 2: TempEventVenue = INPUT "Enter event venue: "
Step 3a.1: IF (venue is not empty)
Step 3a.2: THEN EventVenue[Index] == TempEventVenue
Step 3b1: ELSE
Step 3b.2: PRINT “Event venue cannot be empty”

Step 3b.3: GOTO Step 2
Step 3b.4: ENDIF
Step 4: END FUNCTION InsertEventVenue(Index)

Algorithm for InsertEventDescription(Index):
Step 1: Start FUNCTION InsertEventDescription(Index)
Step 2: TempEventDescription = INPUT "Enter event description (Optional): "
Step 3a.1: IF TempEventDescription NOT EMPTY
Step 3a.2: THEN EventDescription [Index] == TempEventDescription
Step 3b.1: ELSE
Step 3b.2: EventDescription [Index] == "-"
Step 3b.3: ENDIF
Step 4: END FUNCTION InsertEventDescription(Index)

Algorithm for InsertEventInOrder(Index):
Step 1: Start FUNCTION InsertEventInOrder()
Step 2: READ Index of event to be inserted into list
Step 3: Find where event of index's information should be inserted to make the lists sorted by date and time.
Step 4: CALL FUNCTION InsertAndRemove()
Step 5: END FUNCTION InsertEventInOrder()

Algorithm for InsertAndRemove(Insert, Remove):
Step 1: Start FUNCTION InsertAndRemove()
Step 2: READ Index of event to be removed
Step 3: READ Index of spot in list to be inserted
Step 4: Move the information to correct spot
Step 5: END FUNCTION InsertAndRemove()

Algorithms (Delete Events)

Algorithm for DeleteEvent():

Step 1: Start FUNCTION DeleteEvent()
    Step 2a: IF (There are no events) THEN
        Step 2b: GOTO Step 3
Step 3: INPUT “Which event to be deleted”
Step 4: CALL FUNCTION RemoveEvent()
Step 5: INPUT “Continue deleting events”
    Step 6a: IF (Continue deleting)
        Step 6b: THEN GOTO Step 5b
        Step 6c: ENDIF
    Step 7a: IF (NOT Continue deleting)
        Step 7b: THEN GOTO Step 3
        Step 7c: ENDIF
Step 8: END FUNCTION DeleteEvent()

Algorithm for RemoveEvent():

Step 1: Start FUNCTION RemoveEvent()
Step 2: READ Index
Step 3: POP Information of Index
Step 4: END FUNCTION RemoveEvent()

Algorithms (Output Events)

Algorithm for Output():

Step 1: Start FUNCTION Output()
    Step 2a.1: IF (EventName list is empty)
        Step 2a.2: THEN PRINT "No events found in the system."
        Step 2a.3: RETURN
    Step 2b.1: ELSE
        Step 2b.2: PRINT "EVENT MANAGEMENT SYSTEM - SCHEDULE" header
        Step 2b.3: ENDIF
    Step 3a: FOR Index = 0 TO Number of Events - 1
        Step 3b: PRINT Event Name, Date, Time, Venue, Description and Total Registered Attendees
            Step 3c.1: IF (There is attendee)
                Step 3c.2: THEN PRINT attendee table header
                Step 3c.3a: FOR ID IN ID List of Event[Index]
                    Step 3c.3b: PRINT ID, Attendee Name and Attendee Details
                    Step 3c.3c: NEXT ID
            Step 3c.3a: ELSE
                Step 3c.3b: PRINT "No attendees registered yet."
                Step 3c.3c: ENDIF
        Step 3d: NEXT Index
Step 6: END FUNCTION Output()

Algorithms (Attendee Actions)

Algorithm for ChooseEvent():

Step 1: Start FUNCTION ChooseEvent()
Step 2: CALL FUNCTION Output()
Step 3: REPEAT
    Step 3a: INPUT "Which event would you like to edit? (Enter Index): "
    Step 3b: IF (input is a digit)
        Step 3b.1: THEN CONVERT input to integer, SUBTRACT 1
        Step 3b.2: IF (converted index is within range of NumberEvent)
        Step 3b.2a: THEN SET ValidEvent = True
    Step 3b.3: ELSE
        Step 3b.3a: PRINT "Invalid index."
    Step 3c: ELSE
        Step 3c.1: PRINT "Input must be an integer."
Step 4: UNTIL (ValidEvent is True)
Step 5: RETURN EventChoice
Step 6: END FUNCTION ChooseEvent()

Algorithm for ChooseAttendee():

Step 1: Start FUNCTION ChooseAttendee(Prompt)
Step 2: REPEAT
    Step 2a: CALL FUNCTION ShowAttendee()
    Step 2b: INPUT Prompt
    Step 2c: IF (input is a digit)
        Step 2c.1: THEN CONVERT input to integer
        Step 2c.2: IF (integer exists in AttendeeID)
            Step 2c.2a: THEN SET ValidID = True
            Step 2c.2b: RETURN IDChoice
        Step 2c.3: ELSE
            Step 2c.3a: PRINT "Invalid index."
    Step 2d: ELSE
        Step 2d.1: PRINT "Input must be an integer."
Step 3: UNTIL (ValidID is True)
Step 4: END FUNCTION ChooseAttendee()

Algorithm for CheckClashAttendee():

Step 1: Start FUNCTION CheckClashAttendee(ID, EventIndex)
Step 2: FOR each event index IDIndex FROM 0 TO length of EventID - 1
    Step 2a: FOR each IDEvent in EventID[IDIndex]
        Step 2a.1: IF (IDEvent equals ID)
            Step 2a.1a: IF (EventIndex equals IDIndex)
                Step 2a.1a.1: THEN PRINT "Attendee is already registered."
                Step 2a.1a.2: RETURN True
            Step 2a.1b: IF (same date AND overlapping time between EventIndex and IDIndex)

Step 2a.1b.1: THEN PRINT "Clash event detected."
Step 2a.1b.2: RETURN True
Step 3: END FOR
Step 4: RETURN False
Step 5: END FUNCTION CheckClashAttendee()

Algorithm for RegisterAttendee():
Step 1: Start FUNCTION RegisterAttendee()
Step 2: SET Index = CALL FUNCTION ChooseEvent()
Step 3: SET NewUser = "K"
Step 4: REPEAT
Step 4a: IF (AttendeeID list is not empty)
Step 4a.1: THEN INPUT "Are you a new user? (Y/N): "
Step 4b: IF (NewUser equals "Y" OR AttendeeID list is empty)
Step 4b.1: THEN INPUT Attendee Name (repeat until non-empty)
Step 4b.2: APPEND NewName to AttendeeName
Step 4b.3: INPUT Attendee Details (default "-" if blank)
Step 4b.4: APPEND NewDetails to AttendeeDetails
Step 4b.5: APPEND AttendeeCounter to AttendeeID
Step 4b.6: PRINT new Attendee ID
Step 4b.7: ADD new ID to EventID[Index]
Step 4b.8: INCREMENT AttendeeCounter
Step 4b.9: INCREMENT NumberAttendee[Index]
Step 4b.10: PRINT "Attendee registered successfully."
Step 4b.11: BREAK
Step 4c: ELSE IF (NewUser equals "N")
Step 4c.1: REPEAT
Step 4c.1a: INPUT Attendee ID
Step 4c.1b: IF (input is a digit)
Step 4c.1b.1: THEN CONVERT to integer (OldID)
Step 4c.1b.2: IF (OldID exists in AttendeeID)
Step 4c.1b.2a: THEN PRINT matching Attendee Name
Step 4c.1b.2b: IF (CALL FUNCTION CheckClashAttendee(OldID, Index) is True)
Step 4c.1b.2b.1: THEN PRINT "Attendee is not added."
Step 4c.1b.2c: ELSE
Step 4c.1b.2c.1: ADD OldID to EventID[Index]
Step 4c.1b.2c.2: INCREMENT NumberAttendee[Index]
Step 4c.1b.2c.3: PRINT "Attendee registered successfully."
Step 4c.1b.2d: SET ValidID = True
Step 4c.1b.3: ELSE
Step 4c.1b.3a: PRINT "ID not found."
Step 4c.1c: ELSE
Step 4c.1c.1: PRINT "ID can only be integers."
Step 4c.2: UNTIL (ValidID is True)

Step 4d: ELSE
Step 4d.1: PRINT "Invalid Input. Please enter Y or N."
Step 5: UNTIL (NewUser equals "N" OR NewUser equals "Y")
Step 6: END FUNCTION RegisterAttendee()

Algorithm for UpdateAttendee():
Step 1: Start FUNCTION UpdateAttendee()
Step 2: IF (AttendeeID list is empty)
Step 2a: THEN PRINT "No attendees have been registered."
Step 2b: RETURN
Step 3: SET IDChoice = CALL FUNCTION ChooseAttendee("Attendee ID to be updated: ")
Step 4: REPEAT
Step 4a: INPUT "Update Attendee's 1 | Name 2 | Details"
Step 4b: IF (Option equals "1")
Step 4b.1: THEN INPUT new Name (repeat until non-empty)
Step 4b.2: UPDATE AttendeeName at matching index
Step 4b.3: PRINT "Name has been updated."
Step 4b.4: SET ValidOption = True
Step 4b.5: BREAK
Step 4c: ELSE IF (Option equals "2")
Step 4c.1: THEN INPUT new Details (default "-" if blank)
Step 4c.2: UPDATE AttendeeDetails at matching index
Step 4c.3: PRINT "Details have been updated."
Step 4c.4: SET ValidOption = True
Step 4c.5: BREAK
Step 4d: ELSE
Step 4d.1: PRINT "Invalid input."
Step 5: UNTIL (ValidOption is True)
Step 6: END FUNCTION UpdateAttendee()

Algorithm for DeleteAttendee():
Step 1: Start FUNCTION DeleteAttendee()
Step 2: IF (AttendeeID list is empty)
Step 2a: THEN PRINT "No attendees have been registered."
Step 2b: RETURN
Step 3: REPEAT
Step 3a: INPUT "Delete Attendee (1) or Delete Registration (2)?"
Step 3b: IF (DeleteChoice equals "1")
Step 3b.1: THEN SET IDChoice = CALL FUNCTION ChooseAttendee("Attendee ID to be deleted: ")
Step 3b.2: FOR each event containing IDChoice
Step 3b.2a: REMOVE IDChoice from EventID
Step 3b.2b: DECREMENT NumberAttendee at that event
Step 3b.3: REMOVE attendee's Name, Details, and ID records
Step 3b.4: SET DeletingAttendee = False

Step 3b.5: BREAK
Step 3c: ELSE IF (DeleteChoice equals "2")
Step 3c.1: THEN SET IDChoice = CALL FUNCTION ChooseAttendee("Attendee ID to be deleted from an event: ")
Step 3c.2: FOR each event containing IDChoice
Step 3c.2a: PRINT event details
Step 3c.2b: ADD event index to IndexList
Step 3c.3: REPEAT
Step 3c.3a: INPUT "Which event to remove attendee from? (Enter index): "
Step 3c.3b: IF (input is a digit)
Step 3c.3b.1: THEN CONVERT to integer (IndexChoice)
Step 3c.3b.2: IF (IndexChoice exists in IndexList)
Step 3c.3b.2a: THEN SET ValidIndex = True
Step 3c.3b.2b: BREAK
Step 3c.3b.3: ELSE
Step 3c.3b.3a: PRINT "Invalid index."
Step 3c.3c: ELSE
Step 3c.3c.1: PRINT "Input must be an integer."
Step 3c.4: UNTIL (ValidIndex is True)
Step 3c.5: PRINT removal confirmation message
Step 3c.6: REMOVE IDChoice from EventID[IndexChoice]
Step 3c.7: DECREMENT NumberAttendee[IndexChoice]
Step 3c.8: SET DeletingAttendee = False
Step 3c.9: BREAK
Step 3d: ELSE
Step 3d.1: PRINT "Input must be either \"1\" or \"2\""
Step 4: UNTIL (DeletingAttendee is False)
Step 5: END FUNCTION DeleteAttendee()

Algorithm for ShowAttendee():
Step 1: Start FUNCTION ShowAttendee()
Step 2: PRINT "Attendee List" header
Step 3: IF (AttendeeID list is empty)
Step 3a: THEN PRINT "There are no attendees."
Step 3b: RETURN
Step 4: PRINT column headers (ID, Name, Details, Events Registered)
Step 5: FOR each ID in AttendeeID
Step 5a: SET AttendeeEvents = ""
Step 5b: FOR each event containing ID
Step 5b.1: APPEND event name to AttendeeEvents
Step 5c: IF (AttendeeEvents is empty)
Step 5c.1: THEN SET AttendeeEvents = "-"
Step 5d: PRINT ID, Name, Details, AttendeeEvents
Step 6: END FOR
Step 7: END FUNCTION ShowAttendee()

Algorithm for AttendeeAction():

Step 1: Start FUNCTION AttendeeAction()
Step 2: IF (EventName list is empty)
    Step 2a: THEN PRINT "There are no events."
    Step 2b: RETURN
Step 3: REPEAT
    Step 3a: PRINT "Attendee Menu" (options 1-5)
    Step 3b: INPUT "Enter your choice: (1-5)"
    Step 3c: IF (Choice equals "1")
        Step 3c.1: THEN CALL FUNCTION RegisterAttendee()
    Step 3d: ELSE IF (Choice equals "2")
        Step 3d.1: THEN CALL FUNCTION UpdateAttendee()
    Step 3e: ELSE IF (Choice equals "3")
        Step 3e.1: THEN CALL FUNCTION DeleteAttendee()
    Step 3f: ELSE IF (Choice equals "4")
        Step 3f.1: THEN CALL FUNCTION ShowAttendee()
    Step 3g: ELSE IF (Choice equals "5")
        Step 3g.1: THEN SET Run = False
    Step 3h: ELSE
        Step 3h.1: PRINT "Invalid input."
Step 4: UNTIL (Run is False)
Step 5: END FUNCTION AttendeeAction()

Algorithms (Check Clashes)

Algorithm for CheckClashes():

Step 1: Start FUNCTION CheckClashes()

Step 2a: FOR Counter = 0 TO Number of Events

Step 2b: Check Whether there are any clashes

Step 2b.1: IF (there are clashes)

Step 2b.2: THEN RETURN TRUE

Step 2b.3: ELSE

Step 2b.4: RETURN FALSE

Step 2b.5: ENDIF

Step 2d: NEXT Counter

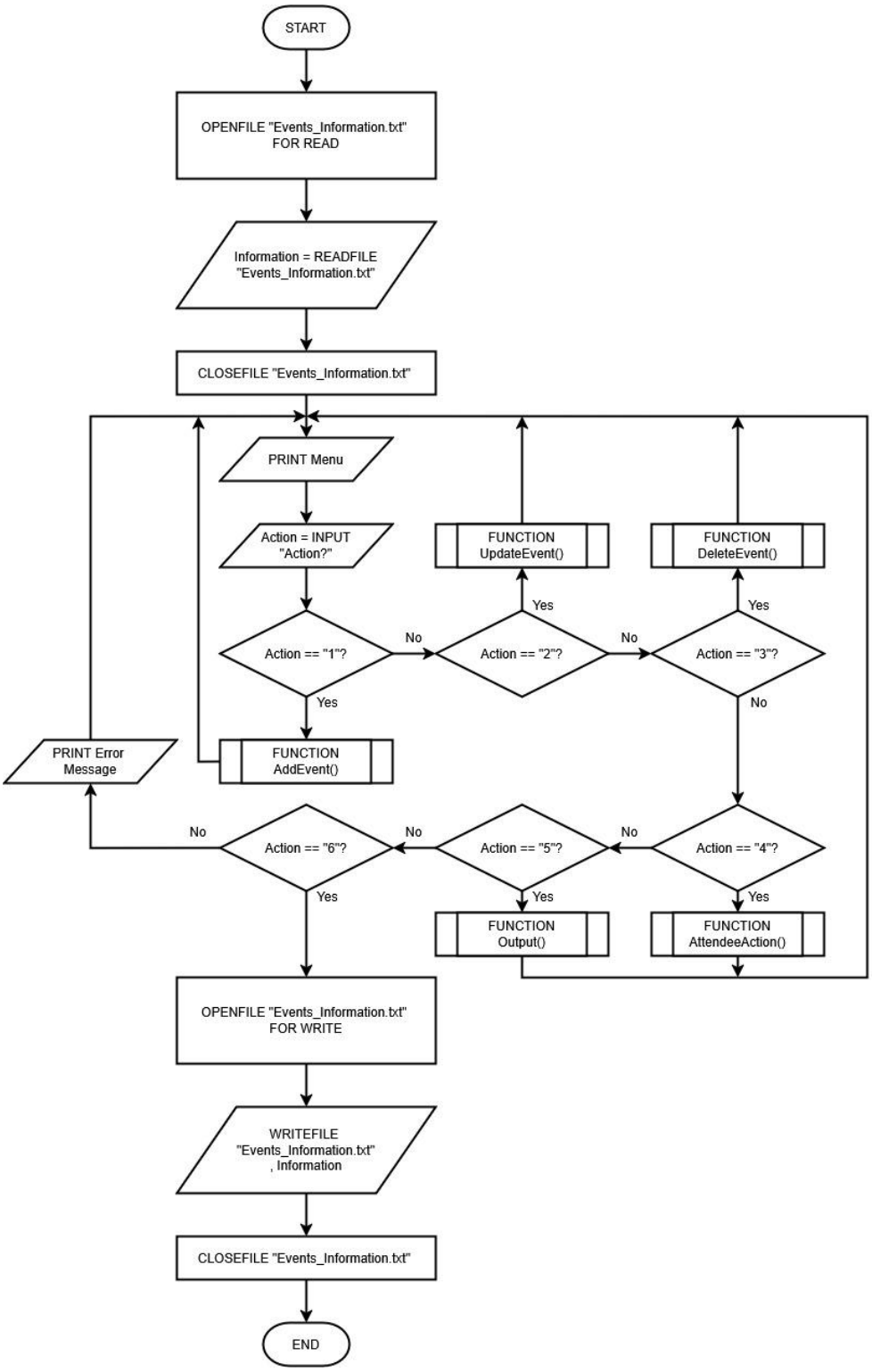
Step 3: END FUNCTION CheckClashes()

Design – System Design

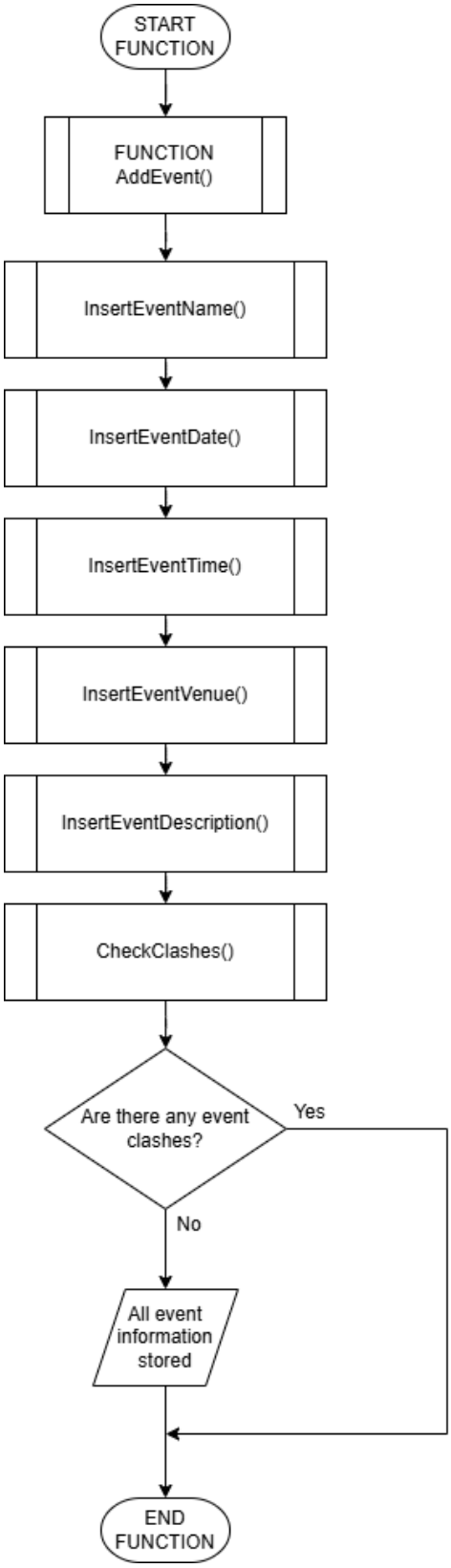
Flowchart

- Main Code ---- 34
- Add Events ---- 35
- Update Events ---- 37
- Insert Events ---- 43
- Delete Events ---- 51
- Output Events ---- 53
- Attendee Actions ---- 54
- Others ---- 62
    - Check Clashes ---- 62

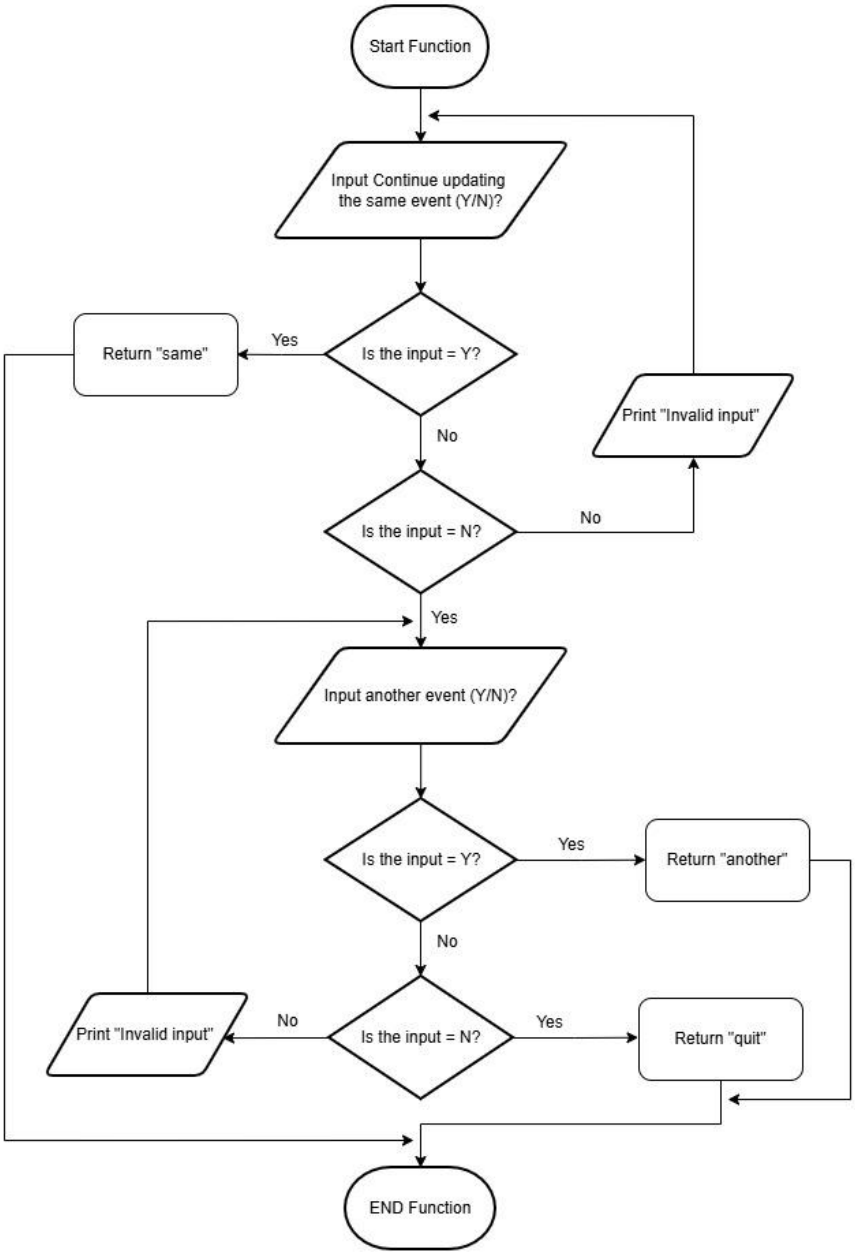
Flowchart


START
OPENFILE "Events_Information.txt" FOR READ
Information = READFILE "Events_Information.txt"
CLOSEFILE "Events_Information.txt"
PRINT Menu
Action = INPUT "Action?"
Action == "1"?
Yes
No
FUNCTION AddEvent()
Action == "2"?
Yes
No
FUNCTION UpdateEvent()
Action == "3"?
Yes
No
FUNCTION DeleteEvent()
Action == "4"?
Yes
No
FUNCTION AttendeeAction()
Action == "5"?
Yes
No
FUNCTION Output()
Action == "6"?
Yes
No
PRINT Error Message
OPENFILE "Events_Information.txt" FOR WRITE
WRITEFILE "Events_Information.txt" , Information
CLOSEFILE "Events_Information.txt"
END

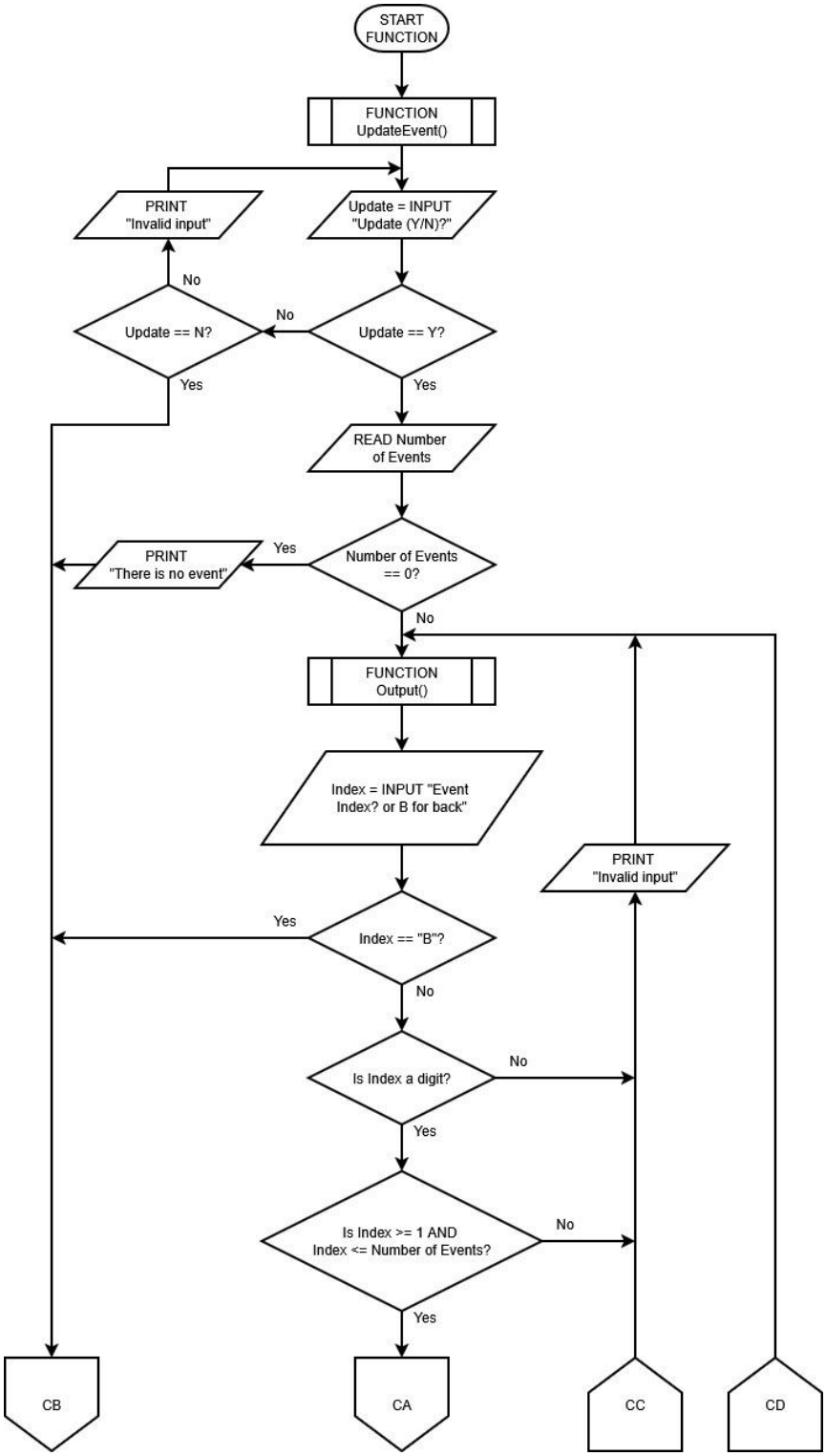
Flowchart for AddEvent()

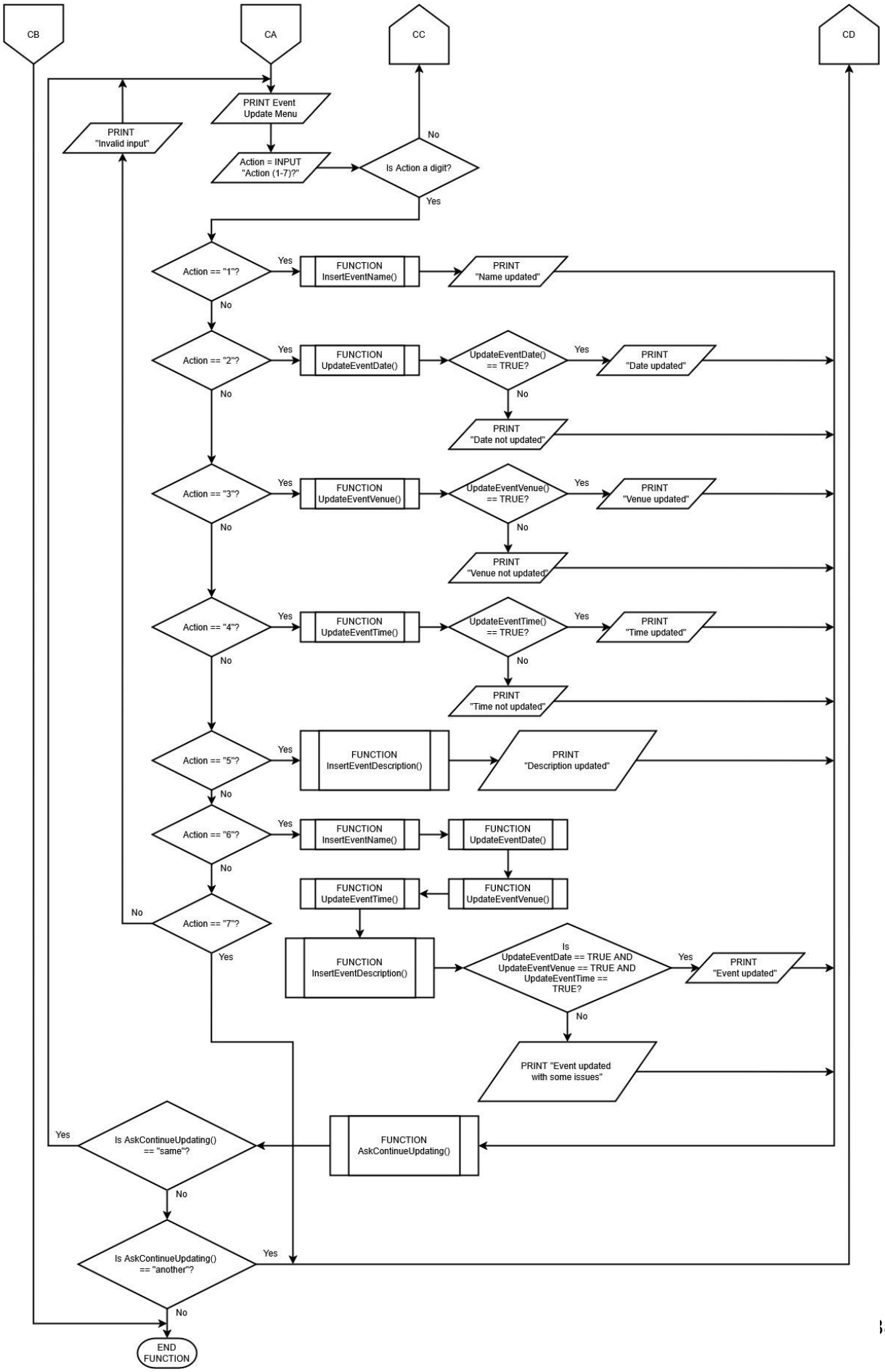


Flowchart for AskContinueUpdating()

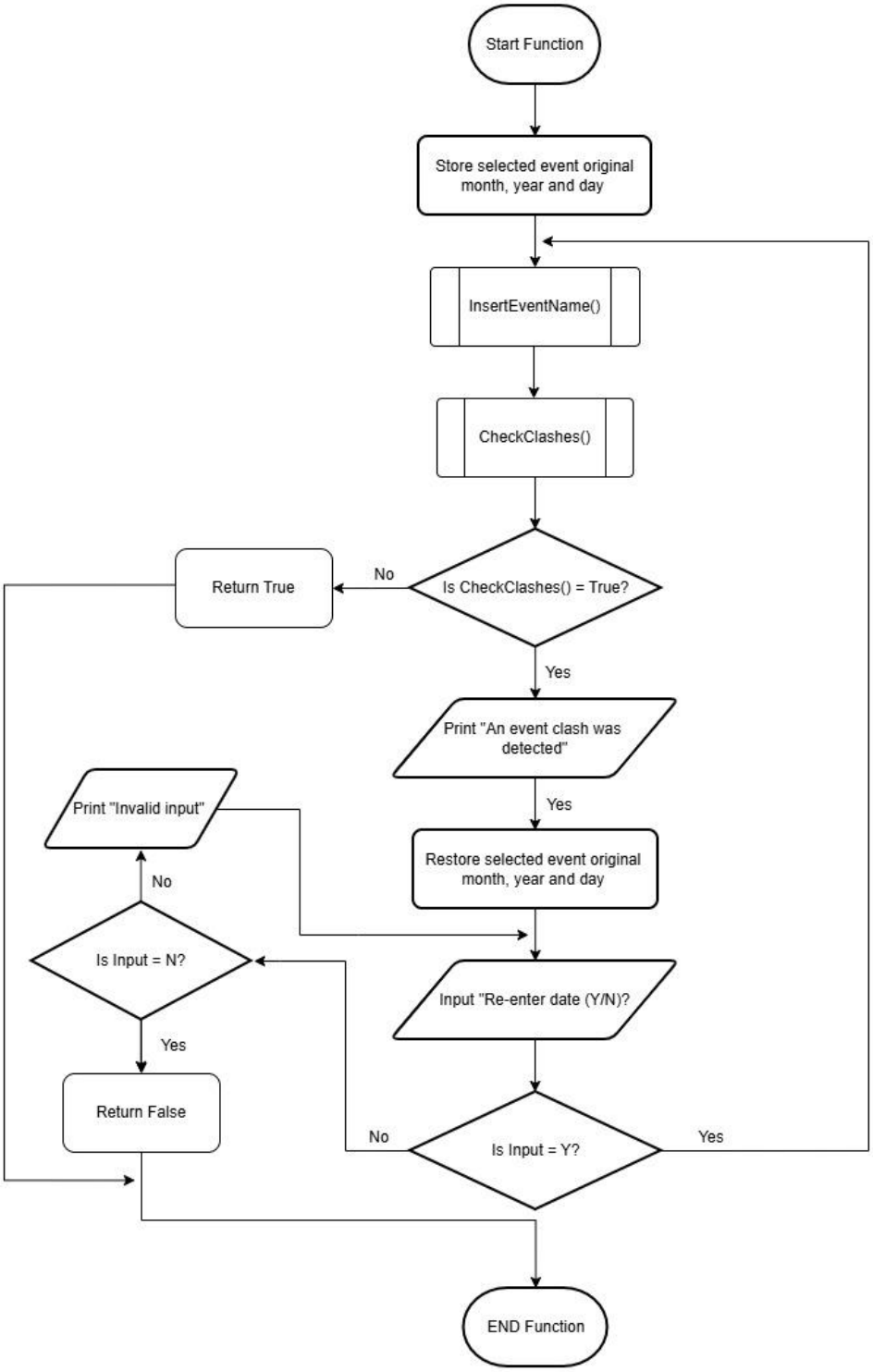


Flowchart for UpdateEvent()

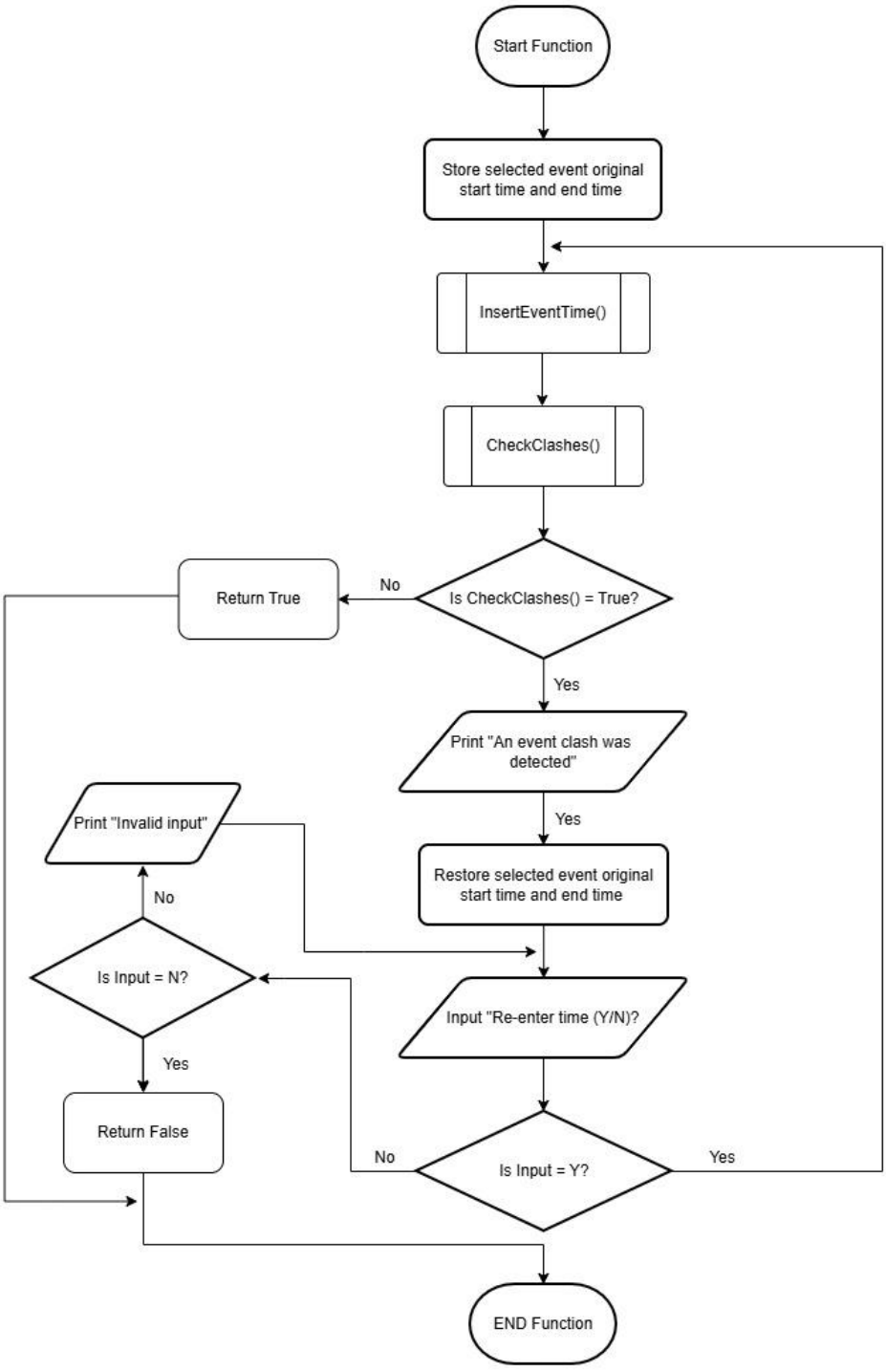



CB
CA
CC
CD
PRINT Event Update Menu
Action = INPUT "Action (1-7)?"
Is Action a digit?
No
Yes
PRINT "Invalid input"
Action == "1"?
FUNCTION InsertEventName()
PRINT "Name updated"
Action == "2"?
FUNCTION UpdateEventDate()
UpdateEventDate() == TRUE?
PRINT "Date updated"
PRINT "Date not updated"
Action == "3"?
FUNCTION UpdateEventVenue()
UpdateEventVenue() == TRUE?
PRINT "Venue updated"
PRINT "Venue not updated"
Action == "4"?
FUNCTION UpdateEventTime()
UpdateEventTime() == TRUE?
PRINT "Time updated"
PRINT "Time not updated"
Action == "5"?
FUNCTION InsertEventDescription()
PRINT "Description updated"
Action == "6"?
FUNCTION InsertEventName()
FUNCTION UpdateEventDate()
FUNCTION UpdateEventTime()
FUNCTION UpdateEventVenue()
Action == "7"?
FUNCTION InsertEventDescription()
Is UpdateEventDate == TRUE AND UpdateEventVenue == TRUE AND UpdateEventTime == TRUE?
PRINT "Event updated"
PRINT "Event updated with some issues"
FUNCTION AskContinueUpdating()
Is AskContinueUpdating() == "same"?
Is AskContinueUpdating() == "another"?
END FUNCTION

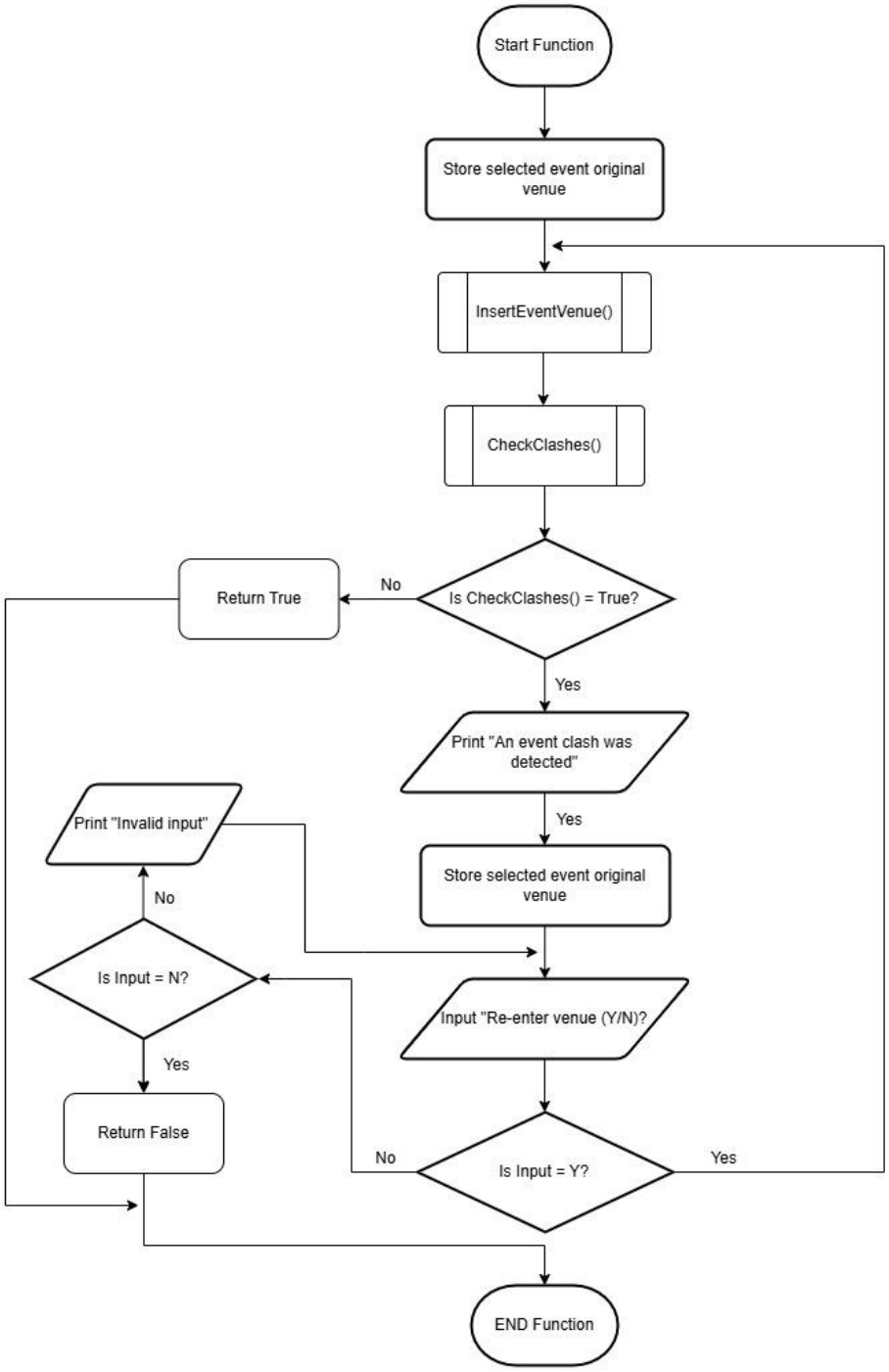
Flowchart for UpdateEventDate()



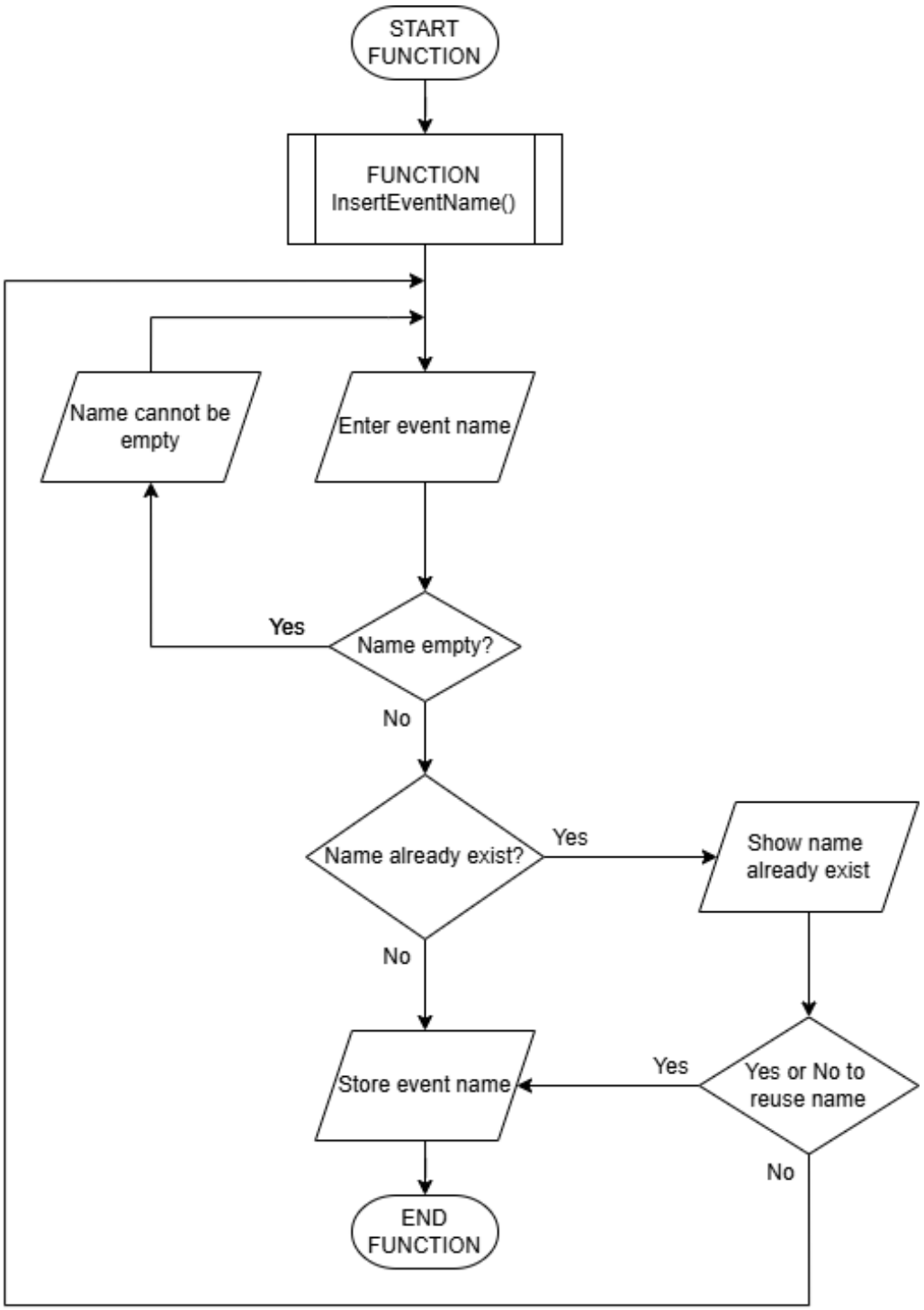
Flowchart for UpdateEventTime()



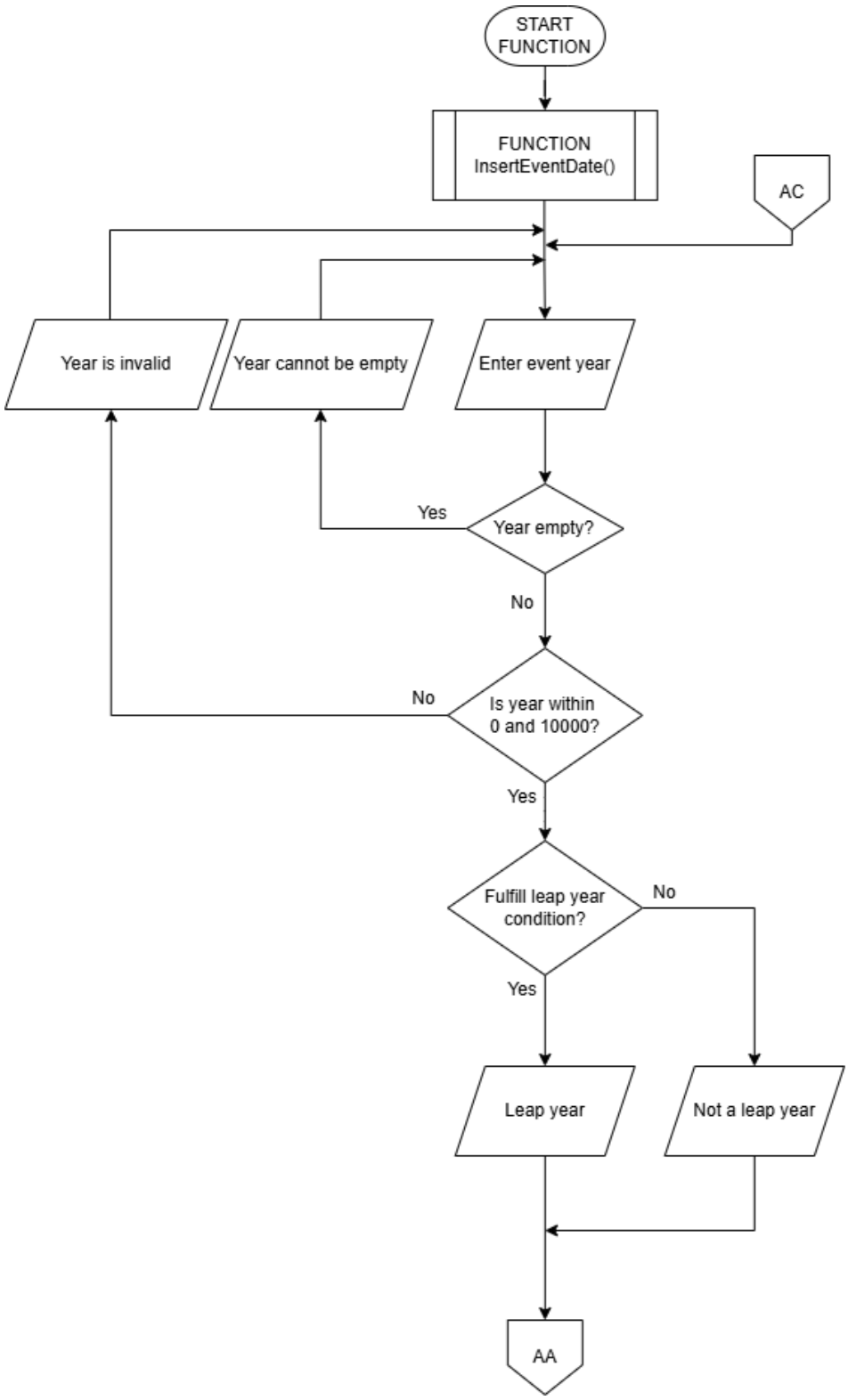
Flowchart for UpdateEventVenue()

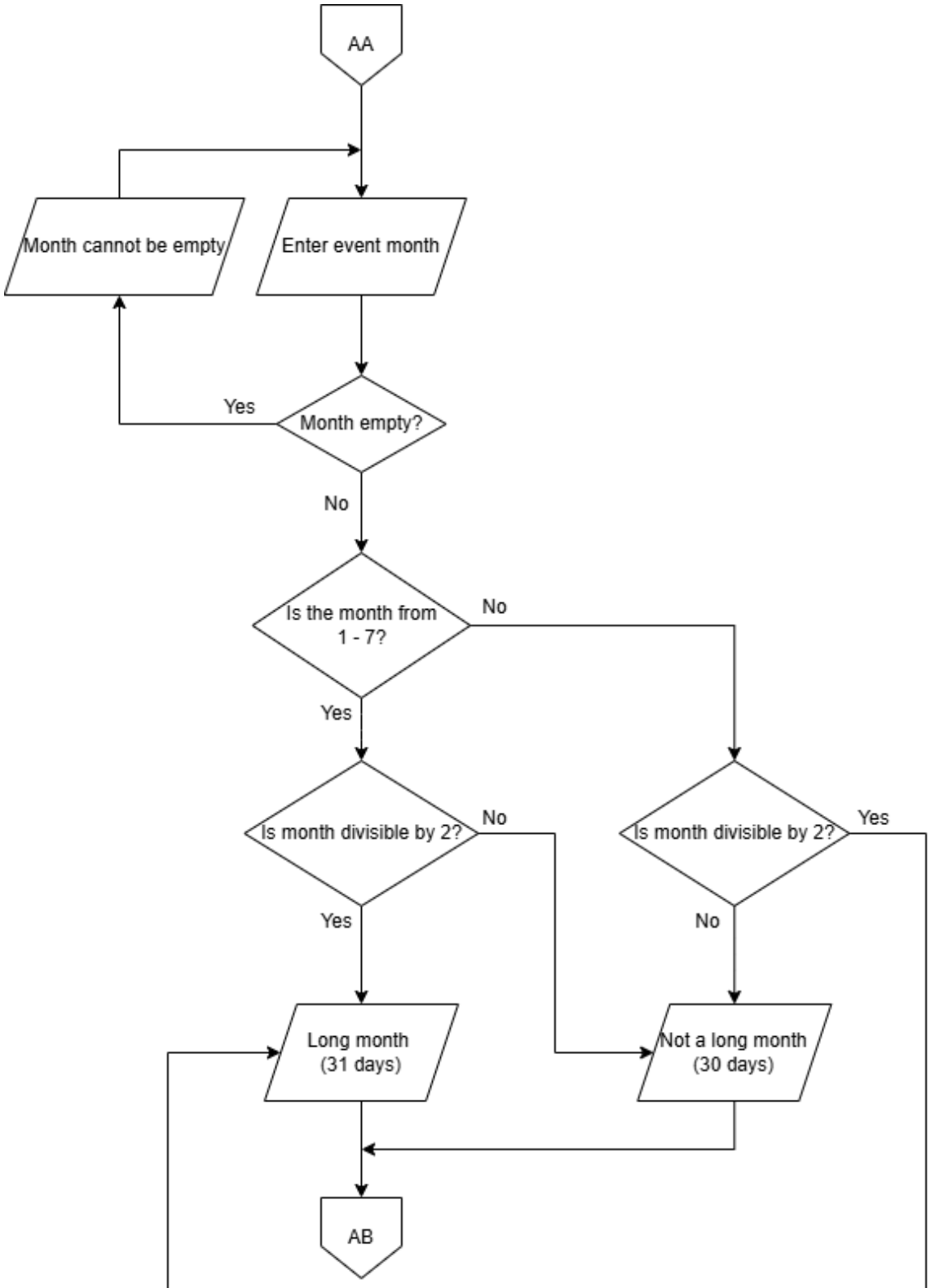


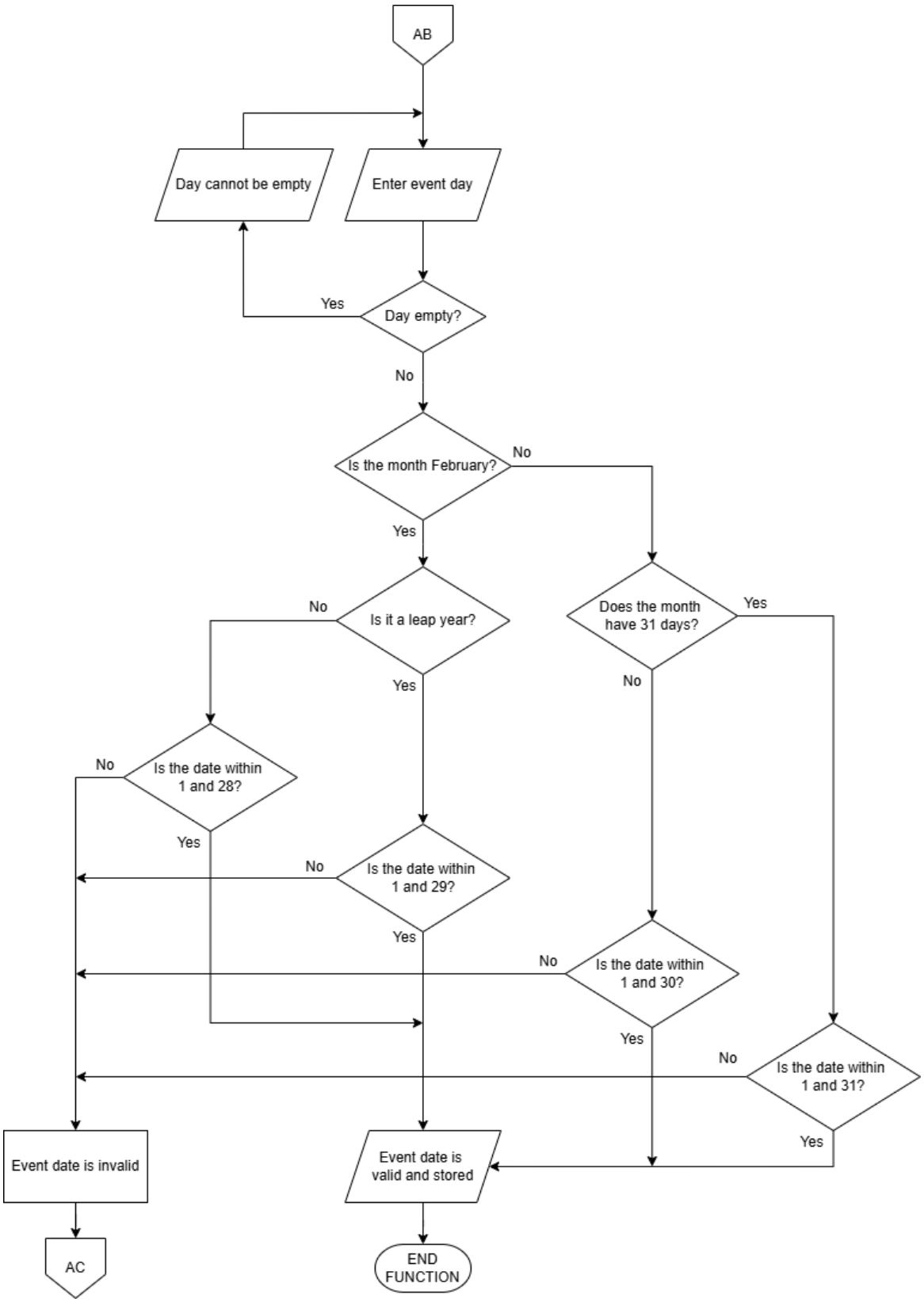
Flowchart for InsertEventName()



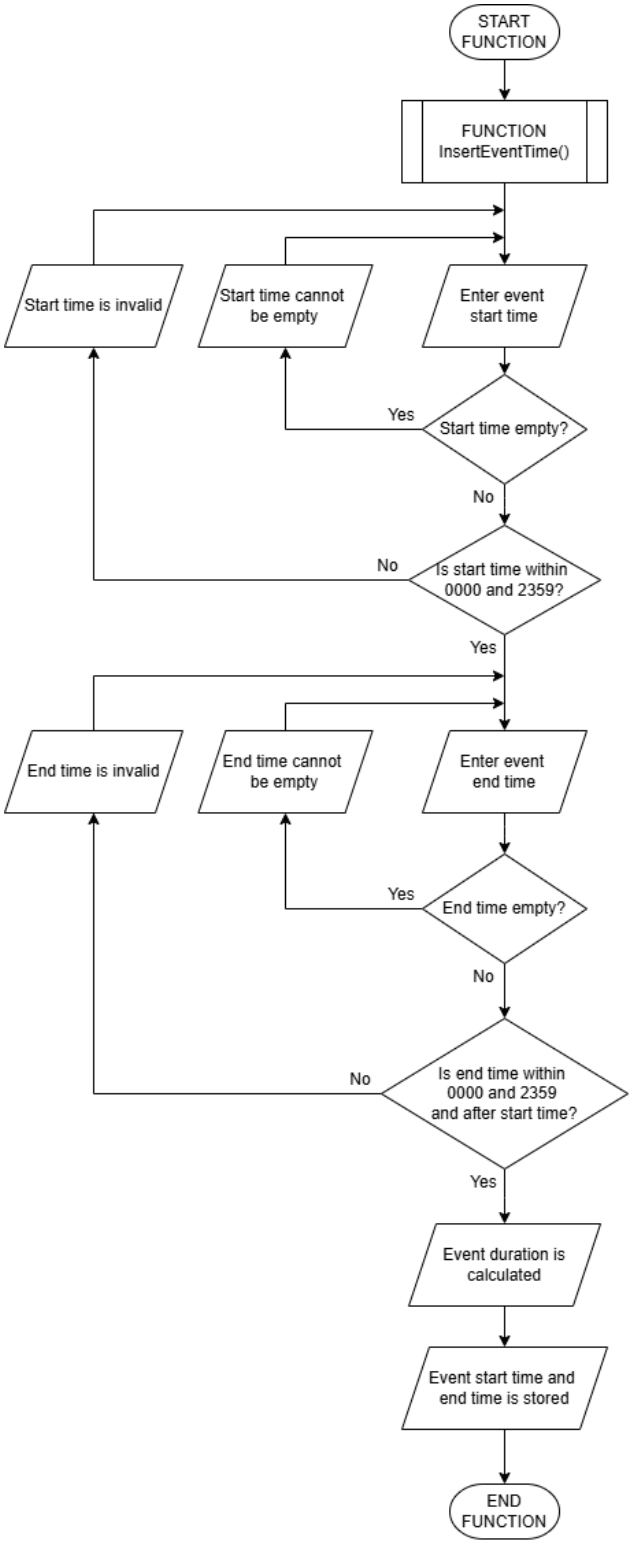
Flowchart for InsertEventDate()



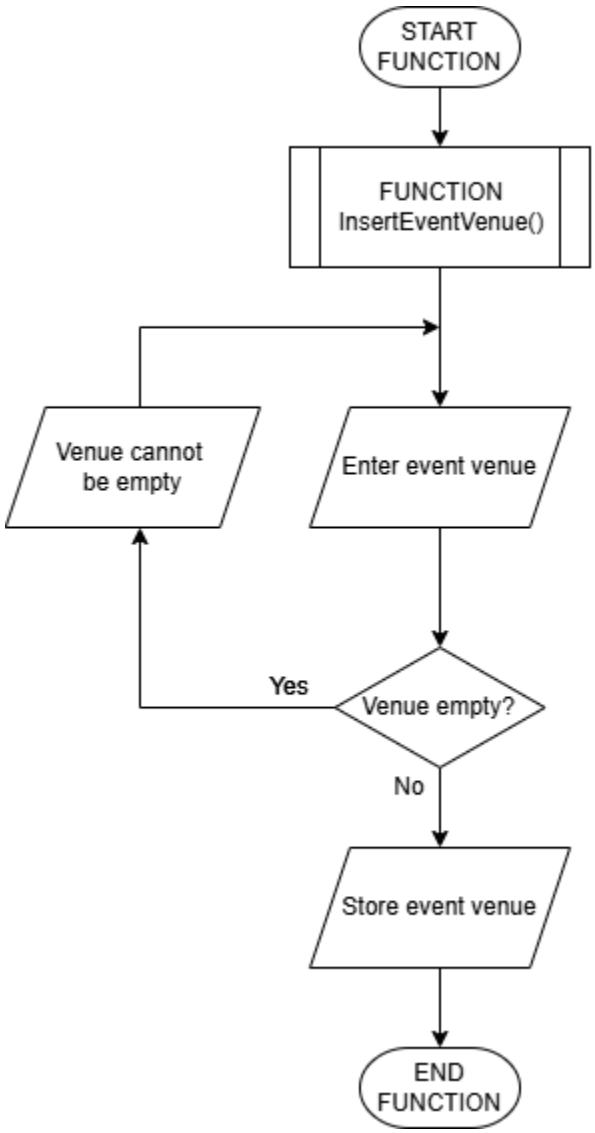

AA
Enter event month
Month cannot be empty
Month empty?
Yes
No
Is the month from 1 - 7?
No
Yes
Is month divisible by 2?
No
Yes
Is month divisible by 2?
Yes
No
Long month (31 days)
Not a long month (30 days)
AB


AB
Day cannot be empty
Enter event day
Yes
Day empty?
No
Is the month February?
No
Yes
Is it a leap year?
No
Yes
Does the month have 31 days?
Yes
No
Is the date within 1 and 28?
No
Yes
Is the date within 1 and 29?
No
Yes
Is the date within 1 and 30?
No
Yes
Is the date within 1 and 31?
No
Yes
Event date is invalid
Event date is valid and stored
AC
END FUNCTION

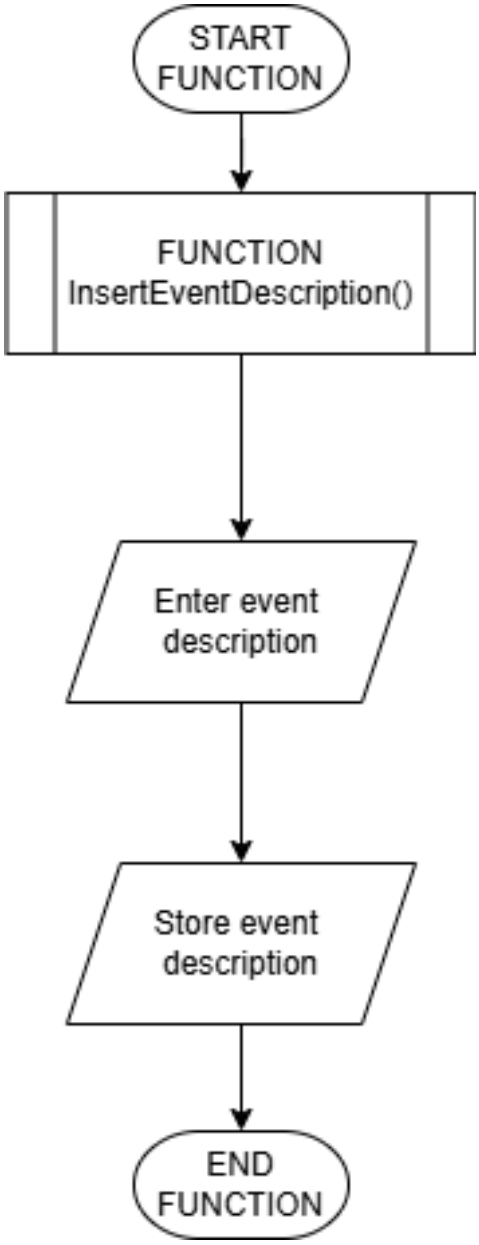
Flowchart for InsertEventTime()



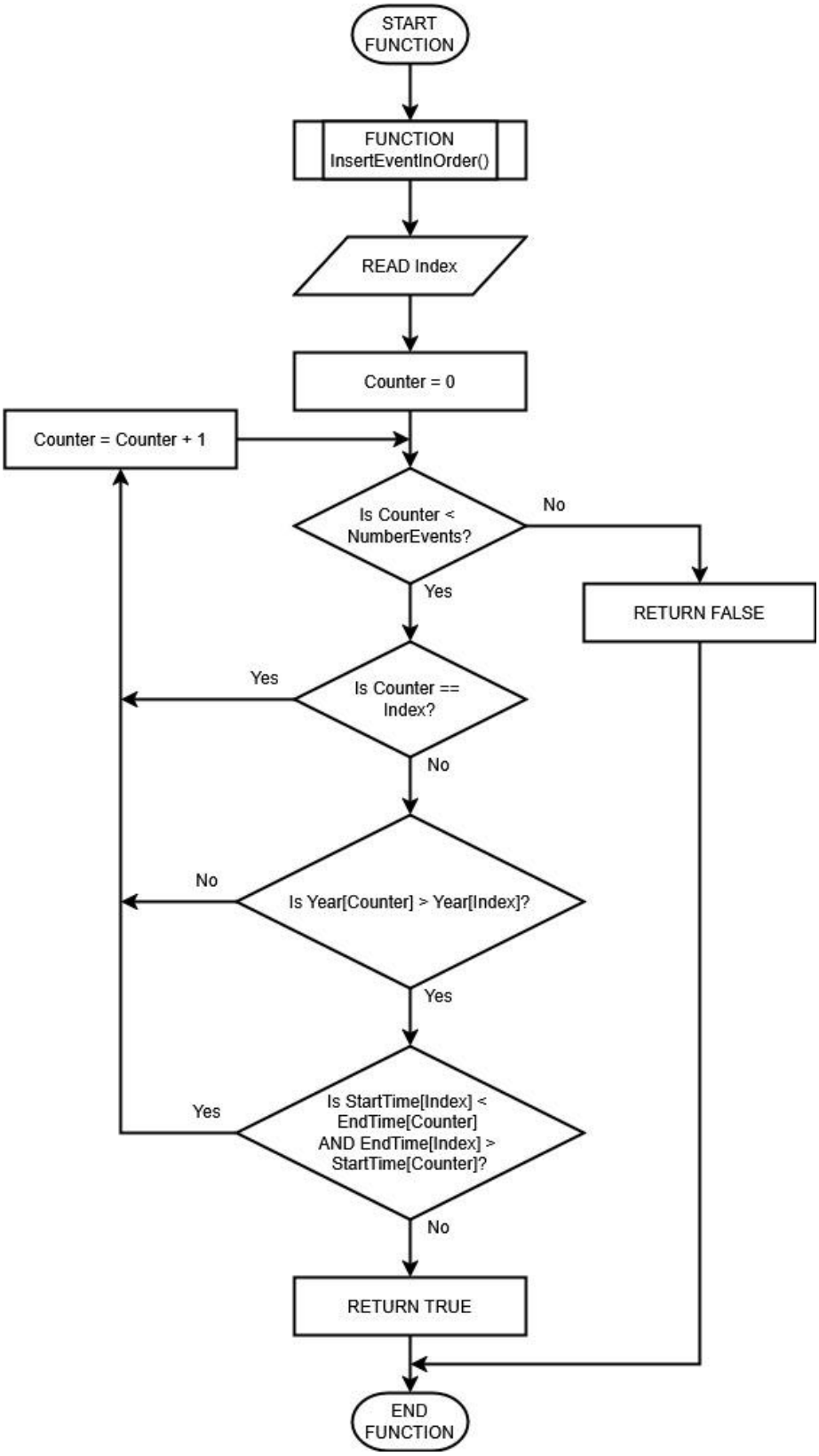
Flowchart for InsertEventVenue()



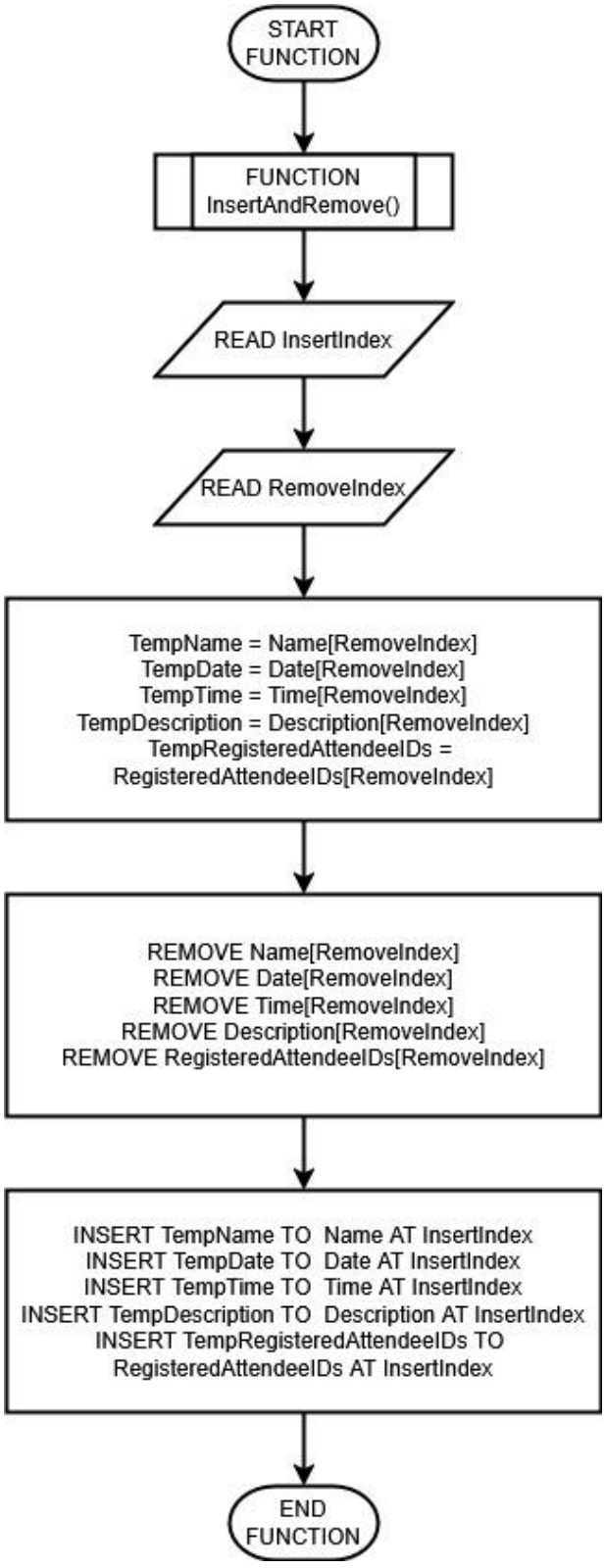
Flowchart for InsertEventDescription()



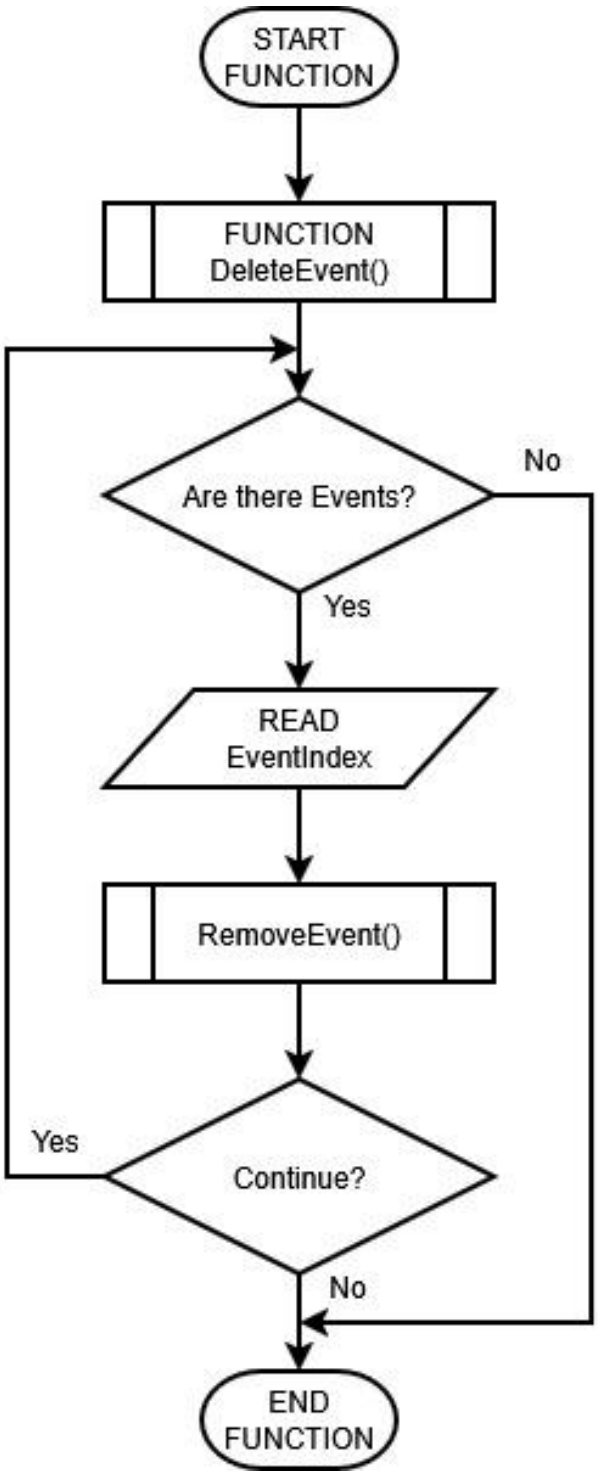
Flowchart for InsertEventInOrder()



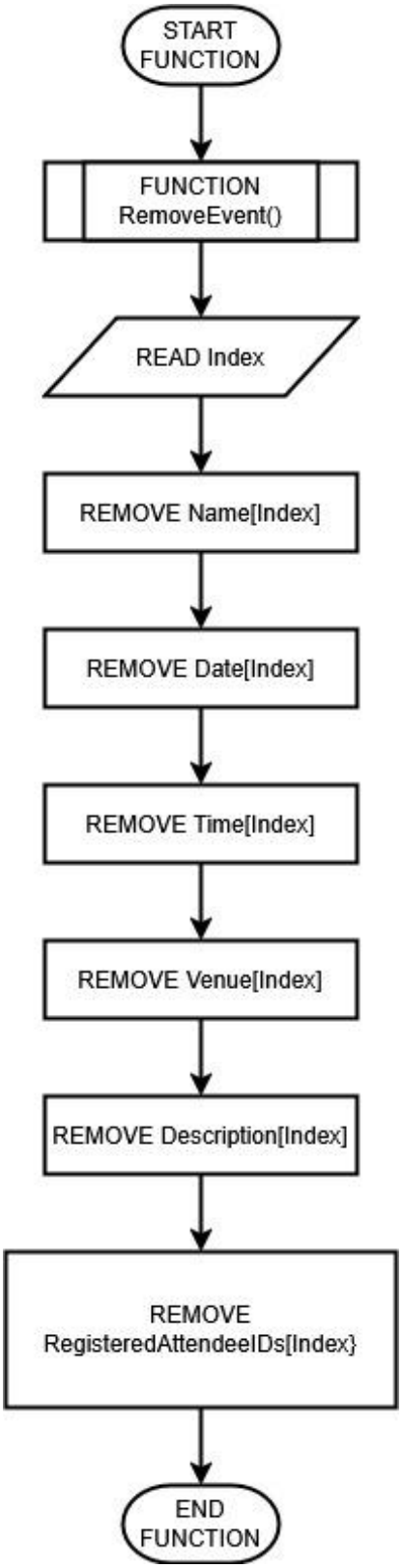
Flowchart for InsertAndRemove()



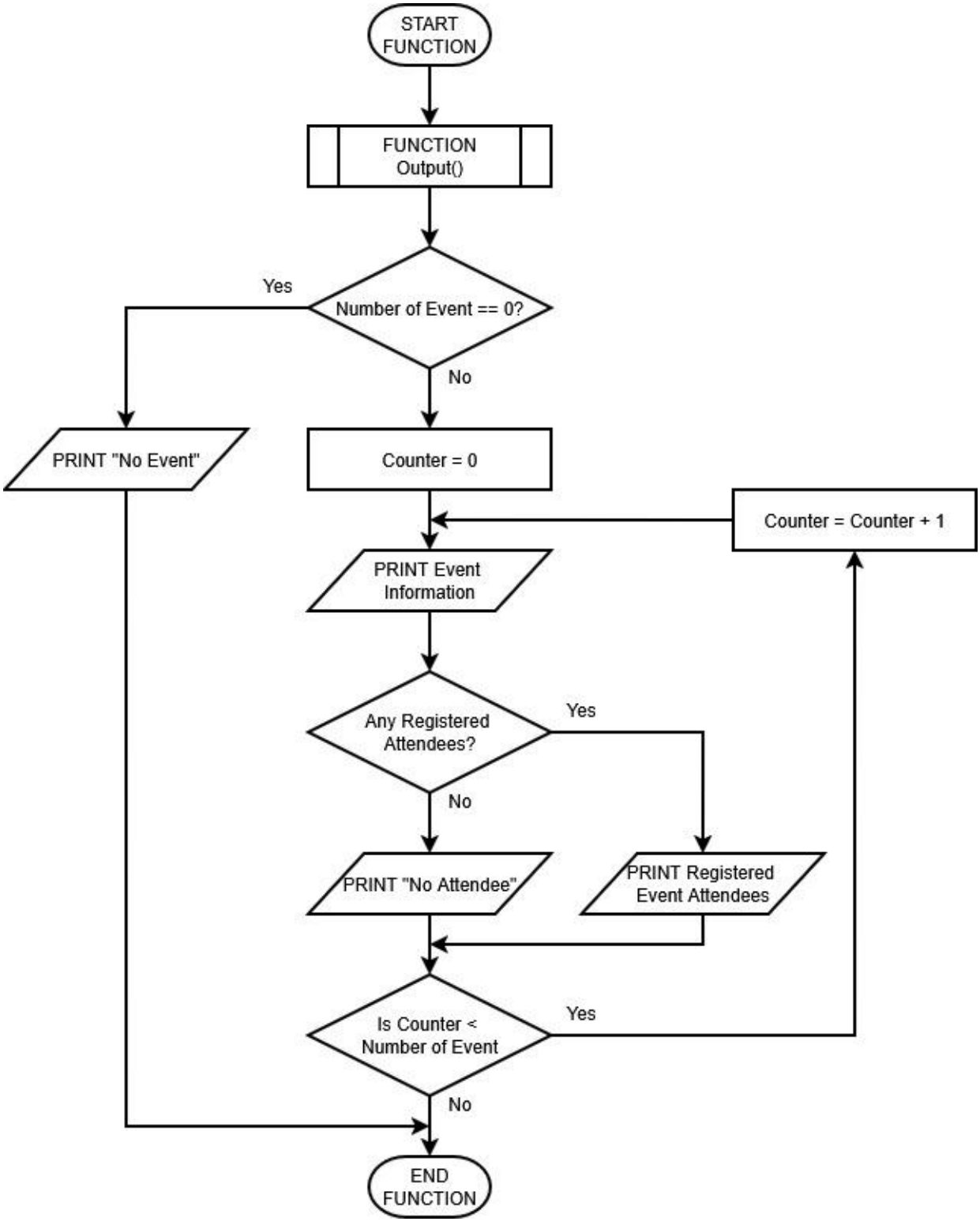
Flowchart for DeleteEvent()



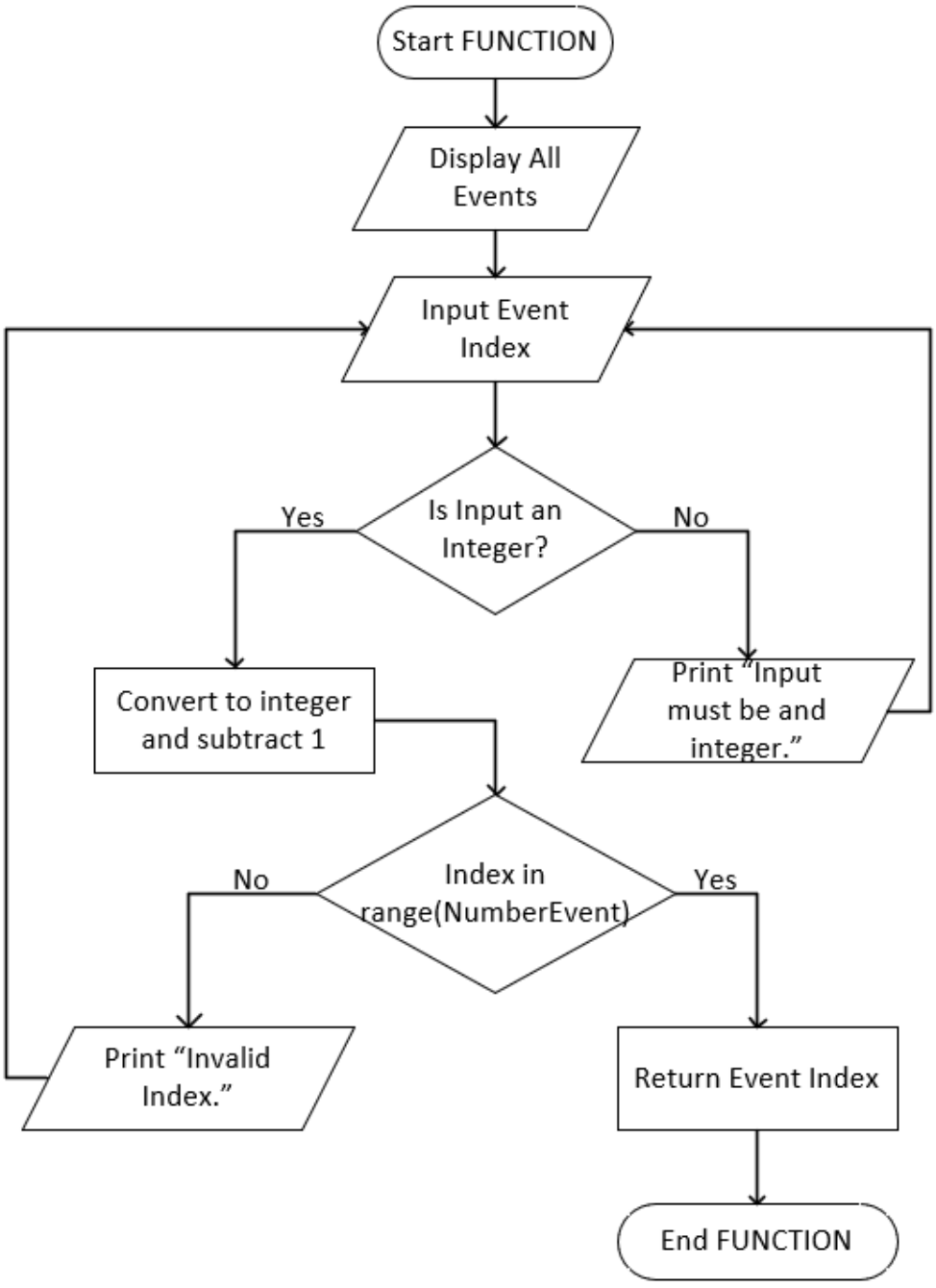
Flowchart for RemoveEvent()



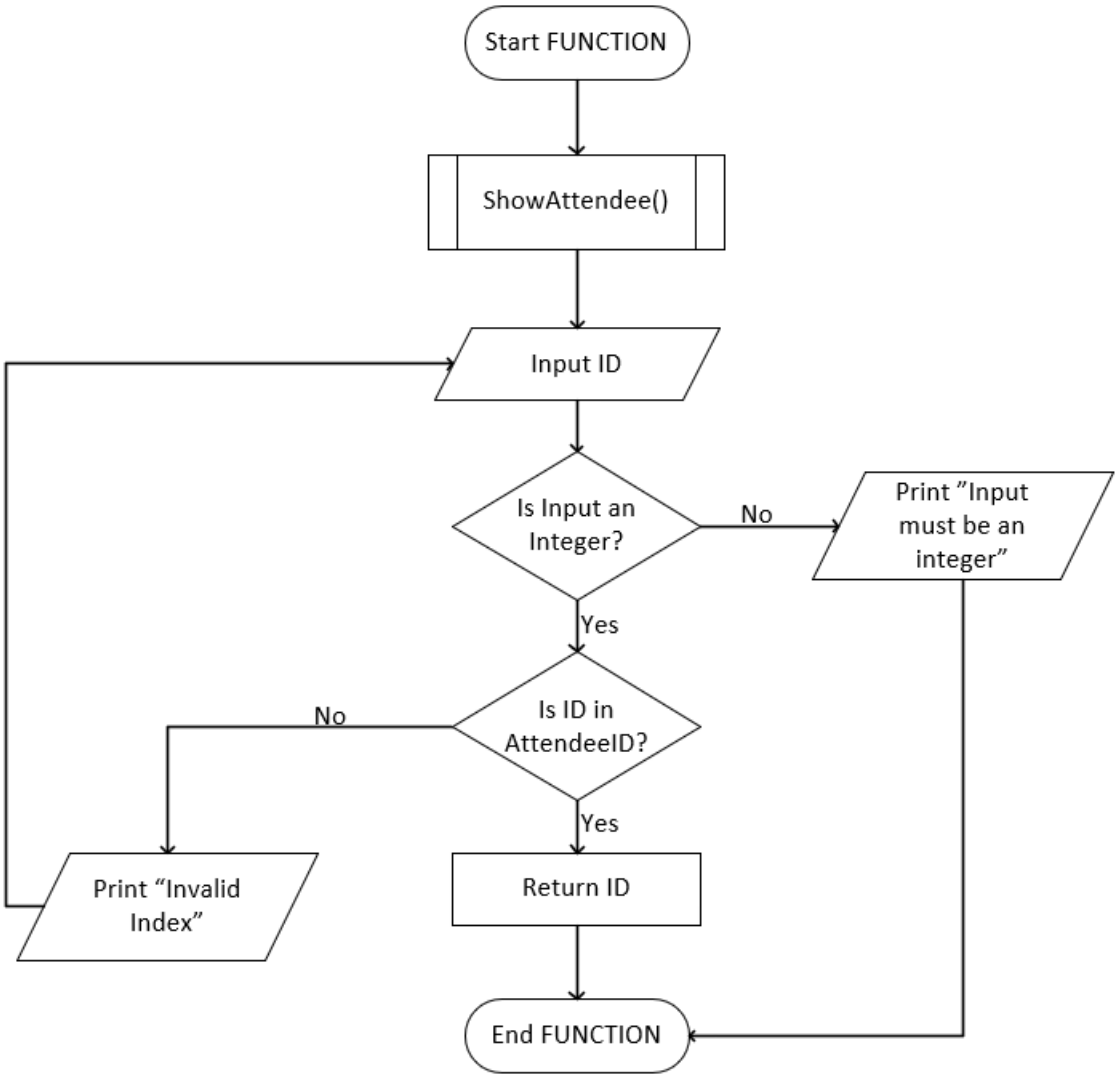
Flowchart for Output()



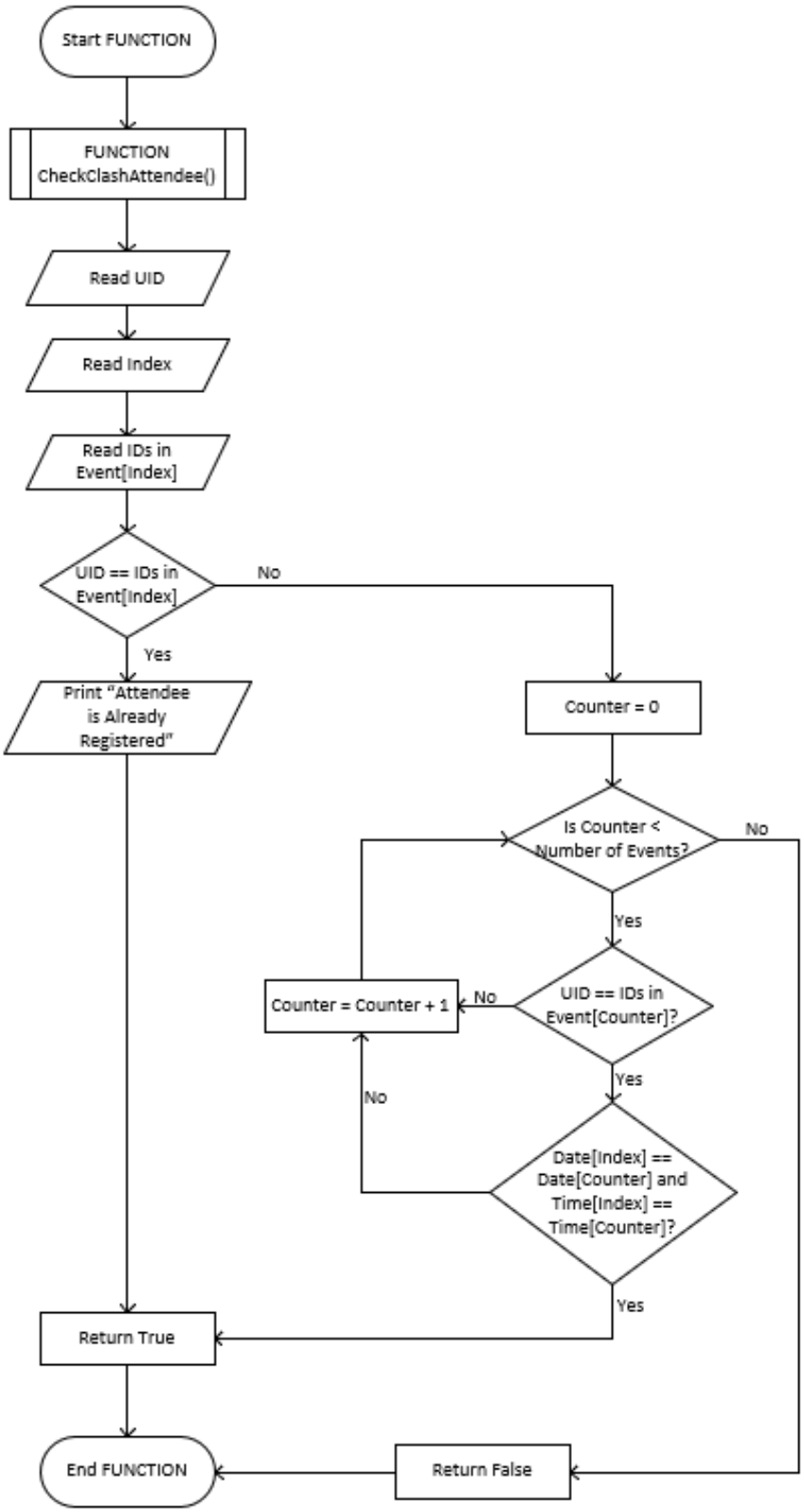
Flowchart for ChooseEvent()



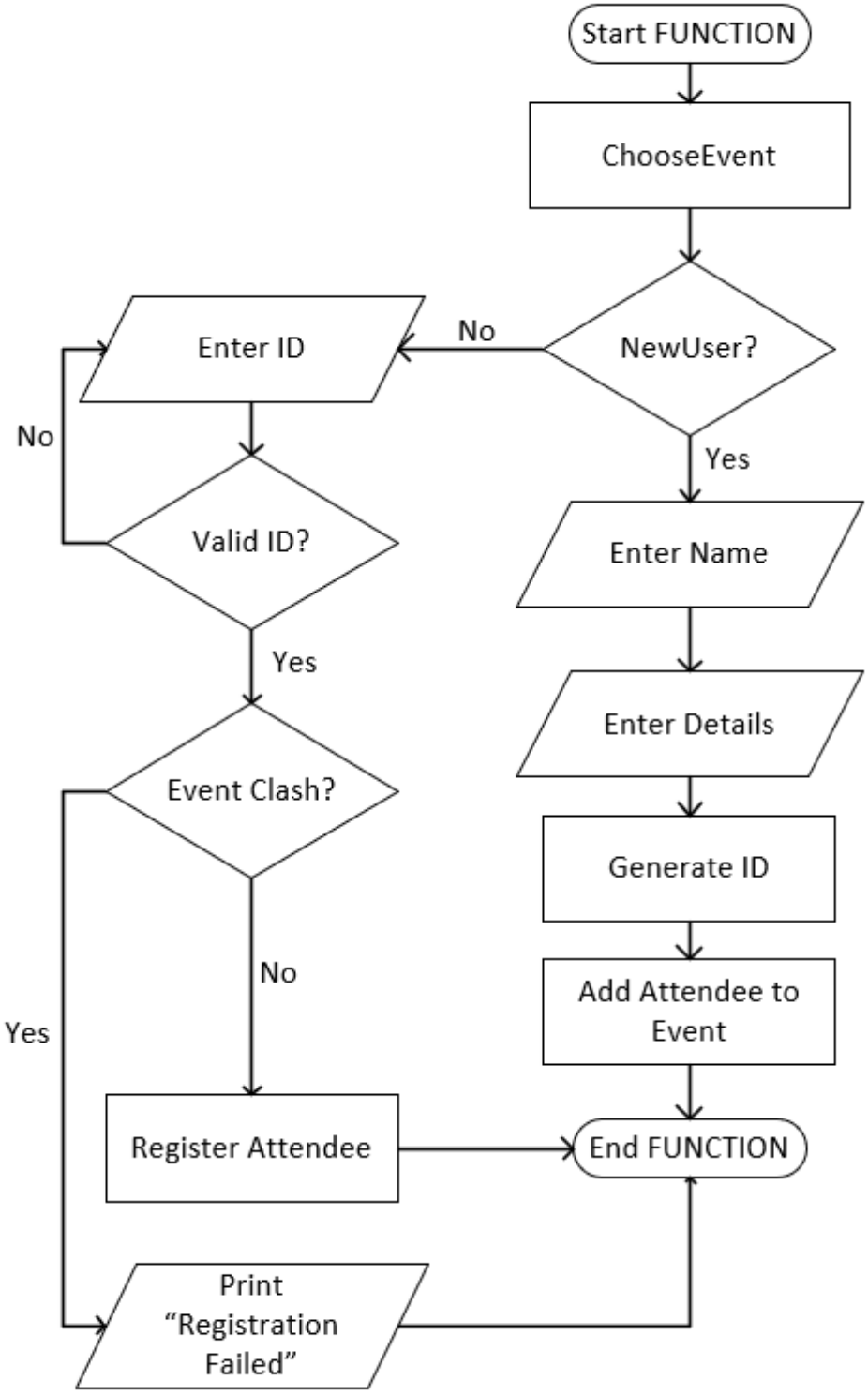
Flowchart for ChooseAttendee()



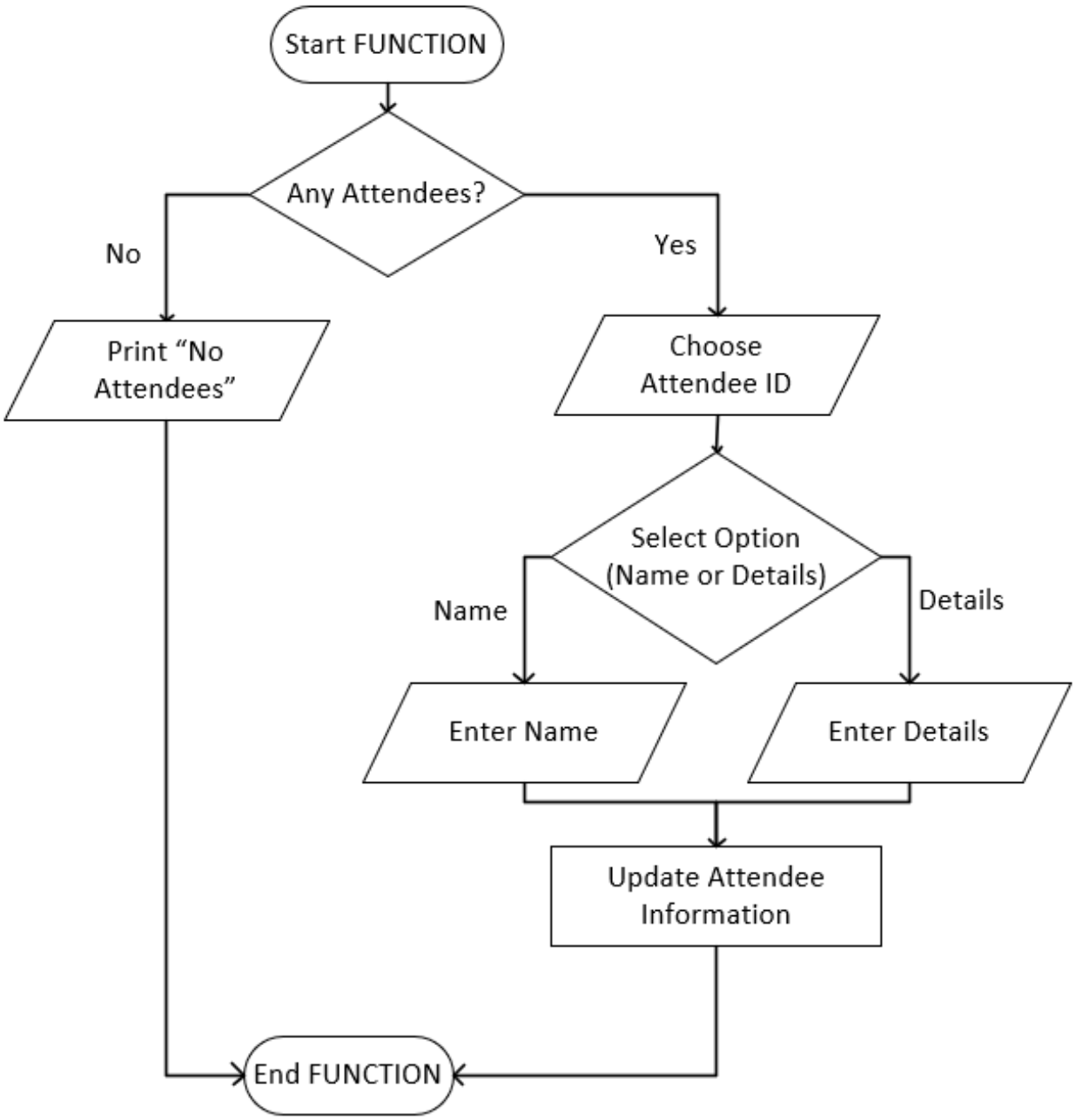
Flowchart for CheckClashAttendee()



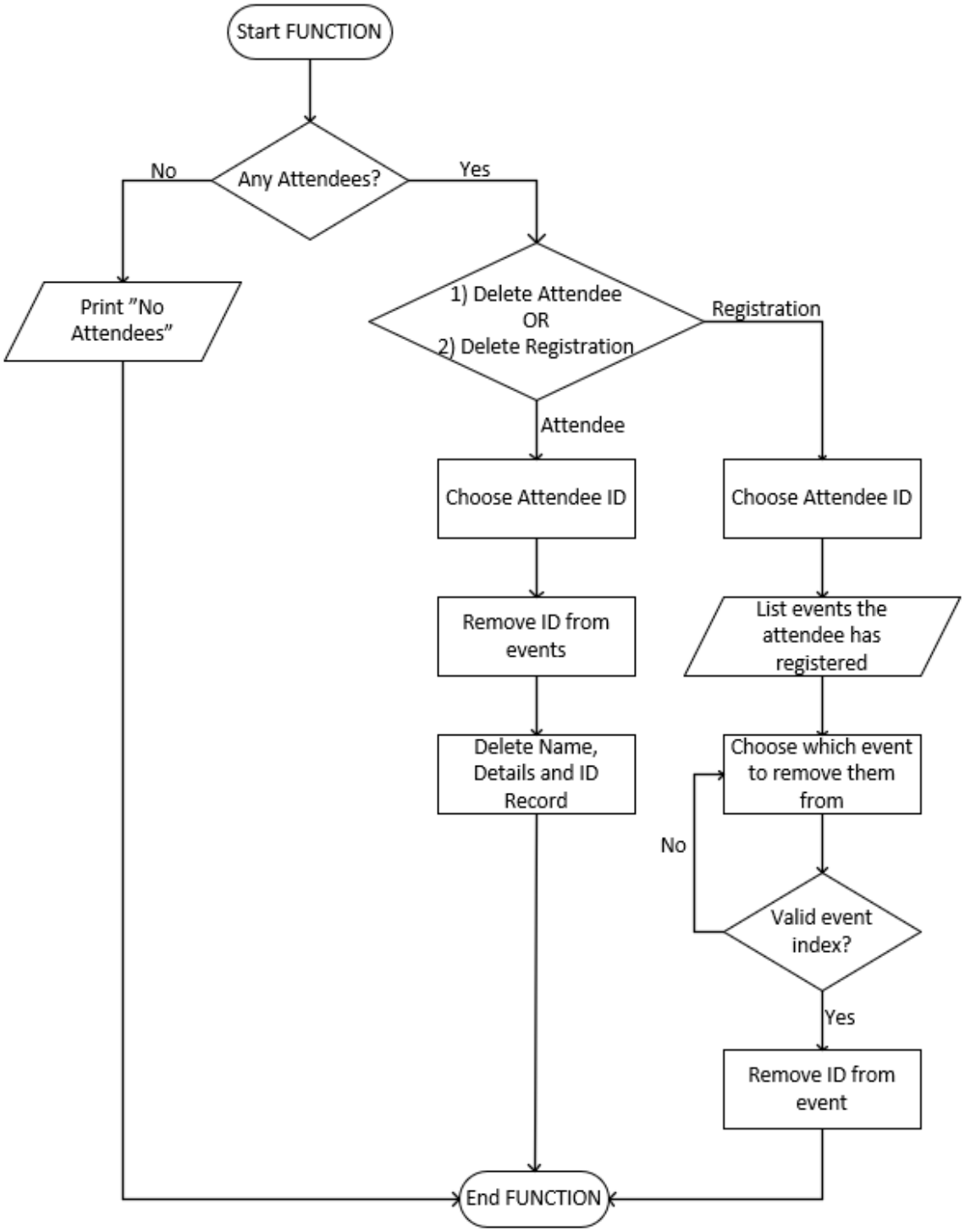
Flowchart for RegisterAttendee()



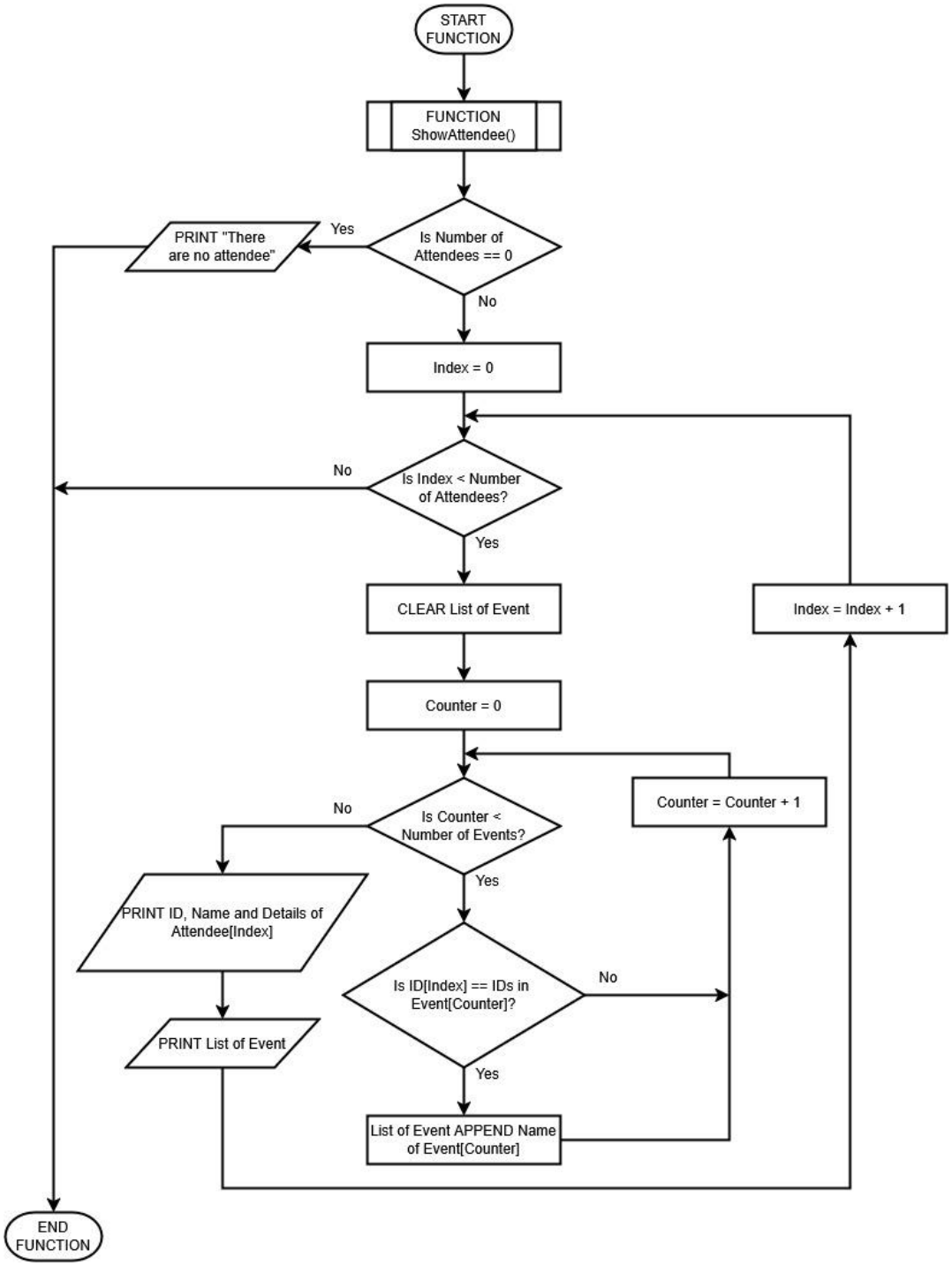
Flowchart for UpdateAttendee()



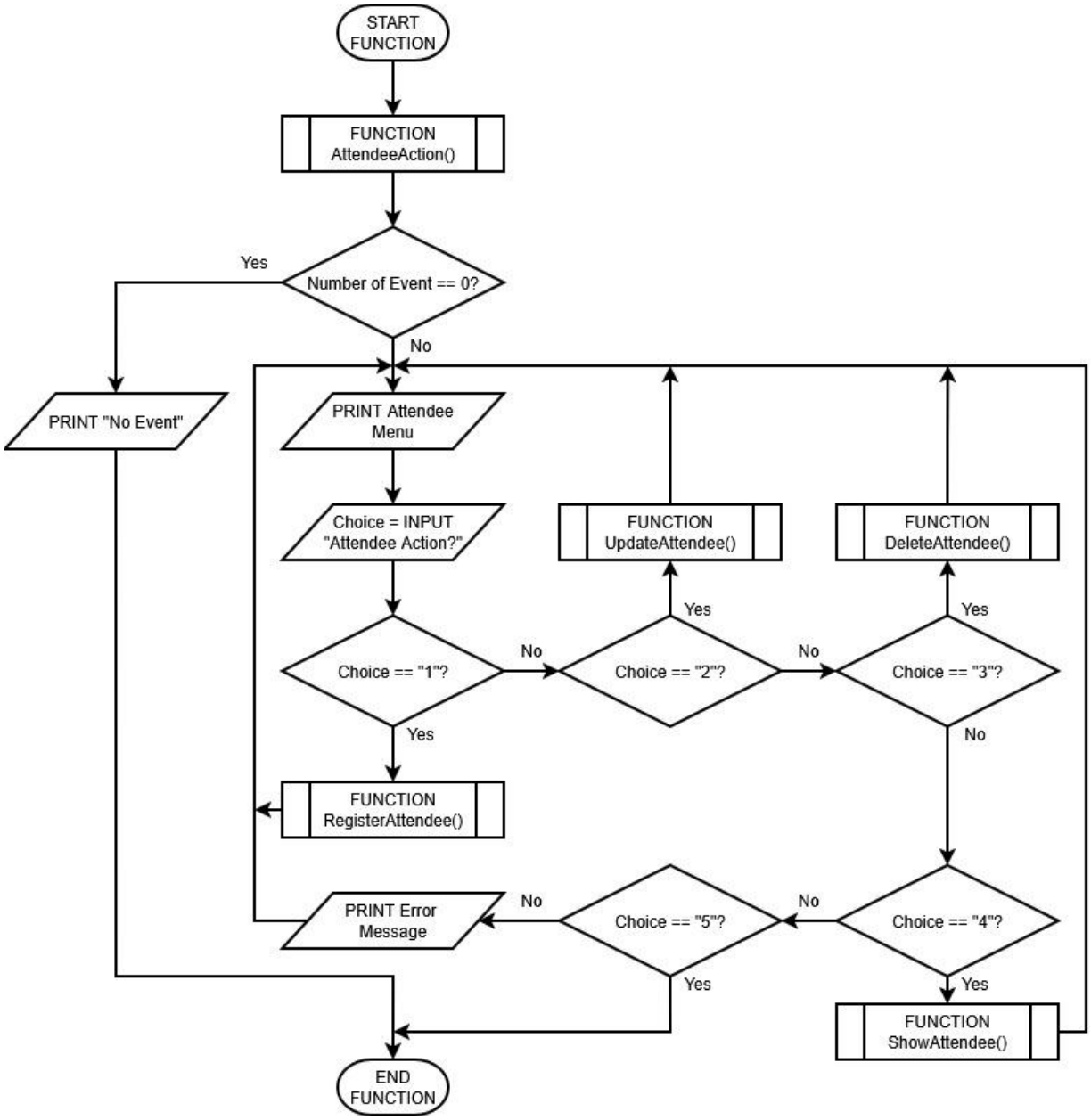
Flowchart for DeleteAttendee()



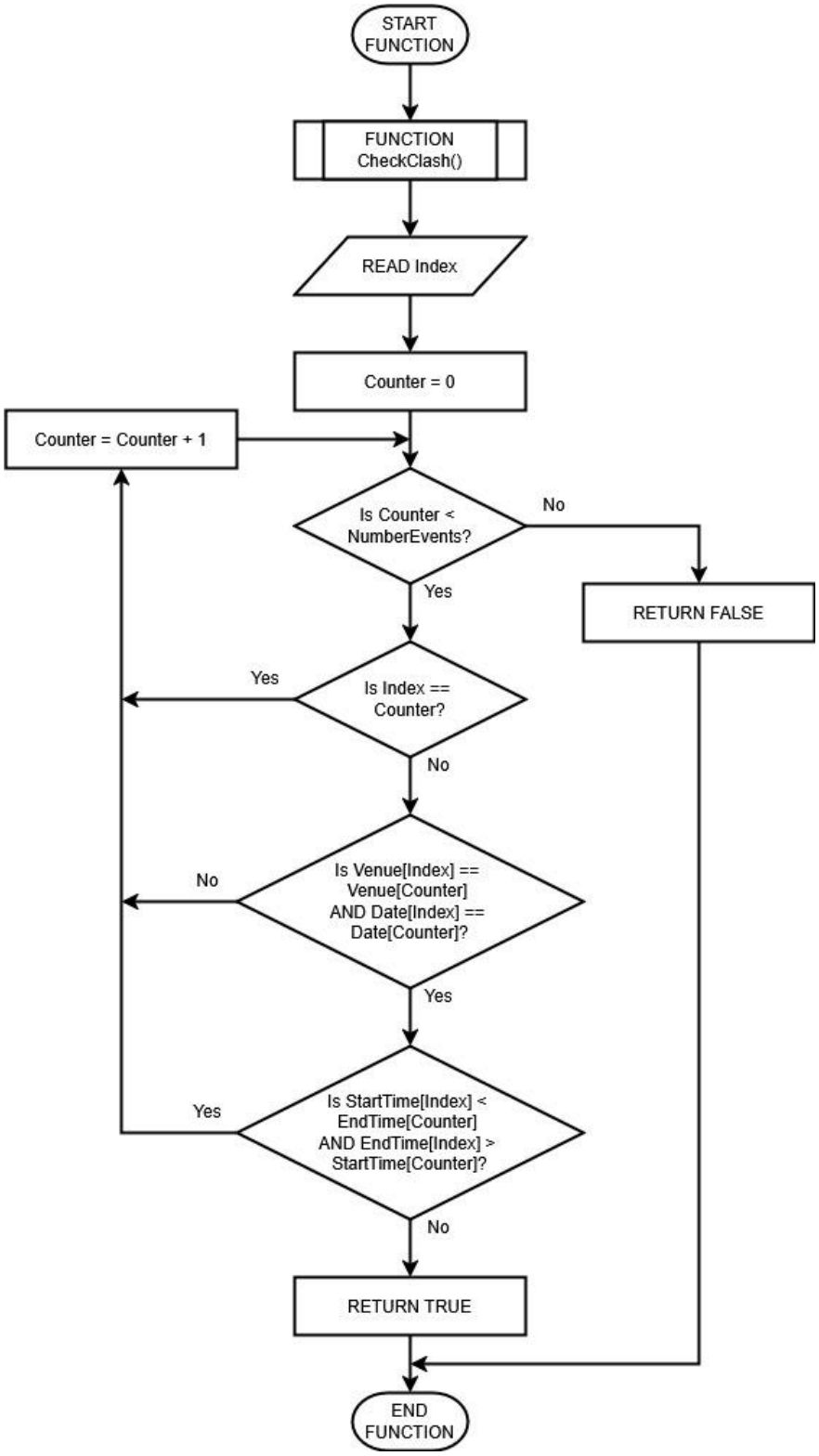
Flowchart for ShowAttendee()



Flowchart for AttendeeAction()



Flowchart for CheckClashes()



Coding – System Development

```
"""
Lists
EventName = [] #List of Strings
EventDateYear = [] #List of Integer
EventDateMonth = [] #List of Integer
EventDateDay = [] #List of Integer
EventStartTime = [] #List of Integer
EventEndTime = [] #List of Integer
EventVenue = [] #List of String
EventDescription = [] #List of String
#Attendee Lists Below
NumberAttendee = [] #Number of Attendees of Indexed Event, List of Integer
AttendeeID = [] #List of Integer
AttendeeName = [] #List of String
AttendeeDetails = [] #List of String
EventID = [] #ID of Attendees of Indexed Event, List of String

Global variables
NumberEvent = 0
AttendeeCounter = 0
"""


#Change List of String to List of Integer
def FormatToInteger(Parameter):
   ListReturn = []
   for Counter in range(len(Parameter)):
       ListReturn.append(int(Parameter[Counter]))
   return ListReturn


#Change List of Integer to List of String
def FormatToString(Parameter):
   ListReturn = []
   for Counter in range(len(Parameter)):
       ListReturn.append(str(Parameter[Counter]))
   return ListReturn


#Read "Events_Information.txt" to get previous information
Separator = "-"
File = open("Events_Information.txt","r")
Line1 = File.readline().strip("\n")
if Line1 != "":
   EventName = Line1.split(Separator) #Line 1
else:
   EventName = []
EventDateYear = FormatToInteger(File.readline().split()) #Line 2
EventDateMonth = FormatToInteger(File.readline().split()) #Line 3
EventDateDay = FormatToInteger(File.readline().split()) #Line 4
EventStartTime = FormatToInteger(File.readline().split()) #Line 5
EventEndTime = FormatToInteger(File.readline().split()) #Line 6
Line7 = File.readline().strip("\n")
if Line7 != "":
   EventVenue = Line7.split(Separator) #Line 7
else:
   EventVenue = []
Line8 = File.readline().strip("\n")
if Line8 != "":
   EventDescription = Line8.split(Separator) #Line 8
else:
   EventDescription = []
```

```
NumberAttendee = FormatToInteger(File.readline().split()) #Line 9
AttendeeID = FormatToInteger(File.readline().split()) #Line 10
Line11 = File.readline().strip("\n")
if Line11 != "":
   AttendeeName = Line11.split(Separator) #Line 11
else:
   AttendeeName = []
Line12 = File.readline().strip("\n")
if Line12 != "":
   AttendeeDetails = Line12.split(Separator) #Line 12
else:
   AttendeeDetails = []
Line13 = File.readline().strip("\n")
if Line13 != "":
   EventID = Line13.split(Separator) #Line 13
else:
   EventID = []
NumberEvent = int(File.readline()) #Line 14
AttendeeCounter = int(File.readline()) #Line 15
File.close()


#Empty strings to allow them to be used
for Index in range(NumberEvent):
   if EventDescription[Index] == "No description":
       EventDescription[Index] = ""
for ID in range(len(AttendeeID)):
   if AttendeeDetails[ID] == "No details":
       AttendeeDetails[ID] = ""


#Get the information of index Remove then put it in index Insert, used only in function,
InsertEventInOrder()
def InsertAndRemove(Insert, Remove): #Insert and Remove are integers
   #Copy all information of event indexed Remove
   TempEventName = EventName[Remove]
   TempEventDateYear = EventDateYear[Remove]
   TempEventDateMonth = EventDateMonth[Remove]
   TempEventDateDay = EventDateDay[Remove]
   TempEventStartTime = EventStartTime[Remove]
   TempEventEndTime = EventEndTime[Remove]
   TempEventVenue = EventVenue[Remove]
   TempEventDescription = EventDescription[Remove]
   TempNumberAttendee = NumberAttendee[Remove]
   TempEventID = EventID[Remove]


   #Remove all information of event indexed Remove
   EventName.pop(Remove)
   EventDateYear.pop(Remove)
   EventDateMonth.pop(Remove)
   EventDateDay.pop(Remove)
   EventStartTime.pop(Remove)
   EventEndTime.pop(Remove)
   EventVenue.pop(Remove)
   EventDescription.pop(Remove)
   NumberAttendee.pop(Remove)
   EventID.pop(Remove)


   #Insert all information of event indexed Remove into position indexed Insert
   EventName.insert(Insert, TempEventName)
   EventDateYear.insert(Insert, TempEventDateYear)
   EventDateMonth.insert(Insert, TempEventDateMonth)
   EventDateDay.insert(Insert, TempEventDateDay)
```

```
    EventStartTime.insert(Insert, TempEventStartTime)
    EventEndTime.insert(Insert, TempEventEndTime)
    EventVenue.insert(Insert, TempEventVenue)
    EventDescription.insert(Insert, TempEventDescription)
    NumberAttendee.insert(Insert, TempNumberAttendee)
    EventID.insert(Insert, TempEventID)


#Insert event in order of date and time, used during Add and Update
def InsertEventInOrder(Index):
    #Loop that checks where the information of event indexed Index should be inserted
    for Counter in range(len(EventName)):
        if Counter == Index: #Prevent checking with oneself
            continue
        elif EventDateYear[Counter] > EventDateYear[Index]: #Compare the Year
            InsertAndRemove(Counter, Index)
            break
        elif EventDateYear[Counter] < EventDateYear[Index]:
            continue
        elif EventDateMonth[Counter] > EventDateMonth[Index]: #Compare the Month
            InsertAndRemove(Counter, Index)
            break
        elif EventDateMonth[Counter] < EventDateMonth[Index]:
            continue
        elif EventDateDay[Counter] > EventDateDay[Index]: #Compare the Day
            InsertAndRemove(Counter, Index)
            break
        elif EventDateDay[Counter] < EventDateDay[Index]:
            continue
        elif EventStartTime[Counter] > EventStartTime[Index]: #Compare the Start Time
            InsertAndRemove(Counter, Index)
            break
        elif EventStartTime[Counter] == EventStartTime[Index] and EventEndTime[Counter] >
EventEndTime[Index]: #Compare the End Time
            InsertAndRemove(Counter, Index)
            break
        else: #End Time of Counter <= End Time of Index
            continue


#Function for inserting event name
def InsertEventName(Index):
    Valid = False
    while not Valid:
        TempEventName = input("Enter event name: ").strip()
        if TempEventName != "" and len(TempEventName) <= 15:
            if TempEventName not in EventName:
                EventName[Index] = TempEventName
                Valid = True


            else:
                while not Valid:
                    print("Event name already exists.")
                    UserCheck = input("Do you still want to use the same name? (Y/N):
").strip().upper()
                    if UserCheck == "Y":
                        EventName[Index] = TempEventName
                        Valid = True

                    elif UserCheck == "N":
                        print("Please reenter the event name.")
                        break
```

```
                else:
                    print("Invalid input. Please enter Y or N.")


        else:
            print("Event name cannot be empty or more than 15 characters.")


#Function for inserting event date
def InsertEventDate(Index):
   CorrectDate = False
   while not CorrectDate:
       ValidYear = False
       while not ValidYear:
           TempEventDateYear = input("Enter event year: ").strip()
           if TempEventDateYear != "":
               if TempEventDateYear.isdigit():
                   TempEventDateYear = int(TempEventDateYear)
                   if TempEventDateYear > 0 and TempEventDateYear < 10000:
                       ValidYear = True
                       if (TempEventDateYear % 4 == 0 and TempEventDateYear % 100 != 0) or
TempEventDateYear % 400 == 0:
                           LeapYear = True

                       else:
                           LeapYear = False


                   else:
                       print("Event year is invalid.")


               else:
                   print("Event year must be a number.")


           else:
               print("Event year cannot be empty.")


       ValidMonth = False
       while not ValidMonth:
           TempEventDateMonth = input("Enter event month: ").strip()
           if TempEventDateMonth != "":
               if TempEventDateMonth.isdigit():
                   TempEventDateMonth = int(TempEventDateMonth)
                   if TempEventDateMonth >= 1 and TempEventDateMonth <= 7:
                       ValidMonth = True
                       if TempEventDateMonth % 2 == 0:
                           LongMonth = False

                       else:
                           LongMonth = True


                   elif TempEventDateMonth >= 8 and TempEventDateMonth <= 12:
                       ValidMonth = True
                       if TempEventDateMonth % 2 == 0:
                           LongMonth = True

                       else:
                           LongMonth = False


                   else:
                       print("Event month must be between 1 and 12 inclusive.")
```

```
                else:
                    print("Event month must be a number.")


            else:
                print("Event month cannot be empty.")


        ValidDay = False
        while not ValidDay:
            TempEventDateDay = input("Enter event day: ").strip()
            if TempEventDateDay != "":
                if TempEventDateDay.isdigit():
                    TempEventDateDay = int(TempEventDateDay)
                    if TempEventDateDay >= 1 and ((TempEventDateMonth == 2 and ((LeapYear and
TempEventDateDay <=29) or (not LeapYear and TempEventDateDay <=28))) or (not TempEventDateMonth
== 2 and ((LongMonth and TempEventDateDay <= 31) or (not LongMonth and TempEventDateDay <=30)))):
                        ValidDay = True


                    else:
                        print("Event date does not exist.")


                else:
                    print("Event day must be a number.")


            else:
                print("Event day cannot be empty.")


        ValidInput = False
        while not ValidInput:
            print(f"Event date is
{TempEventDateDay:02d}/{TempEventDateMonth:02d}/{TempEventDateYear:04d}")
            UserCheck = input("Is the date correct? (Y/N): ").strip().upper()
            if UserCheck != "":
                if UserCheck == "Y":
                    ValidInput = True
                    CorrectDate = True


                elif UserCheck == "N":
                    break

                else:
                    print("Invalid input. Please enter Y or N.")


            else:
                print("Input cannot be empty.")


        if CorrectDate:
            EventDateYear[Index] = TempEventDateYear
            EventDateMonth[Index] = TempEventDateMonth
            EventDateDay[Index] = TempEventDateDay


        else:
            print("Please reenter the date.")


#Function for inserting event time
def InsertEventTime(Index):
```

```
    CorrectTime = False
    while not CorrectTime:
        ValidStartTime = False
        while not ValidStartTime:
            TempEventStartTime = input("Enter event start time (in 24-hour format, e.g. 1300 for
1p.m.): ").strip()
            if TempEventStartTime != "":
                if TempEventStartTime.isdigit():
                    TempEventStartTime = int(TempEventStartTime)
                    if TempEventStartTime >= 0 and TempEventStartTime < 2400 and
TempEventStartTime % 100 >= 0 and TempEventStartTime % 100 < 60:
                        ValidStartTime = True


                    else:
                        print("Event start time's hour must be between 0 and 23 inclusive and
minute must be between 0 and 59 inclusive.")


                else:
                    print("Event start time must be a number.")


            else:
                print("Event start time cannot be empty.")


        ValidEndTime = False
        while not ValidEndTime:
            TempEventEndTime = input("Enter event end time (in 24-hour format, e.g. 1400 for
2p.m.): ").strip()
            if TempEventEndTime != "":
                if TempEventEndTime.isdigit():
                    TempEventEndTime = int(TempEventEndTime)
                    if TempEventEndTime >= 0 and TempEventEndTime < 2400 and TempEventEndTime %
100 >= 0 and TempEventEndTime % 100 < 60:
                        ValidEndTime = True


                    else:
                        print("Event end time's hour must be between 0 and 23 inclusive and minute
must be between 0 and 59 inclusive.")


                else:
                    print("Event end time must be a number.")


            else:
                print("Event end time cannot be empty.")


        ValidInput = False
        while not ValidInput:
            if TempEventEndTime > TempEventStartTime:
                DurationHour = TempEventEndTime // 100 - TempEventStartTime // 100
                DurationMinute = TempEventEndTime % 100 - TempEventStartTime % 100
                if DurationMinute < 0:
                    DurationMinute += 60
                    DurationHour -= 1

                print(f"Event time is from {TempEventStartTime:04d} to {TempEventEndTime:04d} for
{DurationHour} h {DurationMinute} min.")
                UserCheck = input("Is the time correct? (Y/N) ").strip().upper()
                if UserCheck != "":
                    if UserCheck == "Y":
                        ValidInput = True
```

```
                    CorrectTime = True

                elif UserCheck == "N":
                    print("Please reenter the time.")
                    break

                else:
                    print("Invalid input. Please enter Y or N.")

            else:
                print("Input cannot be empty.")

        else:
            print("Event end time cannot be earlier than event start time.")
            break

    if CorrectTime:
        EventStartTime[Index] = TempEventStartTime
        EventEndTime[Index] = TempEventEndTime

    else:
        print("Please reenter the time.")

#Function for inserting event venue
def InsertEventVenue(Index):
    Valid = False
    while not Valid:
        TempEventVenue = input("Enter event venue: ").strip()
        if TempEventVenue != "" and len(TempEventVenue) <= 15:
            EventVenue[Index] = TempEventVenue
            Valid = True

        else:
            print("Event venue cannot be empty or more than 15 characters.")

#Function for inserting event description
def InsertEventDescription(Index):
    Valid = False
    while not Valid:
        TempEventDescription = input("Enter event description (Optional): ").strip()
        if len(TempEventDescription) <= 30:
            Valid = True
            if TempEventDescription != "":
                EventDescription[Index] = TempEventDescription

            else:
                EventDescription[Index] = "-"
        else:
            print("Event description cannot be more than 30 characters.")

#Function for adding event
def AddEvent():
    print(f"\n{"=" * 30}\n{"Add Event Interface":^30}\n{"=" * 30}")
    UserCheck = "K"
    while UserCheck != "Y" or UserCheck != "N":
        UserCheck = input("Are you sure you want to add a new event? (Y/N) ").strip().upper()
        if UserCheck != "":
```

```
            if UserCheck == "Y":
                break

            elif UserCheck == "N":
                print("Event addition cancelled.\n")
                return

            else:
                print("Invalid input. Please enter Y or N.")

        else:
            print("Input cannot be empty.")

    global NumberEvent
    NumberEvent += 1
    EventName.append("")
    InsertEventName(NumberEvent-1)
    EventDateYear.append(0)
    EventDateMonth.append(0)
    EventDateDay.append(0)
    InsertEventDate(NumberEvent-1)
    EventStartTime.append(0)
    EventEndTime.append(0)
    InsertEventTime(NumberEvent-1)
    EventVenue.append("")
    InsertEventVenue(NumberEvent-1)
    EventDescription.append("")
    InsertEventDescription(NumberEvent-1)
    NumberAttendee.append(0)
    EventID.append("")
    if NumberEvent != 1:
        if CheckClashes(NumberEvent-1) == True:
            RemoveEvent(NumberEvent-1)

        else:
            print("Event added successfully.\n")
            InsertEventInOrder(NumberEvent-1)

    else:
        print("Event added successfully.\n")

def UpdateEvent():
    length_of_EventName = len(EventName)
    if length_of_EventName == 0:
        print("There are no events. Please type 1 in the interface to add an event first.\n")
        return

    ChoiceUpdation = True
    while ChoiceUpdation:
        choice_on_updating_event = input("Do you want to update an event? (Y/N):
").strip().upper()
        if choice_on_updating_event == "Y":
            ChoiceUpdation = False

        elif choice_on_updating_event == "N":
            ChoiceUpdation = False
            return

        else:
```

```
            print("Invalid input. Please enter Y or N.")
            continue

    Updating = True
    while Updating:
        Output()
        choice_on_updating = input(f"Which one would you like to update? (Please enter a number
from 1 - {length_of_EventName}) or B to go back: ").strip()
        if choice_on_updating.upper() == "B":
            Updating = False

        if not choice_on_updating.isdigit():
            print("Invalid input. Please enter a number or B to go back.")
            continue
        choice = int(choice_on_updating)
        if 1 <= choice <= length_of_EventName:
            choice -= 1

        else:
            print("Invalid event number.")
            continue
        FinishedUpdatingCurrentEvent = False
        while not FinishedUpdatingCurrentEvent:
            print(f"1. Name: {EventName[choice]}\n2. Date:
{EventDateDay[choice]}/{EventDateMonth[choice]}/{EventDateYear[choice]}\n3. Venue:
{EventVenue[choice]}\n4. Time: {EventStartTime[choice]:04d}-{EventEndTime[choice]:04d}\n5.
Description: {EventDescription[choice]}\n6. Update all\n7. Back")
            Action = input("Select option to update (1-7): ").strip()
            if Action != "":
                FinishedUpdatingCurrentEvent = True
                if Action == "1":
                    InsertEventName(choice)
                    print("Name updated.")

                elif Action == "2":
                    if UpdateEventDate(choice) == True:
                            print("Date updated.")

                    else:
                        print("Date not updated.")

                elif Action == "3":
                    if UpdateEventVenue(choice) == True:
                        print("Venue updated.")

                    else:
                        print("Venue not updated.")

                elif Action == "4":
                    if UpdateEventTime(choice) == True:
                        print("Time updated.")

                    else:
                        print("Time not updated.")

                elif Action == "5":
                    InsertEventDescription(choice)
                    print("Description updated.")

                elif Action == "6":
                    InsertEventName(choice)
                    DateUpdated = UpdateEventDate(choice)
                    VenueUpdated = UpdateEventVenue(choice)
```

```
                    TimeUpdated = UpdateEventTime(choice)
                    InsertEventDescription(choice)
                    if DateUpdated and VenueUpdated and TimeUpdated:
                        print("Event updated.")

                    else:
                        print("Event updated with some issues.")

                elif Action == "7":
                    break

                else:
                    print("Invalid option! Please select an option from 1-7.")
                    continue

                InsertEventInOrder(choice)
                Result = AskContinueUpdating()
                if Result == "same":
                    FinishedUpdatingCurrentEvent = False

                elif Result == "another":
                    FinishedUpdatingCurrentEvent = True

                elif Result == "quit":
                    Updating = False


            else:
                print("Invalid input! Please enter a number.")
                continue


#Function for updating event date
def UpdateEventDate(choice):
   StoreEventDateYear = EventDateYear[choice]
   StoreEventDateMonth = EventDateMonth[choice]
   StoreEventDateDay = EventDateDay[choice]
   ValidUpdation = False
   while not ValidUpdation:
       InsertEventDate(choice)
       if CheckClashes(choice) == True:
           print(f"An event Clash was detected.\nDate has changed back to
{StoreEventDateDay}/{StoreEventDateMonth}/{StoreEventDateYear}")
           EventDateYear[choice] = StoreEventDateYear
           EventDateMonth[choice] = StoreEventDateMonth
           EventDateDay[choice] = StoreEventDateDay
           while True:
               ChoiceOnRedoing= input("Do you want to re-enter your date? (Y/N):
").strip().upper()
               if ChoiceOnRedoing == "Y":
                   break

               elif ChoiceOnRedoing == "N":
                   ValidUpdation = True
                   return False

               else:
                   print("Invalid input! Please enter Y or N.")

       else:
           ValidUpdation = True
           return True

#Function for updating event start time and end time
```

```
def UpdateEventTime(choice):
   StoreEventStartTime = EventStartTime[choice]
   StoreEventEndTime = EventEndTime[choice]
   ValidUpdation = False
   while not ValidUpdation:
       InsertEventTime(choice)
       if CheckClashes(choice) == True:
           print(f"An event Clash was detected.\nTime has changed back to
{StoreEventStartTime:04d}-{StoreEventEndTime:04d}")
           EventStartTime[choice] = StoreEventStartTime
           EventEndTime[choice] = StoreEventEndTime
           while True:
               ChoiceOnRedoing= input("Do you want to re-enter your time? (Y/N):
").strip().upper()
               if ChoiceOnRedoing == "Y":
                   break

               elif ChoiceOnRedoing == "N":
                   ValidUpdation = True
                   return False

               else:
                   print("Invalid input. Please enter Y or N.")

       else:
           ValidUpdation = True
           return True


#Function for updating event venue
def UpdateEventVenue(choice):
   StoreEventVenue = EventVenue[choice]
   ValidUpdation = False
   while not ValidUpdation:
       InsertEventVenue(choice)
       if CheckClashes(choice) == True:
           print(f"An event Clash was detected.\nVenue has changed back to {StoreEventVenue}")
           EventVenue[choice] = StoreEventVenue
           while True:
               ChoiceOnRedoing= input("Do you want to re-enter your venue? (Y/N):
").strip().upper()
               if ChoiceOnRedoing == "Y":
                   break

               elif ChoiceOnRedoing == "N":
                   ValidUpdation = True
                   return False

               else:
                   print("Invalid input. Please enter Y or N.")

       else:
           ValidUpdation = True
           return True


def AskContinueUpdating():
   while True:
       SameEvent = input("Continue updating this event? (Y/N): ").strip().upper()
       if SameEvent == "Y":
           return "same"
       elif SameEvent == "N":
           while True:
               AnotherEvent = input("Update another event? (Y/N): ").strip().upper()
```

```
            if AnotherEvent == "Y":
                return "another"

            elif AnotherEvent == "N":
                return "quit"

            else:
                print("Invalid input. Please enter Y or N.")

        else:
            print("Invalid input. Please enter Y or N.")


#Function to remove all details of an event of given index
def RemoveEvent(Index):
    global NumberEvent
    EventName.pop(Index)
    EventDateYear.pop(Index)
    EventDateDay.pop(Index)
    EventStartTime.pop(Index)
    EventDateMonth.pop(Index)
    EventEndTime.pop(Index)
    EventVenue.pop(Index)
    EventDescription.pop(Index)
    NumberAttendee.pop(Index)
    EventID.pop(Index)
    NumberEvent -= 1


#Function to let user choose the event that is going to be deleted by entering the index
def ChooseDeleteEvent():
    Output()
    ValidEvent = False
    while not ValidEvent:
        EventChoice = input("Which event would you like to delete? (Enter Index): ").strip()
        if EventChoice.isdigit() == True:
            EventChoice = int(EventChoice)-1
            if EventChoice in range(0, NumberEvent):
                ValidEvent = True

            else:
                print("Invalid index.")

        else:
            print("Input must be an integer.")

    return EventChoice


#Function to choose and delete details of an event
def DeleteEvent():
    DeletingEvent = True
    while DeletingEvent:
        if NumberEvent == 0:
            print("There are no events.\n")
            return

        else:
            EventChoice = ChooseDeleteEvent() #Get the index of event that is being deleted
            RemoveEvent(EventChoice)
            #Ask user whether to continue deleting event
            ValidInput = False
            while not ValidInput:
```

```
                ContinueDelete = input("Do you want to continue deleting events? (Y/N):
").strip().upper()
                if ContinueDelete == "Y":
                    print("Continue to delete events.")
                    ValidInput = True

                elif ContinueDelete == "N":
                    print("Exiting.")
                    DeletingEvent = False
                    ValidInput = True

                else:
                    print("Invalid input.")


#Function to check whether a clash with time and venue occur, (Taking Index of event as parameter
and returning a boolean, where True when a clash occurs)
def CheckClashes(Index):
    for Counter in range(0,NumberEvent):
        if Index == Counter:
            continue

        else:
            if EventVenue[Counter] == EventVenue[Index] and EventDateYear[Counter] ==
EventDateYear[Index] and EventDateMonth[Counter] == EventDateMonth[Index] and
EventDateDay[Counter] == EventDateDay[Index] and EventStartTime[Index] < EventEndTime[Counter]
and EventEndTime[Index] > EventStartTime[Counter]:
                print("Clash event detected.\nPlease reenter.")
                return True

    return False


def Output():
    """
    Purpose: Print a formatted table of all events and their details, including registered
attendees.
    Usage: Called by Main and ChooseEvent().
    """

    # Check if there are any events to display
    if not EventName:
        print(f"\n{"=" * 50}\nNo events found in the system.\n{"=" * 50}")
        return


    print(f"\n{"=" * 90}\n{'EVENT MANAGEMENT SYSTEM - SCHEDULE':^90}\n{"=" * 90}")


    # Loop through each event using index matching
    for i in range(len(EventName)):
        print(f"[Event Index: {i+1}]")
        print(f"Event Name: {EventName[i]}")
        print("-" * 50)
        # Formats single-digit days/months with leading zeros (e.g., 05/09/2026)
        print(f"Date: {EventDateDay[i]:02d}/{EventDateMonth[i]:02d}/{EventDateYear[i]:04d}")
        # Formats military time values cleanly (e.g., 0900 - 1700)
        print(f"Time: {EventStartTime[i]:04d} to {EventEndTime[i]:04d}")
        print(f"Venue: {EventVenue[i]}")
        print(f"Description: {EventDescription[i]}")
        print(f"Total Registered Attendees: {NumberAttendee[i]}")
        print("Registered Attendees List:")

        # Check if the nested list has elements for the current event index
        if EventID[i].strip() != "":
```

```
            print(f"{'No.':<5} {'ID':<5} {'Attendee Name':<25} {'Attendee Details'}")
            print("  " + "-" * 60)
            IDList = EventID[i].split()
            for j in range(len(IDList)):
                ID = IDList[j]
                name = AttendeeName[AttendeeID.index(int(ID))]
                details = AttendeeDetails[AttendeeID.index(int(ID))]
                print(f"{j + 1:<5} {ID:<5} {name:<25} {details}")

        else:
            print("No attendees registered yet.")

        print("=" * 90)


#Show all events with respective indexes then get the index of event and return the index
def ChooseEvent():
    Output()
    ValidEvent = False
    while not ValidEvent:
        EventChoice = input("Which event would you like to edit? (Enter Index): ")
        if EventChoice.isdigit():
            EventChoice = int(EventChoice)-1
            if EventChoice in range(NumberEvent):
                ValidEvent = True

            else:
                print("Invalid index.")

        else:
            print("Input must be an integer.")

    return EventChoice


#Show all attendees with respective IDs then get the ID of attendee and return the ID
def ChooseAttendee(Prompt):
    ValidID = False
    while not ValidID:
        ShowAttendee()
        IDChoice = input(f"{Prompt}").strip()
        if IDChoice.isdigit():
            IDChoice = int(IDChoice)
            if IDChoice in AttendeeID:
                ValidID = True
                return IDChoice

            else:
                print("Invalid index.")

        else:
            print("Input must be an integer.")


#Check for any clashes by taking in the ID of Attendee and Index of Event Attendee is registering
for, return True when there is clash event
def CheckClashAttendee(ID,EventIndex):
    for IDIndex in range(len(EventID)):
        for IDEvent in EventID[IDIndex].split(): #IDEvent is list of IDs as string in an Event
            if IDEvent == str(ID):
                if EventIndex == IDIndex: #Check whether ID is already in event
                    print("Attendee is already registered.")
                    return True
```

```
                if EventDateYear[EventIndex] == EventDateYear[IDIndex] and
EventDateMonth[EventIndex] == EventDateMonth[IDIndex] and EventDateDay[EventIndex] ==
EventDateDay[
                    IDIndex] and EventStartTime[IDIndex] < EventEndTime[EventIndex] and
EventEndTime[IDIndex] > EventStartTime[
                    EventIndex]: #Check for clashes
                    print("Clash event detected.")
                    return True

    return False


#Registering an Attendee
def RegisterAttendee():
    global AttendeeCounter
    Index = ChooseEvent() #Let user choose the event they will be registered in


    #Check whether the attendee is a new user
    NewUser = "K"
    while NewUser.upper() != "N" and NewUser.upper() != "Y":
        if len(AttendeeID) != 0: #Go to registering new user if there are no attendee
            NewUser = input("Are you a new user? (Y/N): ").strip().upper()

        if NewUser.upper() == "Y" or len(AttendeeID) == 0:
            NewName = ""
            while NewName == "" or len(NewName) > 25:
                NewName = input("Enter Attendee Name: ").strip() #Get name of user
                if NewName == "" or len(NewName) > 25:
                    print("Invalid name.")
            AttendeeName.append(NewName)
            NewDetails = "-" * 16
            while len(NewDetails) > 15:
                NewDetails = input("Enter Attendee Details: ").strip() #Get details of user
                if len(NewDetails) > 15:
                    print("Details cannot be more than 15 characters.")
            if NewDetails == "":
                NewDetails = "-"
            AttendeeDetails.append(NewDetails) #Add the detail
            AttendeeID.append(AttendeeCounter) #Add a new ID
            print(f"Your ID is {AttendeeID[AttendeeID.index(AttendeeCounter)]}") #Show the ID of
user
            EventID[Index] = f"{EventID[Index]}{AttendeeCounter} " #Add ID to the Event
            AttendeeCounter += 1
            NumberAttendee[Index] += 1
            print("Attendee registered successfully.")
            break

        elif NewUser.upper() == "N":
            ValidID = False
            while not ValidID:
                OldID = input("Enter your ID: ").strip() #Get ID of user
                if OldID.isdigit():
                    OldID = int(OldID)
                    if OldID in AttendeeID: #Find name of user
                        print(f"ID found, with name: {AttendeeName[AttendeeID.index(OldID)]}.")
                        if CheckClashAttendee(OldID,Index): #Check whether attendee have clashing
event
                            print("Attendee is not added.")

                        else:
                            EventID[Index] = f"{EventID[Index]}{OldID} " #Add ID to the Event
                            NumberAttendee[Index] += 1
                            print("Attendee registered successfully.")
```

```
                    ValidID = True

                else:
                    print("ID not found. ")

            else:
                print("ID can only be integers. ")

        else:
            print("Invalid Input. Please enter Y or N.")


#Updating Attendee Name or Details
def UpdateAttendee():
    if len(AttendeeID) == 0:
        print("No attendees have been registered.")
        return


    #Get attendee's ID
    IDChoice = ChooseAttendee("Attendee ID to be updated: ")


    #Get what user wants to update
    ValidOption = False
    while not ValidOption:
        Option = input("Update Attendee\'s\n1 | Name\n2 | Details\n:")
        if Option == "1":
            #Get the new name
            NewName = ""
            while NewName.strip() == "":
                NewName = input("Enter Attendee New Name: ").strip()
                if NewName == "":
                    print("Invalid name.")


            AttendeeName[AttendeeID.index(IDChoice)] = NewName
            print("Name has been updated.")
            ValidOption = True
            break


        elif Option == "2":
            #Get the new details
            NewDetails = input("Enter Attendee New Details: ").strip()
            AttendeeDetails[AttendeeID.index(IDChoice)] = NewDetails
            if AttendeeDetails[AttendeeID.index(IDChoice)] == "":
                AttendeeDetails[AttendeeID.index(IDChoice)] = "-"

            print("Details have been updated.")
            ValidOption = True
            break


        else:
            print("Invalid input.")


#Deleting attendee or registered event from attendee
def DeleteAttendee():
    global AttendeeCounter
    if len(AttendeeID) == 0: #Check whether there is attendee
        print("No attendees have been registered.")
        return
```

```
   #Get whether user wants to delete the all information of on attendee or registration of an
attendee
   DeletingAttendee = True
   while DeletingAttendee:
       DeleteChoice = input("Do you want to delete an attendee or registration of an attendee?\n1
| Delete Attendee\n2 | Delete Registration\n: ").strip()
       if DeleteChoice == "1": #Delete all information of an attendee
           #Get attendee's ID
           IDChoice = ChooseAttendee("Attendee ID to be deleted: ")


           #Remove all instances of attendee
           for IDIndex in range(len(EventID)):
               for IDEvent in EventID[IDIndex].split():
                   if IDEvent == str(IDChoice):
                       IDList = EventID[IDIndex].split()
                       if str(IDChoice) in IDList:
                           IDList.remove(str(IDChoice))
                           if len(IDList) > 0:
                               EventID[IDIndex] = " ".join(IDList) + " "
                           else:
                               EventID[IDIndex] = ""
                       NumberAttendee[IDIndex] -= 1


           AttendeeName.pop(AttendeeID.index(IDChoice))
           AttendeeDetails.pop(AttendeeID.index(IDChoice))
           AttendeeID.remove(IDChoice)
           DeletingAttendee = False
           break



       elif DeleteChoice == "2": #Delete registration of an attendee
           #Get attendee's ID
           IDChoice = ChooseAttendee("Attendee ID to be deleted from an event: ")


           #Display all events that current attendee is registered in
           IndexList = []

print(f"{"Index":<5}|{"Name":<15}|{"Date":<15}|{"Time":<15}|{"Venue":<15}|{"Description":<30}|Num
ber of Attendee")
           for IDIndex in range(len(EventID)):
               for IDEvent in EventID[IDIndex].split():
                   if IDEvent == str(IDChoice):
                       FormattedDate =
f"{EventDateDay[IDIndex]:02d}/{EventDateMonth[IDIndex]:02d}/{EventDateYear[IDIndex]:04d}"
                       FormattedTime = f"{EventStartTime[IDIndex]:04d} -
{EventEndTime[IDIndex]:04d}"
                       print(f"{IDIndex +
1:<5}|{EventName[IDIndex]:<15}|{FormattedDate:<15}|{FormattedTime:<15}|{EventVenue[IDIndex]:<15}|
{EventDescription[IDIndex]:<30}|{NumberAttendee[IDIndex]}")
                       IndexList.append(IDIndex)


           #Get the event to be removed from
           ValidIndex = False
           while not ValidIndex:
               IndexChoice = input(f"Which event do you want to remove
{AttendeeName[AttendeeID.index(IDChoice)]} from? (Enter index):").strip()
               if IndexChoice.isdigit():
                   IndexChoice = int(IndexChoice) - 1
                   if IndexChoice in IndexList:
```

```
                    ValidIndex = True
                    break

                else:
                    print("Invalid index.")

            else:
                print("Input must be an integer.")


        #Remove attendee's ID from event
        print(f"{AttendeeName[AttendeeID.index(IDChoice)]}\'s registration in
{EventName[IndexChoice]} has been removed.")
        IDList = EventID[IndexChoice].split()
        if str(IDChoice) in IDList:
            IDList.remove(str(IDChoice))
            if len(IDList) > 0:
                EventID[IndexChoice] = " ".join(IDList) + " "
            else:
                EventID[IndexChoice] = ""
        NumberAttendee[IndexChoice] -= 1
        DeletingAttendee = False
        break

    else:
        print("Input must be either \"1\" or \"2\"")


def ShowAttendee():
   print(f"{"+" * 30}\n{"Attendee List":^30}\n{"+" * 30}")


   if len(AttendeeID) == 0: #Check whether there is attendee
       print("There are no attendee.\n")
       return


   print(f"{"ID":<5}|{"Name":<25}|{"Details":<15}|Events Registered")


   #Display attendees' ID, Name, Details and Registered Events
   for ID in AttendeeID:
       AttendeeIndex = AttendeeID.index(ID)
       AttendeeEvents = ""
       for IDIndex in range(len(EventID)):
           for IDEvent in EventID[IDIndex].split():
               if IDEvent == str(ID):
                   AttendeeEvents = AttendeeEvents + EventName[IDIndex] + ", "


       if AttendeeEvents == "":
           AttendeeEvents = "-  "

print(f"{ID:<5}|{AttendeeName[AttendeeIndex]:<25}|{AttendeeDetails[AttendeeIndex]:<15}|{AttendeeE
vents[0:len(AttendeeEvents)-2]}")


#Display menu of actions that changes information of attendees
def AttendeeAction():
   if len(EventName) == 0: #Check whether there are events
       print("There are no events.\n")
       return


   Run = True
   while Run:
```

```
        print(f"\n{"*" * 30}\n{"Attendee Menu":^30}\n{"*" * 30}")
        print("1. Register Attendee")
        print("2. Update Attendee")
        print("3. Delete Attendee")
        print("4. Show Attendee")
        print("5. Go Back")
        Choice = input("Enter your choice: (1-5) ").strip()
        if Choice == "1":
            RegisterAttendee()


        elif Choice == "2":
            UpdateAttendee()


        elif Choice == "3":
            DeleteAttendee()


        elif Choice == "4":
            ShowAttendee()


        elif Choice == "5":
            Run = False


        else:
            print("Invalid input.")


#Main Interface
Run = True
while Run:
   ValidAction = False
   while not ValidAction:
        print(f"{"=" * 30}\n{"Main Interface":^30}\n{"=" * 30}")
        print("1. Add event")
        print("2. Update event details")
        print("3. Delete event")
        print("4. Register attendees")
        print("5. Display output")
        print("6. Quit")
        Action = input("What would you like to do? (1-6): ").strip()
        if Action != "":
            ValidAction = True
            if Action == "1":
                AddEvent()


            elif Action == "2":
                UpdateEvent()


            elif Action == "3":
                DeleteEvent()


            elif Action == "4":
                AttendeeAction()


            elif Action == "5":
                Output()


            elif Action == "6":
```

```
                print("Thanks for using the system. Now displaying the final event output.\n")
                Run = False
                break


            else:
                ValidAction = False
                print("Invalid input. Please select an option from 1-6.\n")


        else:
            print("Input cannot be empty.\n")


Output()


#Fill in empty strings to prevent bugs when read
for Index in range(NumberEvent):
   if EventDescription[Index] == "":
       EventDescription[Index] = "No description"
for ID in range(len(AttendeeID)):
   if AttendeeDetails[ID] == "":
       AttendeeDetails[ID] = "No details"


#Write information into "Events_Information.txt"
File = open("Events_Information.txt","w")
File.write((Separator.join(EventName).strip("\n")) + "\n")
File.write(" ".join(FormatToString(EventDateYear)) + "\n")
File.write(" ".join(FormatToString(EventDateMonth)) + "\n")
File.write(" ".join(FormatToString(EventDateDay)) + "\n")
File.write(" ".join(FormatToString(EventStartTime)) + "\n")
File.write(" ".join(FormatToString(EventEndTime)) + "\n")
File.write((Separator.join(EventVenue).strip("\n")) + "\n")
File.write((Separator.join(EventDescription).strip("\n")) + "\n")
File.write(" ".join(FormatToString(NumberAttendee)) + "\n")
File.write(" ".join(FormatToString(AttendeeID)) + "\n")
File.write((Separator.join(AttendeeName).strip("\n")) + "\n")
File.write((Separator.join(AttendeeDetails).strip("\n")) + "\n")
File.write((Separator.join(EventID).strip("\n")) + "\n")
File.write(str(NumberEvent) + "\n")
File.write(str(AttendeeCounter) + "\n")
File.close()
```

Limitations

1. Length of attendee name cannot be greater than 25 characters
2. Length of attendee details cannot be greater than 15 characters
3. Length of event name cannot be greater than 15 characters
4. Length of event venue cannot be greater than 15 characters
5. Length of event description cannot be greater than 30 characters

Testing

Test Data

No.	Test Case	Output

0	Initial Data	“Events_Information.txt”:
		0
		0
		IDLE:
		`==========`
		`Main Interface`
		`==========`
		`1. Add event`
		`2. Update event details`
		`3. Delete event`
		`4. Register attendees`
		`5. Display output`
		`6. Quit`
		`What would you like to do? (1-6):`

1	Invalid options: 0 (Invalid) a (Invalid)	What would you like to do? (1-6): 0 Invalid input. Please select an option from 1-6. ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6): a Invalid input. Please select an option from 1-6. ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6):
2	Valid options but are invalid when there is no event: 2 (Invalid) 3 (Invalid) 4 (Invalid)	What would you like to do? (1-6): 2 There are no events. Please type 1 in the interface to add an event first. ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6): 3 There are no events. ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6): 4 There are no events. ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6):

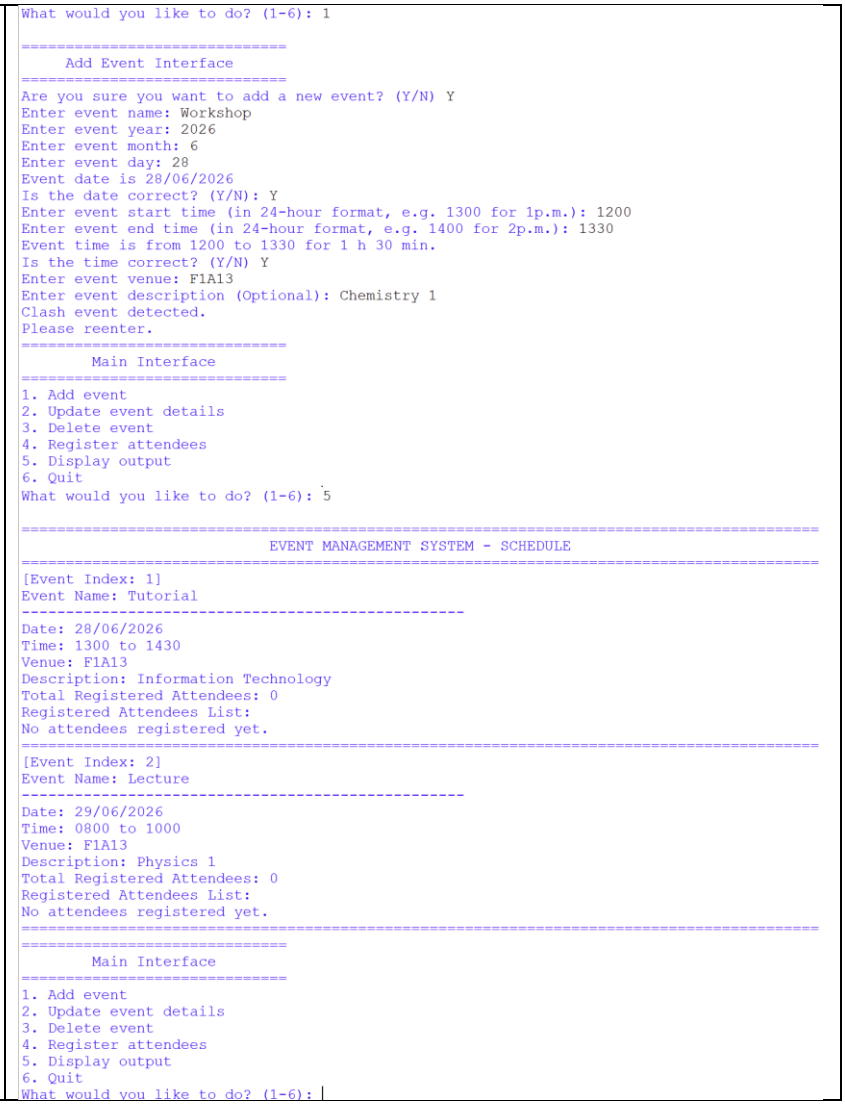
3	Enter add events: 1 (Valid) a (Invalid) N (Valid) 1 (Valid) Y (Valid) Lecture (Valid) 10000 (Invalid) 2026 (Valid) 2 (Valid) 29 (Invalid) 28 (Valid Day) N (Valid) 2026 (Valid) 13 (Invalid) 0 (Invalid) 12 (Valid) 32 (Invalid) 31 (Valid) N (Valid) 2026 (Valid) 6 (Valid) 31 (Invalid) 30 (Valid) Y (Valid) 1200 (Valid) 1000 (Invalid) 1060 (Invalid) 1059 (Valid) 2400 (Invalid) 2359 (Valid) Y (Valid) F1A13 (Valid) Physics 2 (Valid)	

4	Enter update events: 2 (Valid) y (Valid) 0 (Invalid) 2 (Invalid) a (Invalid) 1 (Valid) 1 (Valid) Tutorial (Valid) Y (Valid) 2 (Valid) 2024 (Valid) 2 (Valid) 29 (Valid) y (Valid) Y (Valid)	

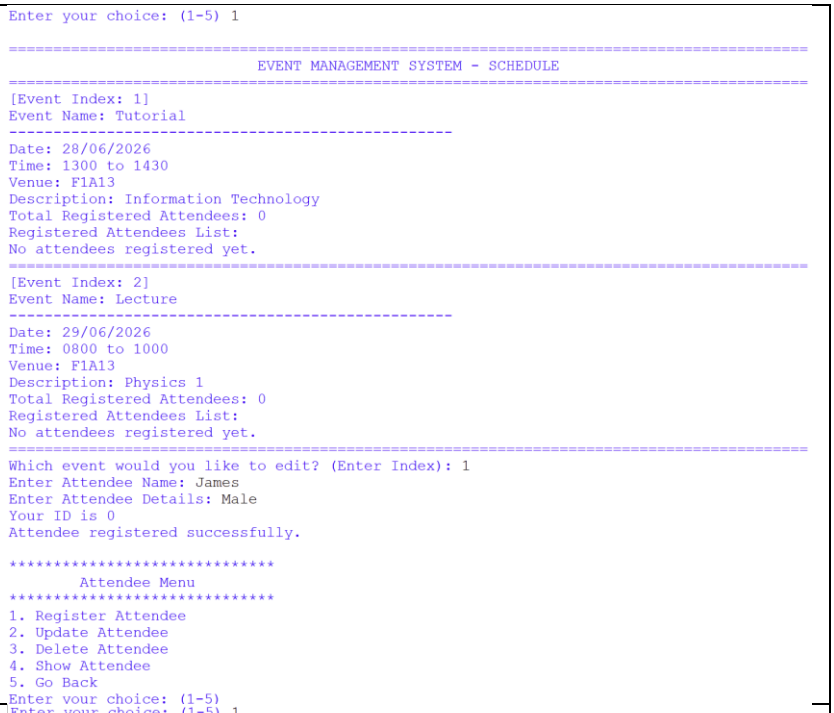
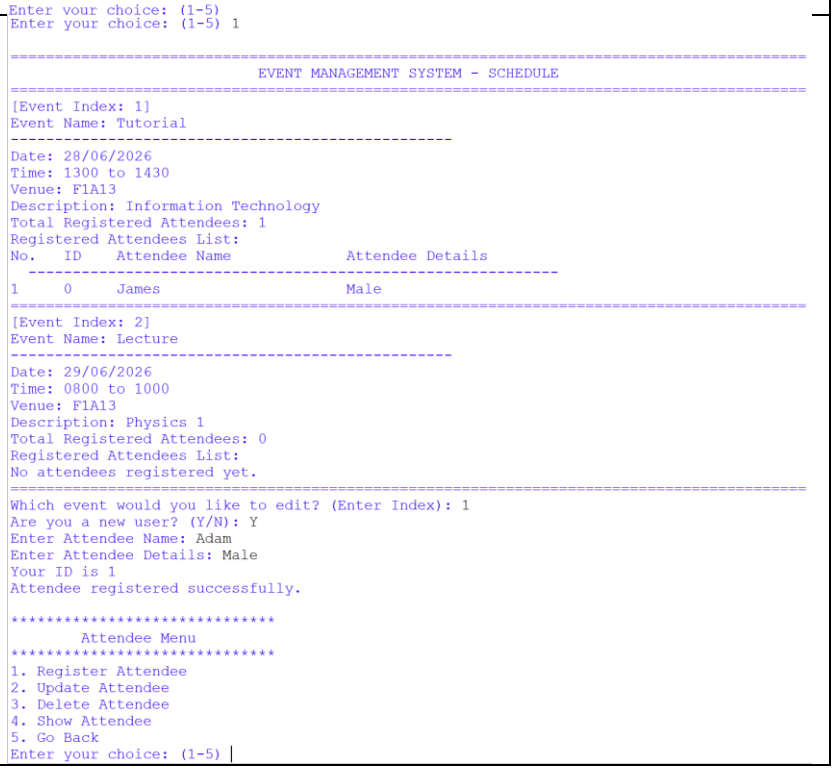
	3 (Valid) F4B09 (Valid) y (Valid) 5 (Valid) y (Valid) 6 (Valid) Workshop (Valid) 2026 (Valid) 2 (Valid) 28 (Valid) y (Valid) F1A13 (Valid) 0900 (Valid) 1100 (Valid) Y (Valid) Mathematics (Valid) N (Valid) n (Valid)	1. Name: Tutorial 2. Date: 29/2/2024 3. Venue: F1A13 4. Time: 1059-2359 5. Description: Physics 2 6. Update all 7. Back Select option to update (1-7): 3 Enter event venue: F4B09 Venue updated. Continue updating this event? (Y/N): y 1. Name: Tutorial 2. Date: 29/2/2024 3. Venue: F4B09 4. Time: 1059-2359 5. Description: Physics 2 6. Update all 7. Back Select option to update (1-7): 5 Enter event description (Optional): Description updated. Continue updating this event? (Y/N): y 1. Name: Tutorial 2. Date: 29/2/2024 3. Venue: F4B09 4. Time: 1059-2359 5. Description: - 6. Update all 7. Back Select option to update (1-7): 6 Enter event name: Workshop Enter event year: 2026 Enter event month: 2 Enter event day: 28 Event date is 28/02/2026 Is the date correct? (Y/N): y Enter event venue: F1A13 Enter event start time (in 24-hour format, e.g. 1300 for 1p.m.): 0900 Enter event end time (in 24-hour format, e.g. 1400 for 2p.m.): 1100 Event time is from 0900 to 1100 for 2 h 0 min. Is the time correct? (Y/N) Y Enter event description (Optional): Mathematics Event updated. Continue updating this event? (Y/N): N Update another event? (Y/N): n ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6):
5	Enter delete event: 3 (Valid) 0 (Invalid) 2 (Invalid) 1 (Valid) N (Valid)	What would you like to do? (1-6): 3 ========== EVENT MANAGEMENT SYSTEM - SCHEDULE ========== [Event Index: 1] Event Name: Workshop ---------- Date: 28/02/2026 Time: 0900 to 1100 Venue: F1A13 Description: Mathematics Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== Which event would you like to delete? (Enter Index): 0 Invalid index. Which event would you like to delete? (Enter Index): 2 Invalid index. Which event would you like to delete? (Enter Index): 1 Do you want to continue deleting events? (Y/N): N Exiting. ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6):

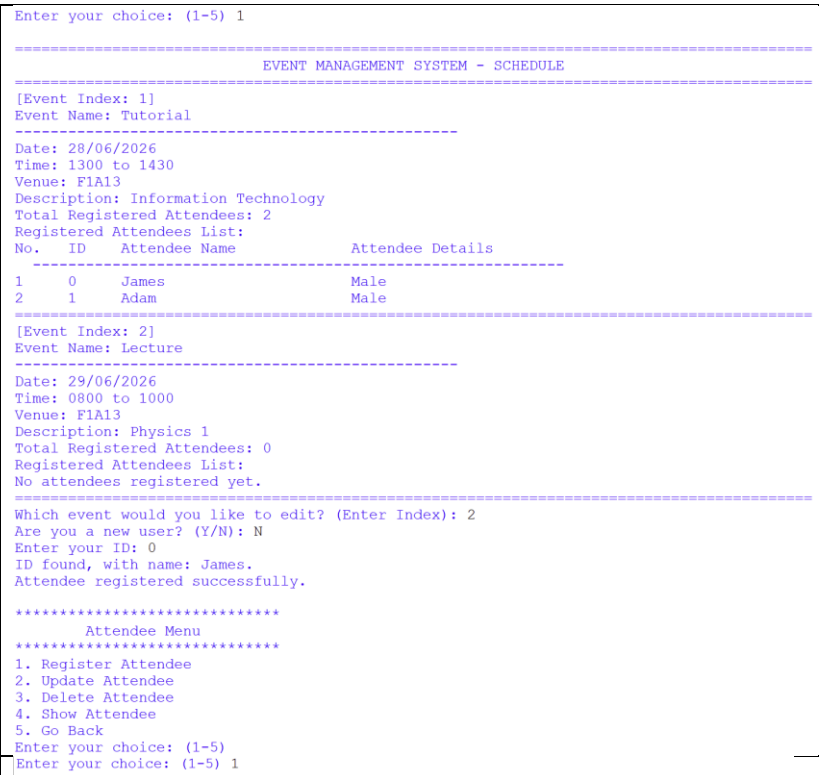
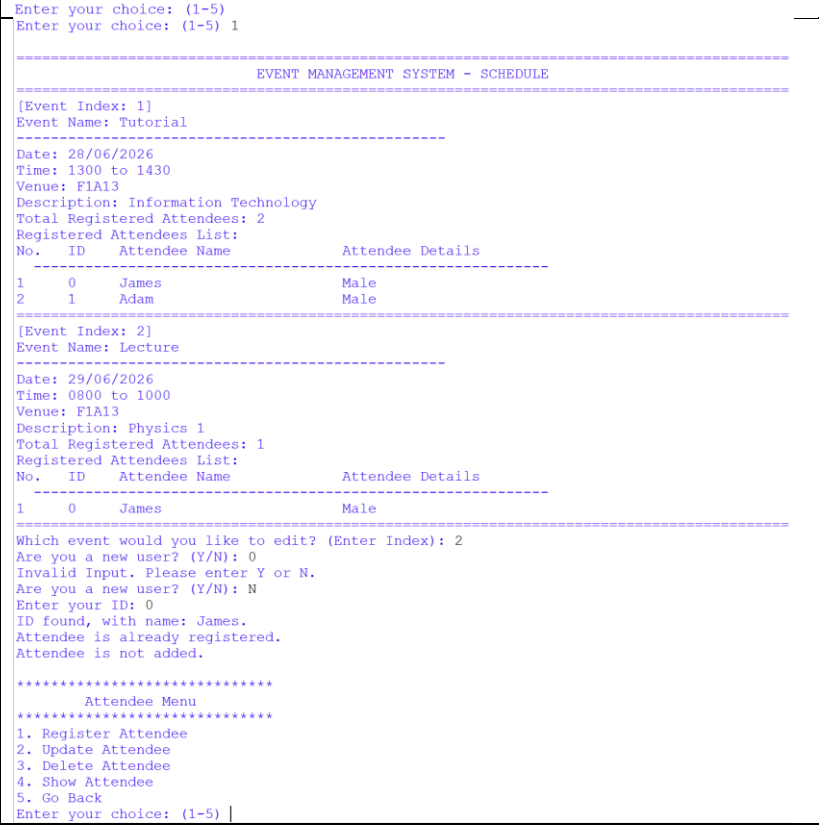
6	Creating two events: 1 (Valid) Y (Valid) Lecture (Valid) 2026 (Valid) 6 (Valid) 29 (Valid) Y (Valid) 0800 (Valid) 1000 (Valid) Y (Valid) F1A13 (Valid) Physics 1 (Valid) 1 (Valid) Y (Valid) Tutorial (Valid) 2026 (Valid) 7 (Valid) 1 (Valid) Y (Valid) 1300 (Valid) 1430 (Valid) Y (Valid) F1A13 (Valid) Information Technology (Valid)	What would you like to do? (1-6): 1 ========== Add Event Interface ========== Are you sure you want to add a new event? (Y/N) Y Enter event name: Lecture Enter event year: 2026 Enter event month: 6 Enter event day: 29 Event date is 29/06/2026 Is the date correct? (Y/N): Y Enter event start time (in 24-hour format, e.g. 1300 for 1p.m.): 0800 Enter event end time (in 24-hour format, e.g. 1400 for 2p.m.): 1000 Event time is from 0800 to 1000 for 2 h 0 min. Is the time correct? (Y/N) Y Enter event venue: F1A13 Enter event description (Optional): Physics 1 Event added successfully. ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6): 1 ========== Add Event Interface ========== Are you sure you want to add a new event? (Y/N) Y Enter event name: Tutorial Enter event year: 2026 Enter event month: 7 Enter event day: 1 Event date is 01/07/2026 Is the date correct? (Y/N): Y Enter event start time (in 24-hour format, e.g. 1300 for 1p.m.): 1300 Enter event end time (in 24-hour format, e.g. 1400 for 2p.m.): 1430 Event time is from 1300 to 1430 for 1 h 30 min. Is the time correct? (Y/N) Y Enter event venue: F1A13 Enter event description (Optional): Information Technology Event added successfully. ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6):
7	Display output: 5 (Valid)	What would you like to do? (1-6): 5 ========== EVENT MANAGEMENT SYSTEM - SCHEDULE ========== [Event Index: 1] Event Name: Lecture ---------- Date: 29/06/2026 Time: 0800 to 1000 Venue: F1A13 Description: Physics 1 Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== [Event Index: 2] Event Name: Tutorial ---------- Date: 01/07/2026 Time: 1300 to 1430 Venue: F1A13 Description: Information Technology Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6):

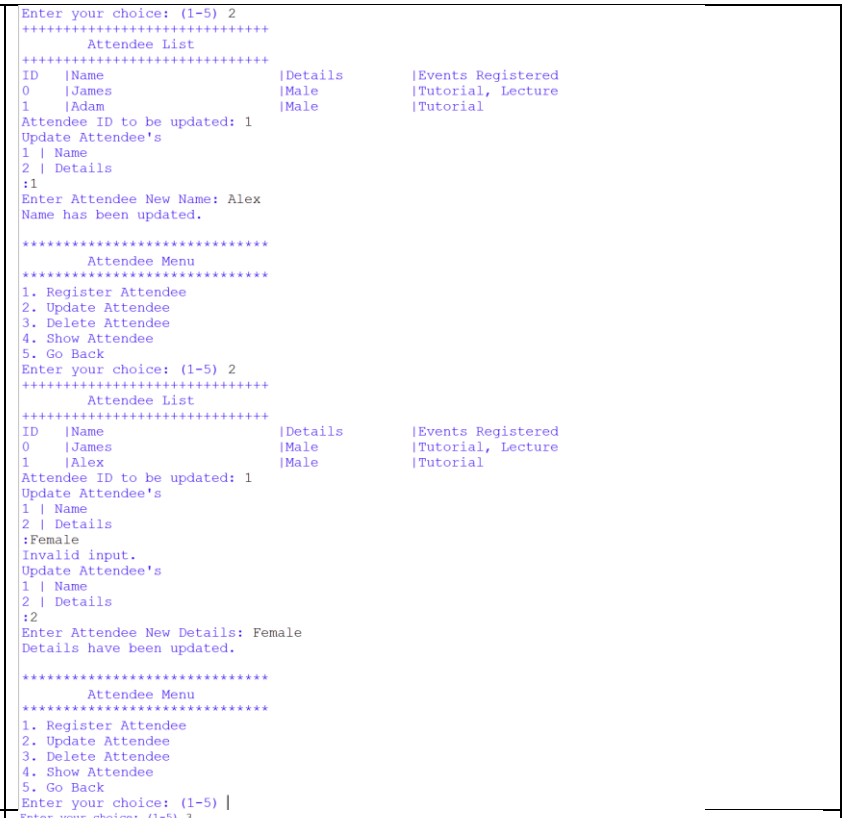
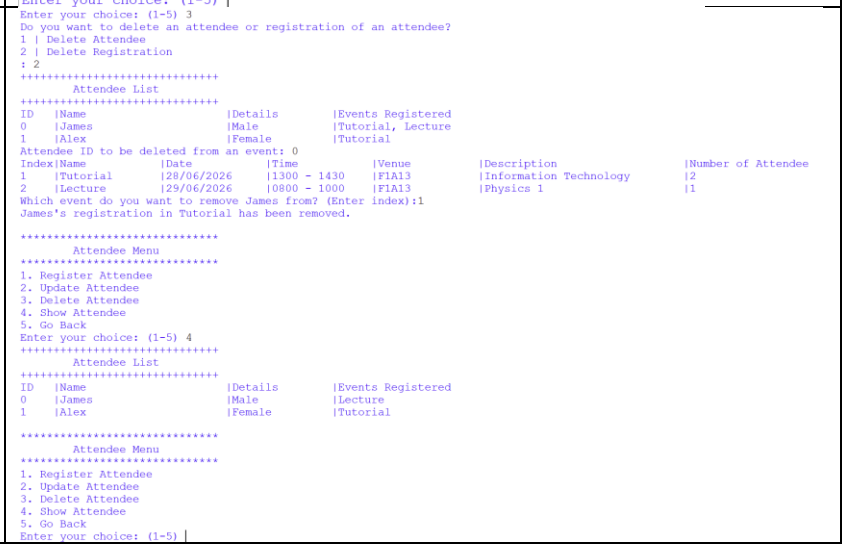
8	Update an event then display events to test Sort by Order: 2 (Valid) Y (Valid) 2 (Valid) 2 (Valid) 2026 (Valid) 6 (Valid) 28 (Valid) Y (Valid) N (Valid) N (Valid) 5 (Valid)	What would you like to do? (1-6): 2 Do you want to update an event? (Y/N): Y ========== EVENT MANAGEMENT SYSTEM - SCHEDULE ========== [Event Index: 1] Event Name: Lecture ---------- Date: 29/06/2026 Time: 0800 to 1000 Venue: F1A13 Description: Physics 1 Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== [Event Index: 2] Event Name: Tutorial ---------- Date: 01/07/2026 Time: 1300 to 1430 Venue: F1A13 Description: Information Technology Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== Which one would you like to update? (Please enter a number from 1 - 2) or B to go back: 2 1. Name: Tutorial 2. Date: 1/7/2026 3. Venue: F1A13 4. Time: 1300-1430 5. Description: Information Technology 6. Update all 7. Back Select option to update (1-7): 2 Enter event year: 2026 Enter event month: 6 Enter event day: 28 Event date is 28/06/2026 Is the date correct? (Y/N): Y Date updated. Continue updating this event? (Y/N): N Update another event? (Y/N): N ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6): 5 ========== EVENT MANAGEMENT SYSTEM - SCHEDULE ========== [Event Index: 1] Event Name: Tutorial ---------- Date: 28/06/2026 Time: 1300 to 1430 Venue: F1A13 Description: Information Technology Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== [Event Index: 2] Event Name: Lecture ---------- Date: 29/06/2026 Time: 0800 to 1000 Venue: F1A13 Description: Physics 1 Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6):

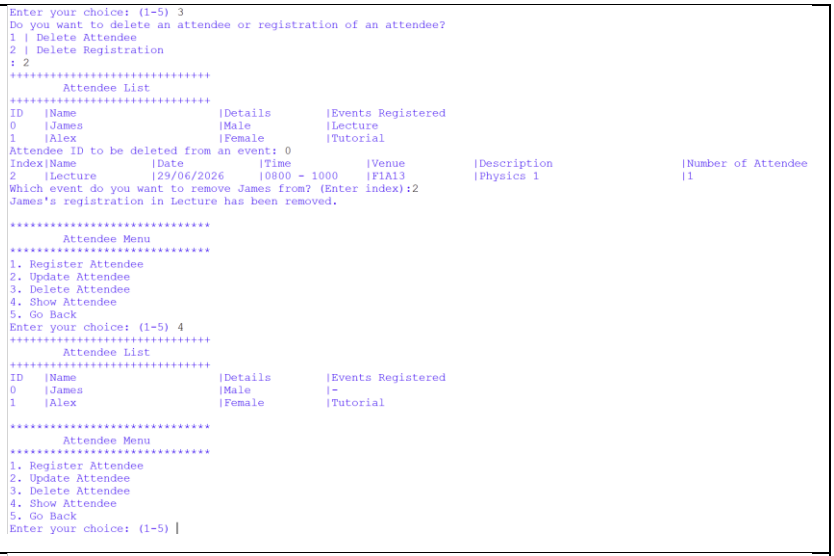
9	Add an event then display events to test Clashes Checking: 1 (Valid) Y (Valid) Workshop (Valid) 2026 (Valid) 6 (Valid) 28 (Valid) Y (Valid) 1200 (Valid) 1330 (Valid) Y (Valid) F1A13 (Valid) Chemistry 1 (Valid) 5 (Valid)	

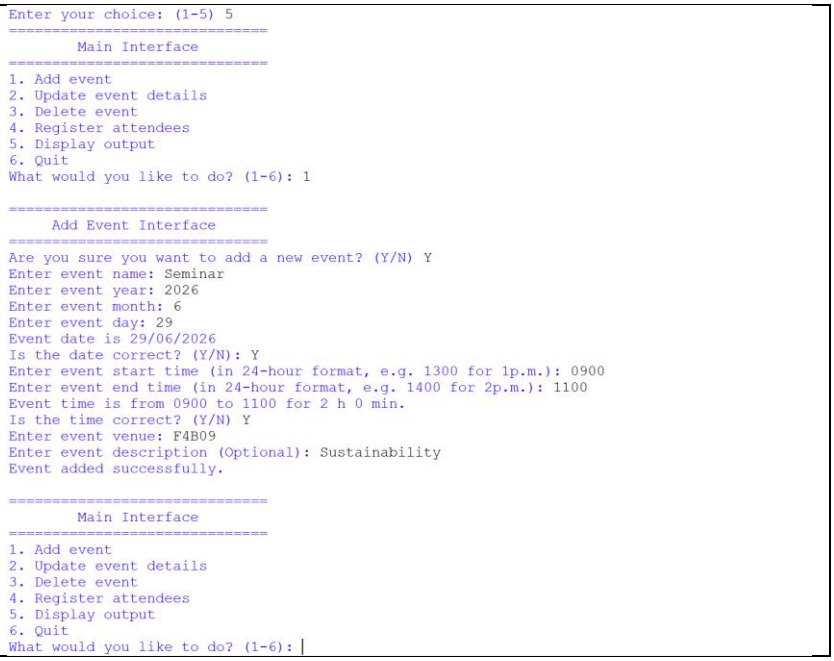
10	Invalid options in attendee management when there is no attendee: 4 (Valid) 2 (Invalid) 3 (Invalid) 4 (Invalid)	What would you like to do? (1-6): 4 ***** Attendee Menu ***** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5) 2 No attendees have been registered. ***** Attendee Menu ***** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5) 3 No attendees have been registered. ***** Attendee Menu ***** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5) 4 ++++++++++++++++++++++++++++ Attendee List ++++++++++++++++++++++++++++ There are no attendee. ***** Attendee Menu ***** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5)
11	Invalid options in Attendee Management: 0 (Invalid) a (Invalid)	Enter your choice: (1-5) 0 Invalid input. ***** Attendee Menu ***** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5) a Invalid input. ***** Attendee Menu ***** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5)

12	Register an Attendee: 1 (Valid) 1 (Valid) James (Valid) Male (Valid)	
13	Register Another Attendee: 1 (Valid) 1 (Valid) Y (Valid) Adam (Valid) Male (Valid)	

14	Register Existing Attendee to Another Event: 1 (Valid) 2 (Valid) N (Valid) 0 (Valid)	
15	Registering Attendee to same event: 1 (Valid) 2 (Valid) 0 (Invalid) N (Valid) 0 (Valid)	

16	Update Attendee Name and Details: 2 (Valid) 1 (Valid) 1 (Valid) Alex (Valid) 2 (Valid) 1 (Valid) Female (Invalid) 2 (Valid) Female (Valid)	
17	Delete Attendee Registration in an event then show attendee list: 3 (Valid) 2 (Valid) 0 (Valid) 1 (Valid) 4 (Valid)	

18	Delete Attendee Registration from an event that leads to attendee with no event registered then show attendee list: 3 (Valid) 2 (Valid) 0 (Valid) 2 (Valid) 4 (Valid)	
19	Delete Attendee with their ID then show attendee list: 3 (Valid) 1 (Valid) 1 (Valid) 4 (Valid)	

20	Add an event which time clashes with existing event: 5 (Valid) 1 (Valid) Y (Valid) Seminar (Valid) 2026 (Valid) 6 (Valid) 29 (Valid) Y (Valid) 0900 (Valid) 1100 (Valid) Y (Valid) F4B09 (Valid) Sustainability (Valid)	

21	Attempt to register an attendee to two clashing events: 4 (Valid) 1 (Valid)

	3 (Valid) N (Valid) 2 (Valid)	[Event Index: 3] Event Name: Seminar -- Date: 29/06/2026 Time: 0900 to 1100 Venue: F4B09 Description: Sustainability Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== Which event would you like to edit? (Enter Index): 3 Are you a new user? (Y/N): N Enter your ID: 2 ID found, with name: Ben. Clash event detected. Attendee is not added. ********** Attendee Menu ********** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5)
22	Register an attendee with name same as an existing attendee then show the attendee list: 1 (Valid) 2 (Valid) Y (Valid) Ben (Valid) Male (Valid) 4 (Valid)	Enter your choice: (1-5) 1 ========== EVENT MANAGEMENT SYSTEM - SCHEDULE ========== [Event Index: 1] Event Name: Tutorial -- Date: 28/06/2026 Time: 1300 to 1430 Venue: F1A13 Description: Information Technology Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== [Event Index: 2] Event Name: Lecture -- Date: 29/06/2026 Time: 0800 to 1000 Venue: F1A13 Description: Physics 1 Total Registered Attendees: 1 Registered Attendees List: No. ID Attendee Name Attendee Details ---------- 1 2 Ben Male ========== [Event Index: 3] Event Name: Seminar -- Date: 29/06/2026 Time: 0900 to 1100 Venue: F4B09 Description: Sustainability Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== Which event would you like to edit? (Enter Index): 2 Are you a new user? (Y/N): Y Enter Attendee Name: Ben Enter Attendee Details: Male Your ID is 3 Attendee registered successfully. ********** Attendee Menu ********** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5) 4 ++++++++++ Attendee List ++++++++++ ID |Name |Details |Events Registered 0 |James |Male |- 2 |Ben |Male |Lecture 3 |Ben |Male |Lecture ********** Attendee Menu ********** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5)
23	Exit the program: 5 (Valid)	“Events_Information.txt”:

	6 (Valid)	1 Tutorial-Lecture-Seminar 2 2026 2026 2026 3 6 6 6 4 28 29 29 5 1300 800 900 6 1430 1000 1100 7 F1A13-F1A13-F4B09 8 Information Technology-Physics 1-Sustainability 9 0 2 0 10 0 2 3 11 James-Ben-Ben 12 Male-Male-Male 13 -2 3 - 14 3 15 4 16 IDLE: Enter your choice: (1-5) 5 ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6): 6 Thanks for using the system. Now displaying the final event output. ========== EVENT MANAGEMENT SYSTEM - SCHEDULE ========== [Event Index: 1] Event Name: Tutorial ---------- Date: 28/06/2026 Time: 1300 to 1430 Venue: F1A13 Description: Information Technology Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== [Event Index: 2] Event Name: Lecture ---------- Date: 29/06/2026 Time: 0800 to 1000 Venue: F1A13 Description: Physics 1 Total Registered Attendees: 2 Registered Attendees List: No. ID Attendee Name Attendee Details ---------- 1 2 Ben Male 2 3 Ben Male ========== [Event Index: 3] Event Name: Seminar ---------- Date: 29/06/2026 Time: 0900 to 1100 Venue: F4B09 Description: Sustainability Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ==========

24	Rerun the program and display the events: 5 (Valid)	

25	Delete all events and attendees: 4 (Valid) 3 (Valid) 1 (Valid) 0 (Valid) 3 (Valid) 1 (Valid) 2 (Valid)	What would you like to do? (1-6): 4 ***************************** Attendee Menu ***************************** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5) 3 Do you want to delete an attendee or registration of an attendee? 1 | Delete Attendee 2 | Delete Registration : 1 +++++++++++++++++++++++++++++ Attendee List +++++++++++++++++++++++++++++ ID |Name |Details |Events Registered 0 |James |Male |- 2 |Ben |Male |Lecture 3 |Ben |Male |Lecture Attendee ID to be deleted: 0 ***************************** Attendee Menu ***************************** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5) 3 Do you want to delete an attendee or registration of an attendee? 1 | Delete Attendee 2 | Delete Registration : 1 +++++++++++++++++++++++++++++ Attendee List +++++++++++++++++++++++++++++ ID |Name |Details |Events Registered 2 |Ben |Male |Lecture 3 |Ben |Male |Lecture Attendee ID to be deleted: 2 ***************************** Attendee Menu ***************************** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5) 3 Do you want to delete an attendee or registration of an attendee? 1 | Delete Attendee 2 | Delete Registration : 1 +++++++++++++++++++++++++++++ Attendee List +++++++++++++++++++++++++++++ ID |Name |Details |Events Registered 3 |Ben |Male |Lecture Attendee ID to be deleted: 3 ***************************** Attendee Menu ***************************** 1. Register Attendee 2. Update Attendee 3. Delete Attendee 4. Show Attendee 5. Go Back Enter your choice: (1-5) 5 ============================== Main Interface ============================== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6): 3 ========== EVENT MANAGEMENT SYSTEM - SCHEDULE ========== [Event Index: 1] Event Name: Tutorial ----------

	3 (Valid) 1 (Valid) 3 (Valid) 5 (Valid) 3 (Valid) 1 (Valid) Y (Valid)	Date: 28/06/2026 Time: 1300 to 1430 Venue: F1A13 Description: Information Technology Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== [Event Index: 2] Event Name: Lecture ---------- Date: 29/06/2026 Time: 0800 to 1000 Venue: F1A13 Description: Physics 1 Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== [Event Index: 3] Event Name: Seminar ---------- Date: 29/06/2026 Time: 0900 to 1100 Venue: F4B09 Description: Sustainability Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== Which event would you like to delete? (Enter Index): 1 Do you want to continue deleting events? (Y/N): Y Continue to delete events. ========== EVENT MANAGEMENT SYSTEM - SCHEDULE ========== [Event Index: 1] Event Name: Lecture ---------- Date: 29/06/2026 Time: 0800 to 1000 Venue: F1A13 Description: Physics 1 Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== [Event Index: 2] Event Name: Seminar ---------- Date: 29/06/2026 Time: 0900 to 1100 Venue: F4B09 Description: Sustainability Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== Which event would you like to delete? (Enter Index): 1 Do you want to continue deleting events? (Y/N): Y Continue to delete events. ========== EVENT MANAGEMENT SYSTEM - SCHEDULE ========== [Event Index: 1] Event Name: Seminar ---------- Date: 29/06/2026 Time: 0900 to 1100 Venue: F4B09 Description: Sustainability Total Registered Attendees: 0 Registered Attendees List: No attendees registered yet. ========== Which event would you like to delete? (Enter Index): 1 Do you want to continue deleting events? (Y/N): N Exiting. ========== Main Interface ========== 1. Add event 2. Update event details 3. Delete event 4. Register attendees 5. Display output 6. Quit What would you like to do? (1-6):

	1 (Valid) Y (Valid) 1 (Valid) N (Valid)	
26	Exit program: 6 (Valid)	“Events_Information.txt”: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 0 15 4 16 IDLE: What would you like to do? (1-6): 6 Thanks for using the system. Now displaying the final event output. ========== No events found in the system. ==========

Artificial Intelligence (AI) Usage Declaration

Declaration Item	Student Response
Did you use any AI tools for this assignment?	**Yes**
Name of AI tool(s) used	ChatGPT, Claude, and Gemini
Purpose of usage	Bugs checking Grammar checking
Sections/code influenced by AI tools	Bugs checking for entire code Grammar checking for documentation
I (group leader) confirm that the submitted work is our own and we understand the code implemented in this assignment.	Signature: